Eccentric Conical Bearing

Complete engineering notebook and OpenFOAM CFD handoff

29 July 2026

Contents

Reader guide

This volume contains all 55 project Org documents under `docs/`, including the complete OpenFOAM handoff series. Links between included documents have been converted to internal PDF destinations.

Overview

Eccentric Conical Bearing — Engineering Notebook

Source: `docs/index.org` *Programmatic CAD, mesh construction, solver transfer, and OpenFOAM evidence Updated: [2026-07-29 Wed]*

Purpose

This notebook explains how the geometry and meshes are constructed, transferred, and used in the downstream OpenFOAM study. It assumes familiarity with Python and basic journal-bearing physics; it concentrates on programmatic B-rep modelling, mesh topology, data exchange, evidence boundaries, and solver handoff.

Current state

Stage	State	Meaning
Native build123d/OCCT CAD	geometry PASS	The B-rep is valid, manifold, connected, and analytically consistent.
STEP exchange	strict audit FAIL	Re-imported port-intersection volume exceeds the project's `1e-6` relative regression gate.
Native BREP into Gmsh OCC	PASS	Three zones, shared interfaces, dimensions, and SI scaling survive.
Boundary surface smoke mesh	PASS	A conformal first-order Tri3 skin can be generated and classified.
No-port analytic Hex8 mesh	PASS	A full-360 structured reference mesh passes topology and round-trip checks.
Surface-inlet analytic Hex8 mesh	PASS	Clustered film-only Hex8 mesh; near-circular pressure patch and no feed column.
Central-feed layered Prism6 mesh	PASS	Film and tube are one conformal fluid mesh with an open internal mouth.
Clean MSH/CGNS interchange	static PASS	Canonical arrays survive MSH 4.1, MSH 2.2, and CGNS round trips with exact zones and scale.
Canonical Fluent import audit	not accepted	The earlier S100 solve diverged; it is evidence, not `FLUENT_IMPORT_PASS`.
OpenFOAM 14 source build	accepted	The solver and conversion tools were built and run locally.
S100 OpenFOAM mesh import	accepted	`gmshToFoam`, patch correction, and standard `checkMesh` accepted 294,912 Hex8 cells.
Zero-flow solver proof	accepted	Fields, patches, numerics, and solver plumbing are consistent.
Pressure-fed 0-rpm case	accepted	A stationary 0.5 MPa commissioning case converges; it is not pure hydrodynamic operation.
200-rpm liquid-only diagnostic	rejected	It drives pressure below absolute vacuum and demonstrates missing rupture physics.

Stage	State	Meaning
Zero-bias liquid continuation	accepted to 20 rpm	The 0, 1, 5, 10, 12, 14, 15, and 20-rpm checkpoints remain bounded and single-liquid.
First 20-to-50-rpm update	guarded stop	At 20.2 rpm, `p_min=-0.177 Pa < pSat=0.5 Pa`; this is a threshold bracket, not cavitation.
Native Reynolds/JFO result	unvalidated candidate	Its active-set kernel passes a coarse published graph-consistency check, but the conical feed mask is grid-dependent and the severe 64.455% rupture region is not physically validated.

The STEP result does not invalidate the native CAD or BREP mesh path. See Interchange formats and strict STEP failure.

Documentation map

  - Direct-jump diagnosis and accepted native ramp
  - Accepted 20-rpm point and guarded 20.2-rpm stop
  - Native OpenFOAM Reynolds/JFO candidate and validation blockers
  - Published JFO benchmark, numerical audit, and Fluent path

7. Rationale and provenance

  - Decision register
  - Modelling and meshing glossary
  - Bearing literature
  - Software manuals
  - Project provenance and conversational context

Source entry points

- CAD: bearing$_{\text{film}}$.main
- Native BREP preflight: run$_{\text{preflight}}$
- Surface smoke mesh: run$_{\text{surfacesmoke}}$
- No-port Hex8 mesh: run$_{\text{structured}}$
- Surface-inlet Hex8 mesh: run$_{\text{surfaceinlet}}$
- Ported Prism6 mesh: run$_{\text{ported}}$
- Solver-neutral export: run$_{\text{interchange}}$
- Live Fluent audit: run$_{\text{fluentimportaudit}}$
- Tests: tests/
- OpenFOAM study: CFD handoff

Scope boundary

The CAD and mesh-generation stages prove geometry, topology/quality, and static solver-neutral transfer without assigning CFD physics. The later OpenFOAM study assigns its own boundary conditions and material/model inputs in isolated cases documented under `docs/handoff/`. Its accepted result boundary is currently the single-liquid branch through 20 rpm. The 20.2-rpm threshold crossing, rejected 200-rpm liquid-only run, and rejected exploratory cavitation runs must not be reported as physical bearing performance. The separate native Reynolds/JFO high-speed result is likewise an unvalidated numerical candidate, not a promoted physical result.

Maintenance

- Documentation conventions
- Decision register

Project

Pipeline and Trust Boundaries

Source: `docs/project/pipeline.org`

The pipeline is a chain of contracts

The project does not have one global notion of "the model passed." Each stage answers a different question and records enough evidence for the next stage to reject the input if its contract changes.

```
Input parameters
      |
      v
exact OCCT fluid B-rep -------→ STEP exchange audit
      |                             |
      | native .brep                +-- strict FAIL is isolated here
      v
Gmsh OCC import + fragmentation
      |
      +---→ 2-D boundary smoke mesh
      |
      +---→ analytic no-port Hex8 regression baseline
      |
```

```
   +---→ analytic surface-inlet Hex8 reduced model
   |
   +---→ analytic central-feed Prism6 full-passage model
                 |
                 v
     MSH/VTU/NPZ round trips
                 |
                 v
     canonical NPZ → clean MSH/CGNS round trips
                 |
       +---------+----------+
       |                    |
       v                    v
optional Fluent import   optional gmshToFoam
and native .msh.h5       + checkMesh audit
```

The arrows are file-and-report contracts, not informal assumptions. Hashes, units, counts, volumes, topology, and physical-group membership are rechecked at each boundary.

Stage 1: exact fluid CAD

$\text{bearing}_{\text{film}}$.main resolves parameters, constructs the bore blank, journal cutter, feed cylinder, base film and full film, splits the full film into three axial zones, validates everything, and then exports through a staging directory.

The production object is the **fluid volume**, not the journal or bushing metal. Context solids are exported separately and never enter the mesh contract.

The native OCCT representation is accepted only after analytic and topological checks. STEP is additionally audited as an exchange format. The present strict STEP failure is explained in the interchange chapter.

Stage 2: geometry preflight inside Gmsh

$\text{run}_{\text{preflight}}$ imports native BREP into a deliberately non-healing Gmsh OCC configuration. It checks the unsplit volume, imports the three zones in a separate model, fragments them to establish shared topology, classifies volumes and surfaces by geometry rather than tags, and writes both millimetre and SI copies.

This stage proves that Gmsh sees the same geometry and conformal interfaces. It does not prove that any mesh can be generated.

Stage 3A: boundary surface smoke test

$\text{run}_{\text{surfacesmoke}}$ consumes the preflight report and the fragmented BREP by hash, scales millimetres to metres exactly once, and asks Gmsh for a first-order triangular **surface** mesh. It asserts that no 3-D elements exist.

This is a cheap probe of boundary classification, interface sharing, inlet resolution, and local surface-mesh quality before committing to a volume-mesh strategy.

Stage 3B: no-port structured Hex8 reference

$\text{generate}_{\text{mesh}}$ computes canonical nodes and Hex8 connectivity directly with NumPy. Gmsh is a container, inspector, and quality evaluator; it does not generate the 3-D mesh.

The feed is deliberately absent. This gives a clean, full-360, all-Hex regression case for the eccentric conical film and isolates thin-film layering from port-junction topology.

Stage 3C: surface-inlet structured Hex8 mesh

$\text{build}_{\text{surfaceinletmesh}}$ reuses the periodic analytic Hex8 film topology, redistributes theta and axial coordinates around the feed location, and applies symmetric through-gap inflation. A connected subset of outer bore Quad4 faces is reclassified from `stationary_wall` to `pressure_feed`.

This is a reduced physical model, not a ported geometry: it contains no feed tube or drilling cells. The central empty region seen in a skin rendering is the journal, which is intentionally outside the lubricant cell zone. See the surface-inlet chapter.

Stage 3D: ported layered Prism6 mesh

$\text{build}_{\text{mastermesh}}$ uses Gmsh only to triangulate a two-dimensional unwrapped master surface. $\text{build}_{\text{prismmesh}}$ maps and sweeps those Tri3 cells analytically into the conical film and feed tube, constructing every Prism6 explicitly.

The bore-side feed disk is not a wall. Each triangle has two incident cells, one film Prism6 and one tube Prism6, so it is an internal owner-neighbour face. A one-incident-cell triangle there would be an external cap and would incorrectly block the passage. See the ported mesh chapter.

Stage 4: solver-neutral interchange

$\text{run}_{\text{interchange}}$ reads the validated ported NPZ and source reports, reconstructs one clean discrete mesh, and writes MSH 4.1, MSH 2.2, and CGNS. All three are reopened against coordinate/connectivity signatures independent of imported tag numbering. Exact BC names and memberships are required inside CGNS itself.

The reduced surface-inlet family has its own canonical Hex8 NPZ/MSH/VTU round trips. Its current Ansys CGNS handoff is recorded separately and remains `STATICALLY_VALIDATED_NOT_IMPORTED` until Fluent performs the live import audit.

This stage is documented in Solver-neutral interchange.

Stage 5: optional solver import audits

The OpenFOAM branch converts the MSH 2.2 file and runs `checkMesh` when its utilities exist. The Fluent branch imports CGNS in a real 3-D, double-precision process and saves native `.msh.h5` only after live checks match the canonical source. Neither branch initializes or solves a flow.

That last sentence is the boundary of the canonical mesh-production automation. A separate downstream study later imported the validated S100 surface-inlet Hex8 mesh into OpenFOAM 14, commissioned the solver, and continued the zero-pressure-bias liquid branch through an accepted 20-rpm checkpoint. See the OpenFOAM CFD handoff. The first 20-to-50-rpm update stopped at the configured saturation threshold; it is not an accepted cavity solution. A later native midsurface Reynolds/JFO application is preserved separately as an unvalidated numerical candidate; its internal convergence must not be confused with physical validation.

- OpenFOAM hand-off boundary
- Fluent import-only boundary

Why failures are stage-local

A neutral-format translation can perturb curved intersections without changing the native B-rep. A CAD-valid solid can still have unsuitable mesh topology. A topologically conformal mesh can still contain negative Jacobians. A valid mesh can still be converted into the wrong patch set. Therefore every stage has its own PASS/FAIL record and downstream code checks the upstream report and file hash instead of trusting a filename.

Evidence paths

- CAD report: `out/strict_default/params.json`
- OCC preflight: `out/gmsh_preflight/preflight_report.json`
- Surface smoke test: `out/gmsh_surface_smoke/surface_mesh_report.json`
- No-port mesh family: `out/structured_no_port_default/run_report.json`
- Surface-inlet mesh family: `out/structured_surface_inlet_default/run_report.json`
- Ported mesh family: `out/ported_prism_default/run_report.json`
- Solver-neutral handoff: `out/interchange_default/nGap_04/interchange_report.json`
- Downstream OpenFOAM evidence: CFD artifact map

Generated outputs are evidence, not source. Recreate them with the commands in Running and testing.

Architecture and Sources of Truth

Source: `docs/project/architecture.org`

Design rule: one authoritative representation per layer

The implementation avoids a vague “current model” that changes meaning as it moves between libraries. Instead, each layer has an explicit source of truth:

Layer	Source of truth	Derived representations
Inputs	frozen input/resolved dataclasses and `params.json`	console tables, plots
CAD	native OCCT B-rep solids	STEP exchange, context assembly
no-port mesh	immutable NumPy `MeshData` arrays	Gmsh MSH, VTU, NPZ
ported mesh	`MasterMesh` plus `PrismMesh` arrays	Gmsh MSH, VTU, NPZ
neutral mesh hand-off	canonical ported NPZ plus source reports	clean MSH 4.1, MSH 2.2, CGNS
proprietary import	live Fluent process and canonical comparison	native Fluent `.msh.h5` only after PASS

Exports are not silently treated as equivalent to their source. Each is re-imported and compared.

Parameter ownership

CAD inputs live in InputParams. ResolvedParams contains the resolved feed position and quantities derived from the independent inputs. Both are frozen dataclasses so geometry cannot drift through incidental mutation.

Meshing scripts read `resolved_parameters` from `params.json` rather than reconstructing CAD choices from defaults:

- $\text{load}_{\text{params}}$
- $\text{load}_{\text{contract}}$

The ported pipeline also binds the parameter file, BREP, and preflight report by hash and by measured mass/bounding box. A plausible path is not sufficient.

Coordinate representations and unit boundaries

There are three intentionally different coordinate spaces:

1. **CAD space**: millimetres, global bearing axis `Z`.
2. **Master-surface space**: unwrapped millimetres `(u,z)` with `u=R_m theta`.
3. **Solver mesh space**: metres, obtained by one multiplication by `0.001`.

The unwrapped plane is a meshing parameterization, not a physical flattening of the final fluid volume. The final nodes are evaluated from the analytic eccentric-cone equations before the single SI conversion.

Each report records coordinate units. Unit conversion belongs at a named boundary so it can be asserted and cannot be applied twice.

Script boundaries

1. `bearing_film.py`

   Owns exact B-rep construction, CAD invariants, CAD exports, and CAD round trips. It knows nothing about solver cells.

2. `gmsh_brep_preflight.py`

   Owns the interpretation of native CAD inside Gmsh OCC: import options, fragmentation, volume/surface classification, SI copy, and BREP rechecks. It does not create a mesh.

3. `gmsh_surface_smoke.py`

   Owns the low-cost 2-D boundary triangulation probe. It intentionally refuses to create volume elements.

4. `structured_hex_no_port.py`

   Owns the frozen all-Hex no-port reference. Its canonical arrays are computed analytically and must not be changed as a shortcut for ported-mesh work.

5. `layered_prism_central_feed.py`

   Owns the actual feed-connected volume mesh. It reuses a Gmsh-generated master Tri3 topology, but constructs all 3-D Prism6 cells analytically.

6. `export_solver_neutral.py`

   Owns format-neutral reconstruction from the validated `mesh_arrays.npz`. It does not read CAD or remesh. It creates a clean discrete model, writes MSH and CGNS, and reopens every format against canonical signatures.

7. `fluent_import_check.py`

   Owns the optional proprietary runtime boundary. It launches Fluent only for CGNS import, mesh/quality checks, canonical count/scale comparison, and a guarded native-mesh save. It never initializes or iterates.

Data-model choices

The mesh dataclasses carry arrays and metadata together:

- MeshData
- MasterMesh
- PrismMesh
- CanonicalCase
- CanonicalAudit

Connectivity uses stable integer node tags, while NumPy arrays use compact zero-based rows internally where convenient. Conversion functions are explicit because an off-by-one error can preserve visual appearance while inverting or disconnecting cells.

The canonical arrays are written to `mesh_arrays.npz`. Generator MSH/VTU and interchange MSH/CGNS round trips are compared back to these arrays; the export library is never the authority.

Classification policy

Imported CAD entity tags and Boolean return order are unstable implementation details. Geometry is classified using centres of mass, bounding boxes, areas, adjacency, and known planes. Analytic meshes derive boundary groups from logical indices and a generic face census.

The generator's declared Gmsh physical IDs are stable local contracts, while elementary entity tags are model-local. Neutral-format checks use physical names and exact memberships because another reader may remap both numeric ID classes. See Topology proofs and quality metrics.

Validation and publication architecture

Validation functions receive a record list and add named PASS/FAIL evidence. Exceptions stop the pipeline; they do not request relaxed tolerances. Output is built in a sibling staging directory and atomically moved into place only after all mandatory checks and round trips succeed.

The mesh generators publish an explicit failure directory with solver-eligible artifacts removed. The interchange exporter instead discards its incomplete staging directory; an already published destination is left untouched. In both cases partial new MSH/CGNS/VTU files cannot be mistaken for a completed run.

Relevant entry points:

- CAD validation
- preflight records
- Hex atomic run
- Prism atomic run
- interchange atomic run
- live Fluent import audit

What is intentionally not abstracted

The stages share ideas, but they do not inherit from a general mesh framework. Their geometry, element types, and validation contracts differ enough that a common class hierarchy would obscure the physical reasoning. The interchange stage reuses only the already-tested Prism6 discrete registration and face/ metric helpers; it does not introduce an exporter framework for hypothetical future meshes.

Decision Register

Source: `docs/project/decisions.org`

Records decisions affecting geometry semantics, mesh topology, or artifact trust. Derivations remain in the subsystem chapters.

D001 — Model the fluid, not the metal

Status accepted
Context CFD requires the connected lubricant volume and its wetted boundaries; subtractive metal models invite a second interpretation step.
Decision construct the bore-volume blank, unite the feed volume, and subtract the journal. Export metal only as separate context.
Rejected export the bushing and ask a downstream tool to infer voids.
Consequence CAD volume and mesh volume describe the same physical domain.
Evidence CAD construction.

D002 — Preserve the same-z radial-clearance definition

Status accepted
Context radial clearance and shortest distance between conical surfaces differ by a factor involving the cone angle.
Decision build and layer the film along bearing-centred radial rays at the same axial coordinate. Validate surface-normal distance separately.
Rejected offset the cone by a normal distance or compare OCCT minimum distance directly with `c-e`.
Consequence the geometry matches the stated bearing convention while both measures remain reportable.
Evidence Clearance mathematics.

D003 — Use a full 360-degree domain

Status accepted
Context the geometric half is mirrorable, but the rotating journal-wall velocity is not a mirror-symmetry boundary condition.
Decision all solver-eligible CAD and meshes close periodically around the complete circumference.
Rejected diametral half-domain CFD.
Consequence there is no circumferential seam patch; debug cutaways are marked non-solver artifacts.

D004 — Subtract the journal last

Status accepted
Context the radial feed cylinder deliberately begins inside the journal so it robustly overlaps the bore blank.
Decision compute `wet = bore_blank + feed_cylinder`, validate it as one solid, then compute `film = wet - journal`.
Rejected trim the feed by guessed coordinates or subtract it from metal before constructing the fluid.
Consequence the journal surface trims the passage exactly and no internal fluid overlap remains.
Evidence make$_{\text{fullfilm}}$.

D005 — Native BREP is the open-source geometry contract

Status accepted
Context strict STEP re-import errors are localized around the feed intersection, while native BREP round trips remain at numerical noise.
Decision retain STEP for engineering interchange, but use native OCCT BREP for the Gmsh/OpenCASCADE path.
Rejected loosen the STEP tolerance until the report turns green.
Consequence format accuracy is visible and does not block trustworthy native geometry.
Evidence Interchange.

D006 — Disable automatic OCC repair during preflight

Status accepted
Context healing can make an import appear successful while changing the entity graph or tolerances.
Decision explicitly disable auto-fix, sewing, small-edge/face repair, and fuzzy Boolean behaviour; compare topology with Boolean simplification off.
Rejected accept Gmsh defaults and trust the imported appearance.
Consequence a failure remains diagnostic instead of being silently healed.
Evidence configure$_{\text{occoptions}}$.

D007 — Retain the no-port all-Hex mesh as a frozen baseline

Status accepted
Context the thin annulus naturally maps to a structured circumferential/axial/gap grid; the circular port junction does not.

Decision preserve a simple analytic Hex8 reference without the feed.
Rejected distort the baseline into an increasingly complex port topology.
Consequence changes to cone geometry, periodic connectivity, layering, quality, and file round trips have a stable regression target.
Evidence No-port Hex8 baseline.

D008 — Use a Tri3 master mesh and explicit Prism6 sweeps for the port

Status accepted
Context forcing an all-Hex block decomposition around a circular branched junction is brittle and unnecessary for the current objective.
Decision let Gmsh triangulate only an unwrapped 2-D master surface, then sweep it analytically through the film and tube.
Rejected Gmsh tetrahedral volume generation, an all-Hex port decomposition, STL/snappyHexMesh, or a commercial mesher.
Consequence layers remain explicit, the mouth is conformal, and every 3-D cell is derived from auditable arrays.
Evidence Ported Prism6 mesh.

D009 — Treat the feed mouth as an internal owner-neighbour interface

Status accepted
Context fluid passes from the tube into the film; there is no physical wall at their common mouth.
Decision reuse the same bore-side feed-disk nodes and triangles. Every mouth face must have exactly two incident cells, one film and one feed, and must not be exported as a boundary element.
Rejected create a `feed_mouth` patch, cap, coincident node set, or later node merge.
Consequence the mesh has one connected fluid region and an open passage.
Evidence $\text{validate}_{\text{topology}}$.

D010 — Keep canonical arrays independent of export formats

Status accepted
Context a writer can reorder tags or cells and still produce a valid file; an export can also silently drop metadata.
Decision NumPy arrays are authoritative. Re-import MSH4, MSH2, VTU, NPZ, and clean CGNS and compare coordinates, connectivity, data lengths, groups, and counts.
Rejected consider a successful write call sufficient validation.
Consequence visualization and solver files are proven derivatives of one mesh rather than parallel constructions.

D011 — Publish atomically and fail closed

Status accepted
Context a failed rerun in an existing output directory can otherwise leave an old solver file beside a new failure report.
Decision generate in staging, validate there, and replace the destination only on success. Mesh-generation stages may publish a failure-only directory; interchange discards its failed staging tree and preserves any previously published destination.
Rejected incremental writes directly into the final directory.
Consequence no partial artifact from the current attempt reaches the publication path.

D012 — Keep CFD physics out of mesh generation

Status accepted stage boundary; downstream CFD stage implemented separately
Context mesh boundary names do not determine pressure, mass flow, rotation speed, lubricant rheology, cavitation, or turbulence assumptions.
Decision keep the canonical geometry/mesh pipeline ending after geometry, mesh, optional format conversion, and `checkMesh`. Build solver cases as a separate evidence stream with explicit physical assumptions and acceptance gates.
Rejected add placeholder solver dictionaries that appear physically authoritative.
Consequence the later OpenFOAM S100 study makes and documents its own physics decisions under `docs/handoff/` without weakening the mesh-stage contract.

D013 — Reject concentric ported cases until BREP classification is unambiguous

Status accepted limitation
Context for zero eccentricity, journal and bore conical surfaces share the same axis. The current BREP probe's centre-distance classifier would assign both to the bore rather than distinguish their analytic surface equations.

Decision require `0 < eccentricity_ratio < 1` in the ported-mesh input contract and report a clear concentric-geometry error.
Rejected silently accept zero eccentricity and weaken or skip the BREP surface check.
Consequence a future concentric implementation must classify using residuals against both journal and bore radius equations, with regression coverage, before relaxing the guard.
Evidence explicit rejection regression.

D014 — Keep optional GUI launch outside canonical publication failure handling

Status accepted
Context an FLTK/Wayland error can occur after a fully validated mesh has been atomically published.
Decision complete validation and directory replacement first; catch GUI launch errors separately and emit a warning only.
Rejected let a display failure replace valid artifacts with a mesh failure report.
Consequence GUI availability cannot alter solve-eligible mesh state.
Evidence $\text{open}_{\text{optionalgui}}$ and GUI isolation regression.

D015 — Reconstruct neutral files from canonical arrays

Status accepted
Context copying the generator MSH would retain post-processing views and make a prior export, rather than the validated arrays, the interchange source.
Decision load `mesh_arrays.npz` plus its manifests, create a clean discrete model, and write every neutral format from that one model.
Rejected copy/rename MSH, rebuild from BREP, or generate new volume cells.
Consequence format transfer preserves the exact validated mesh identity while remaining independent of CAD and solver choice.
Evidence Canonical interchange source.

D016 — Use Gmsh's CGNS writer and correct surface BC locations

Status accepted
Context CGNS boundary names and memberships must exist in the CGNS hierarchy; a sidecar cannot supply BC definitions to an importer.
Decision use the pinned Gmsh build's CGNS-library writer; change only surface-element BC locations from `CellCenter` to the standard `FaceCenter`; then reopen CGNS and require one fluid zone plus the exact boundary groups and memberships.
Rejected a generic hand-written HDF5 exporter, geometry-only CGNS with sidecar names, or a second mesh writer with parallel connectivity mapping.
Consequence name loss produces `GEOMETRY_ONLY_NOT_FLUENT_READY` and no successful publication. The correction fails unless every non-location dataset remains byte-identical.
Evidence Formats and CGNS.

D017 — Separate static transfer from live Fluent acceptance

Status accepted
Context Gmsh reading its own CGNS output cannot prove Fluent's importer, zone mapping, unit interpretation, or licensed runtime behavior.
Decision report `STATIC_PASS_FLUENT_NOT_RUN` until a real Fluent process imports, checks, queries, compares, and saves the mesh.
Rejected infer `FLUENT_IMPORT_PASS` from file creation, mocked output, or parser fixtures.
Consequence `bearing_prism_imported.msh.h5` becomes a guarded promotion artifact and is absent from static-only outputs.
Evidence Fluent import-only audit.

D018 — Apply supply pressure directly on the bore for the reduced film model

Status accepted
Context the resolved feed passage is useful when its pressure loss and jet dynamics are in scope, but it also introduces a fluid column that is outside the intended film-only hydrodynamic study.
Decision retain the full-passage Prism6 branch, and add a separate pure Hex8 annular-film branch whose `pressure_feed` is a mesh-aligned subset of the stationary bore faces. Use symmetric `5:1` through-gap inflation and smooth cubic theta/z clustering with `s=0.82` about the inlet.

Rejected delete boundary faces and leave an invalid open volume; extend the tube into the gap; call Prism6 cells tetrahedra; silently reinterpret the full-passage branch; globally multiply the mesh merely to resolve the 4 mm footprint; introduce an abrupt locally refined block.

Consequence the new domain has no feed-volume cells or hydrostatic column. The inlet remains a boundary-face group, as every finite-volume opening must, and its fine stair-stepped 4 mm approximation is measured explicitly. Clustering preserves conforming tensor connectivity but coarsens remote cells, so theta/axial convergence remains necessary.

Evidence Surface-inlet Hex8 mesh.

D019 — Bypass the failed CGNS reader with a native Fluent Hex8 file

Status accepted after live Fluent failure

Context corrected HDF5 and ADF CGNS files both terminated Fluent 26.1 Student inside `CeetronSAM.dll` before `Mesh Check`. Switching the CGNS storage backend therefore did not remove the failing reader path.

Decision for the reduced surface-inlet Hex8 branch, reconstruct Fluent's native face-owner representation directly from `mesh_arrays.npz` and read it through **File → Read → Mesh**.

Rejected keep retrying CGNS; rename a Gmsh MSH; use meshio's ANSYS writer, which does not emit the required Fluent owner/neighbour face sections.

Consequence the nodes and Hex8 cells remain unchanged, while the live test no longer enters Ceetron/VKI. Static validation still does not become `FLUENT_IMPORT_PASS` until Fluent reads, checks, and saves the native mesh.

Evidence native Fluent writer and format rationale.

D020 — Preserve the native JFO result as an unvalidated candidate

Status accepted evidence boundary

Context the native OpenFOAM Reynolds/JFO application converges to 2000 rpm and agrees with a second implementation. The shared active-set kernel also passes a coarse graph-consistency check against a published 60-cell slider comparison, but the full implementations encode the same conical equations and their cell-centre feed footprint changes discontinuously with grid resolution.

Decision preserve source, checkpoints, plots, and diagnostics while withholding physical validation until the native assembly passes external or manufactured checks, the feed geometry is grid-controlled, local conservation is measured, and a valid grid study passes.

Rejected call internal convergence, global balance, Python/C++ agreement, one kernel benchmark, or one maximum-pressure match to the paper a validation of cavity fraction, load, flow, or torque.

Consequence Fluent is an independent model comparison unless a separately verified UDF implements the same pressure–fill complementarity law.

Evidence Native JFO candidate and validation and Fluent path.

Physics

Lubrication Physics and Model Boundary

Source: `docs/physics/lubrication-context.org`

Hydrodynamic film mechanism

Journal rotation imposes no-slip tangential motion on the lubricant at the journal surface. Eccentricity creates circumferentially converging and diverging film regions. Viscous transport into the converging region generates a pressure rise whose surface integral produces the load-carrying force.

The thin-film velocity field is commonly interpreted as a superposition of:

- Couette flow driven by the moving journal wall;
- Poiseuille flow driven by pressure gradients.

Under steady, incompressible, cylindrical thin-film assumptions, one common Reynolds-equation convention is

$$\frac{1}{R^2}\frac{\partial}{\partial\theta}\left(h^3\frac{\partial p}{\partial\theta}\right)+\frac{\partial}{\partial z}\left(h^3\frac{\partial p}{\partial z}\right)=6\mu\Omega\frac{\partial h}{\partial\theta}.$$

Signs and numerical factors depend on coordinate and wall-velocity conventions. The 3-D mesh pipeline does not solve this equation. The later native midsurface application does solve its conservative conical Reynolds counterpart.

Reynolds equation versus JFO

These names describe different layers, not competing equations:

- The **Reynolds equation** is the thin-film pressure/mass-conservation governing equation obtained from Navier–Stokes under lubrication assumptions.
- A plain Reynolds solve still needs a rupture condition when its diverging wedge would otherwise produce nonphysical negative absolute pressure.
- **JFO** (Jakobsson–Floberg–Olsson) supplies the mass-conserving rupture and reformation conditions. Full-film cells carry pressure with liquid fill `theta=1`; ruptured cells remain at the declared rupture pressure and transport `0 <` theta < 1=.
- **Elrod–Adams** and complementarity methods are practical numerical formulations used to solve the Reynolds equation subject to JFO conditions.

Thus the current reduced model is "Reynolds plus JFO," implemented as an active-set complementarity problem. JFO was selected because the plain single-liquid calculation demanded impossible negative absolute pressure and because clipping that pressure would lose mass. It is not automatically the best physical model: thermohydrodynamic or elastohydrodynamic Reynolds models are needed when temperature or deformation matters; a 3-D liquid–vapour model is needed for phase change; and an oil–air interface model is needed when an open bearing ingests air. The declared mechanism decides which comparison is meaningful.

Conical surface coordinates

Parameterising the coaxial bore by `(theta,z)` gives orthogonal metric factors

$$h_\theta = R_b(z), \qquad h_z = \sqrt{1 + \left(\frac{dR_b}{dz}\right)^2} = \sec\gamma.$$

A reduced Reynolds model on the developed cone must retain the changing circumferential length scale and the generator-length factor. The 3-D mesh branches instead represent the physical fluid volume directly. The full-passage Prism6 branch resolves the feed passage and its local disturbance; the reduced surface-inlet Hex8 branch omits that passage and applies supply pressure directly on the bore.

The developed-surface model in the conical-bearing paper motivates the circumferential/axial parameterisation but does not validate this project's three-dimensional port geometry or mesh.

Rotating-wall kinematics and the full circumference

For journal angular velocity `Omega` about the displaced axis `(e_x,e_y)`,

$$\mathbf{u}_w = \Omega\hat{\mathbf{z}} \times \left[(x - e_x)\hat{\mathbf{x}} + (y - e_y)\hat{\mathbf{y}}\right] = \left[-\Omega(y - e_y),\ \Omega(x - e_x),\ 0\right].$$

Although the default static geometry is mirror-symmetric about `x=0`, this vector field is not represented by an ordinary diametral symmetry boundary. The solver-eligible domain is therefore full 360 degrees; half and cutaway artifacts are diagnostic only.

Eccentricity direction versus hydrodynamic load

The eccentricity vector fixes the geometric minimum-gap meridian. The pressure maximum occurs downstream in the converging wedge, and the integrated force also depends on speed, viscosity, feed conditions, axial leakage, cavitation, and cone geometry. The eccentricity line is consequently not assumed to be the external or hydrodynamic load direction.

Feed boundary role

The project retains two different feed models:

- In the surface-inlet Hex8 branch, `pressure_feed` is a near-circular set of stationary-bore faces. There is no feed tube, hydrostatic column, pipe-wall friction, developing pipe flow, or inlet jet in the domain.
- In the full-passage Prism6 branch, the external tube extension terminates in a planar circular disk beyond the context bushing. The mesh resolves the drilling and tube-to-film connection.

Both names identify candidate supply boundaries only. This repository does not assign a pressure, mass flow, velocity, or other solver condition.

Quantities not predicted here

The repository defines geometry, topology, cell quality, named patches, and format transfer. Without a lubricant constitutive model, density, journal speed, supply and outlet conditions, cavitation treatment, and thermal model, it cannot predict pressure, load, attitude angle, torque, leakage, temperature, or stability. Those choices belong to the later CFD model and its verification plan.

Geometry

Geometry and CAD

Source: *`docs/geometry/index.org`*

Purpose

This section explains how the project constructs and accepts the exact lubricant-fluid CAD volume for an eccentric conical journal bearing. The production body is fluid, not metal. It contains the complete 360-degree conical film, the radial feed drilling, and the external inlet extension. Journal and bushing solids are separate visual context.

The executable source of truth is bearing$_{\text{film}}$.py: main(). It uses build123d algebra mode and OCCT B-rep operations; it does not create a mesh or run CFD.

Reading map

Document	Question answered
Coordinates and parameters	What is specified, what is derived, and which direction is positive?
Clearance mathematics	What does radial clearance mean, and why is it not the normal gap?
CAD construction	How do profiles, revolutions, overlaps, Booleans, and splits create the fluid?
CAD validation	Which independent checks make a geometry result trustworthy?
Acceptance results	What did the two strict CAD runs measure, and which exchange checks failed?
Interchange	Why are native BREP and STEP audited separately, and what is published?

Geometry contract

The CAD stage promises all of the following before an artifact is accepted:

- millimetre construction with exact B-rep solids;
- one connected, valid, manifold full-fluid solid;
- radial-clearance and surface-normal checks kept distinct;
- a feed passage open to the film and ending in one selectable circular inlet;
- three optional axial bodies whose volumes sum to the unsplit fluid volume;
- native BREP and STEP re-import checks performed independently;
- explicit failure and nonzero exit when a mandatory check fails.

The main construction order is visible directly in main(). The validation and export phases are not optional post-processing: they decide whether output is safe to publish.

Current test boundary

`bearing_film.py` is a self-validating command: mandatory checks use custom exceptions and recorded PASS/FAIL entries rather than removable Python `assert` statements. The accepted native BREP is also independently consumed by test$_{\text{nativebreppreflight}}$(). Meshing tests then use its parameters and geometry contract, for example the ported-mesh geometry/volume test.

There is currently no separate unit-test module dedicated solely to `bearing_film.py`. Its executable validation record and downstream native-BREP preflight are therefore both important parts of the geometry acceptance path.

Coordinates and Parameters

Source: *`docs/geometry/coordinates-and-parameters.org`*

Coordinate system and units

All CAD construction uses millimetres. The bearing axis is global `Z` and the physical interval is `0 < z <=` L=. The coaxial bore is centred on `Z`. The journal axis remains parallel to `Z` but is translated in the `XY` plane.

The bearing-frame eccentricity direction is measured from `+X` toward `+Y`:

$$e = \epsilon c, \qquad e_x = e\cos\phi, \qquad e_y = e\sin\phi.$$

The default `phi`-90 deg= gives `e_x approximately 0` and `e_y-e=`. Thus `-Y` is the eccentricity and minimum-radial-clearance line, while the feed axis points along `+Y`. This geometric line is not automatically the external load line; load direction requires the later flow solution.

See InputParams and $\text{resolve}_{\text{params}}$().

Independent inputs

InputParams is frozen so the requested case cannot change after parsing. Each field has a command-line counterpart.

Name	Default	Meaning
`length`	60 mm	Axial length `L`
`mean_radius`	50 mm	Journal radius at `z=L/2`
`semicone_angle_deg`	10 deg	Cone half-angle `gamma`; large end at `z=0`
`radial_clearance`	0.050 mm	Same-`z` radial clearance `c`
`eccentricity_ratio`	0.6	`epsilon=e/c`
`eccentricity_angle_deg`	-90 deg	Journal-axis displacement direction `phi`
`hole_diameter`	4 mm	Feed diameter `d_h`
`hole_axial_pos`	omitted	Resolves to the current `L/2`
`split_halfwidth`	4 mm	Half-width `w` of the feed band
`bushing_wall_thickness`	10 mm	Illustrative radial wall thickness
`inlet_extension`	3 mm	Fluid stub beyond the outer wall
`axial_cutter_extension`	1 mm	Cutter overrun beyond each axial end
`max_face_count`	100	Sliver-face explosion guard

`export_debug_half`, `preview`, and `outdir` control diagnostics and publication, not the production fluid dimensions. The actual parser is $_{\text{parseargs}}$().

Radius laws

Let `m=tan(gamma)`. The three coaxial radius laws are

$$R_j(z) = R_m + (L/2 - z)m,$$
$$R_b(z) = R_j(z) + c,$$
$$R_o(z) = R_b(z) + t_w.$$

They are implemented once in $\text{journal}_{\text{radius}}$(), $\text{bore}_{\text{radius}}$(), and $\text{outer}_{\text{radius}}$(). The large end is at `z=0` for non-negative `gamma`.

Resolved feed placement

The hole radius and touched axial interval are

$$r_h = d_h/2, \qquad z_{hole,min} = z_h - r_h, \qquad z_{hole,max} = z_h + r_h.$$

Because the allowed cone grows toward lower `z`, the largest context-wall radius touched by the finite inlet disk occurs at `z_h-r_h`. Therefore

$$y_{feed,end} = R_o(z_h - r_h) + s_u.$$

This is why the inlet coordinate must be resolved from the current geometry; using the wall radius only at `z_h` would not prove that the entire disk clears the conical wall. The split locations are `z_1=z_h-w` and `z_2=z_h+w`.

ResolvedParams stores these values, the two gap definitions, analytic base volume, and diagnostics. It is a second frozen dataclass, keeping user inputs separate from derived state.

Pre-construction guards

$\text{validate}_{\text{params}}$() rejects bad cases before OCCT sees them. Important guards include:

- positive length, radii, clearance, hole size, wall, extension, and cutter overrun;
- `0` < epsilon < 1= and `0` < gamma < 90 deg=;
- the complete hole and both split planes lying inside `0 < z < L`;
- `w > r_h`, keeping the feed inside the middle axial band;
- positive small-end journal and bore radii;
- the complete feed-start disk at `y=0` being inside the journal cutter;
- a remote feed endpoint outside the context bushing.

The validator accumulates all detected parameter errors and raises one `ParameterValidationError`, so a case is corrected from a complete diagnostic rather than one failing scalar at a time.

The start-disk proof uses a conservative maximum transverse distance

$$\sqrt{(|e_x| + r_h)^2 + e_y^2}$$

and subtracts it from the smallest journal radius touched by the disk. A positive stored margin makes the later deliberate feed/journal overlap a validated construction choice rather than an assumption.

Clearance Mathematics

Source: *`docs/geometry/clearance-mathematics.org`*

Radial versus surface-normal clearance

> Radial clearance is measured in a plane of constant `z` along a ray from the bearing axis. It is not the shortest Euclidean distance normal to the conical surfaces.

The input `c` and every film-thickness field in this project use the first definition. An unrestricted OCCT distance query returns the second. The two targets must never be interchanged.

The conical thin-film setting and developed-surface formulation are consistent with the Gangrade–Phalle–Mantha paper indexed in Literature. The exact ray intersection, normal-distance distinction, and validation tolerances below are project derivations; the paper is context, not implementation evidence.

Exact same-z radial thickness

At fixed `z`, the journal cross-section has radius `R_j(z)` and centre `(e_x,e_y)`. Let the bearing-frame ray direction be

$$\hat{u}(\theta) = (\cos\theta, \sin\theta),$$

and define `alpha=theta-phi`. If `r_j` is the positive ray intersection with the journal circle, then

$$\left|r_j\hat{u}(\theta) - e\hat{u}(\phi)\right|^2 = R_j(z)^2.$$

Solving the resulting quadratic and taking the outward root gives

$$r_j(\theta, z) = e\cos\alpha + \sqrt{R_j(z)^2 - e^2\sin^2\alpha}.$$

The exact radial film thickness is therefore

$$h_{radial}(\theta, z) = R_b(z) - r_j(\theta, z).$$

At `alpha=0` and `alpha=pi` this reduces exactly to

$$h_{radial,min} = c - e = c(1-\epsilon), \qquad h_{radial,max} = c + e = c(1+\epsilon).$$

The resolved targets are stored by $\text{resolve}_{\text{params}}()$. Their five-station geometric measurement is in $\text{validate}_{\text{geometry}}()$.

Why the common formula is only approximate

The thin-clearance expression

$$c(1 - \epsilon \cos \alpha)$$

is the first-order expansion of the exact square-root result for small `e/R_j`. It is exact at the minimum and maximum meridians, but not at general angle. Away from those meridians, the exact result also has a small `z` dependence because `R_j(z)` varies along the cone.

$_{\text{plotthicknessmap}}()$ uses the exact expression. Its PNG is a geometry diagnostic, not pressure, temperature, velocity, cavitation, or load.

Shortest surface-normal distance

In an axial meridian the journal and bore generators are parallel lines with slope magnitude `tan(gamma)`. A same-`z` radial separation `Delta r` has perpendicular separation

$$\frac{\Delta r}{\sqrt{1 + \tan^2 \gamma}} = \Delta r \cos \gamma.$$

At the extreme meridians,

$$h_{normal,min} = (c - e)\cos \gamma, \qquad h_{normal,max} = (c + e)\cos \gamma.$$

The code selects only the lateral conical faces and calls `distance_to_with_closest_points()`. It compares the returned minimum with `(c-e)cos(gamma)`, not with `c-e`. The closest points generally have different `z` coordinates, which is direct evidence that this segment is not a same-`z` radial measurement. See $_{\text{lateralface}}()$ and $\text{validate}_{\text{geometry}}()$.

Exact base-film volume

Before adding the feed, each coaxial annular section has area

$$A(z) = \pi \left(R_b(z)^2 - R_j(z)^2 \right) = \pi \left(2cR_j(z) + c^2 \right).$$

The linear radius law has mean value `R_m` over `0 < z <=` L=, so

$$V_{base} = \pi L \left(2R_m c + c^2 \right).$$

This result is independent of transverse journal translation while the journal remains contained, and independent of cone angle when `L`, `R_m`, and `c` remain fixed. It provides an analytic check independent of the Boolean algorithm. The implementation stores it in $\text{resolve}_{\text{params}}()$ and checks the Boolean volume to relative tolerance `1e-8` in $\text{validate}_{\text{geometry}}()$.

How radial gaps are measured from CAD

The validator avoids global minimum distance for the radial test:

1. section bore and journal with five interior `XY` planes;
2. rotate each section by `-phi` so the eccentricity vector aligns with `+X`;
3. use the two bounding extrema along that aligned axis;
4. compare with `c-e` and `c+e` to `0.0005 mm`.

This works for non-default eccentricity angle and excludes end-cap artifacts. The separate normal-distance check then tests the actual conical lateral faces. Keeping both records is more useful than asking either measurement to stand in for the other.

CAD Construction

Source: `docs/geometry/cad-construction.org`

Exact solids, not surface approximations

The geometry is made from planar profiles, exact revolutions, an exact cylinder, and OCCT set operations through build123d algebra mode. It does not use STL, tessellated offsets, mesh-derived geometry, fuzzy Booleans, or relaxed modelling tolerance.

The shared helper $_{\text{revolvedfrustum}}$() draws a closed polygon in the `XZ` plane,

`(0,z0) → (R(z0),z0) → (R(z1),z1) → (0,z1) → close,`

makes a planar face, and revolves it 360 degrees about global `Z`. Including the axis closes the profile and yields a solid rather than a conical shell.

Construction sequence

1. Bore-volume blank

   make$_{\text{boreblank}}$() revolves the bore radius law over `[0,L]`. This solid is everything inside the bearing bore. It is a Boolean blank for the fluid, not the bushing metal.

2. Extended eccentric journal cutter

   make$_{\text{journal}}$() first revolves the journal about global `Z` and then translates the completed solid by `(e_x,e_y,0)`. Revolving about an offset axis would describe a different surface.

   For subtraction, the journal spans `[-delta,L+delta]` while preserving the same radius law. The overrun prevents journal and bore end caps from being coplanar, avoiding a fragile coincident-face Boolean. A separate exact-length copy over `[0,L]` is created for context export.

3. Deliberately overlapping feed cylinder

   make$_{\text{feedcylinder}}$() constructs a radius-`r_h` cylinder from `y=0` to `y_feed_end` along global `+Y`. Parameter validation has already proven that its entire starting disk lies inside the journal cutter. The remote end clears the largest outer-wall radius touched by its finite disk.

   Immediately after construction, the function finds one planar face having the expected circular area, centre, and `+Y` normal, then requires one circular edge. This catches an axis, alignment, or length mistake before any Boolean can obscure the inlet.

   The external extension also separates the future supply boundary from the conical wall transition. Its remote planar disk can receive pressure, mass flow, or velocity data without ambiguous face selection; this CAD stage assigns none of those values.

4. Boolean order

   Let `B` be the bore blank, `F` the feed cylinder, and `J` the extended journal. The production order is

   ```
   base_film = bore_blank - journal
   wet = bore_blank + feed_cylinder
   film = wet - journal
   ```

   The corresponding functions are make$_{\text{basefilm}}$() and make$_{\text{fullfilm}}$().

   This order matters. `F` intentionally overlaps both `B` and `J`. Fusing `B+F` first must produce one valid connected wet solid. Subtracting `J` last then trims the overlong feed exactly to the journal surface and opens it into the film. Attempting to start the cylinder on the journal/feed intersection would require constructing against a more fragile intersection curve.

   The code validates `wet` before the final subtraction. It does not silently retry a failed operation with fuzzy or loosened tolerances.

Axial zones

split$_{\text{axialzones}}$() splits the already valid full film at `z_1=z_h-w` and `z_2=z_h+w`. It expects exactly three solids, sorts them by centre-of-mass `Z`, and verifies each bounding interval before naming them:

- `ring_A`: `0` < z <= z$_1$=;
- `hole_band`: `z_1` < z <= z$_2$=;

- `ring_B`: `z_2 < z <= L=`.

They are not re-fused. The coincident split faces are retained for downstream partition-aware meshing. These three bodies alone do not promise an all-Hex mesh; see Meshing.

Context metal

$\text{make}_{\text{contextbushing}}()$ constructs

`outer frustum - bore blank - feed cylinder.`

Together with the exact-length journal it provides visual context only. The CFD geometry remains the fluid solid. Keeping context separate prevents an exporter or downstream tool from mistaking metal for a second fluid region.

Full model and optional half

The production body is always 360 degrees. For rotation `Omega` about the displaced journal axis,

$$\mathbf{u}_w = \Omega\hat{\mathbf{z}} \times \left[(x - e_x)\hat{\mathbf{x}} + (y - e_y)\hat{\mathbf{y}}\right] = \left[-\Omega(y - e_y),\ \Omega(x - e_x),\ 0\right].$$

Even when the static default geometry is mirror-symmetric, this vector field is not represented by an ordinary diametral symmetry boundary. The physical interpretation is developed in Lubrication physics and model boundary.

`--export-debug-half` creates an explicitly labelled inspection artifact only. $\text{export}_{\text{all}}()$ prints `GEOMETRY DEBUG ONLY — NOT VALID FOR ROTATING-JOURNAL CFD` and never presents the half as a solver model.

Common construction failures

Failure	Likely cause	Guard
Feed is detached	Cylinder does not overlap bore, or journal was subtracted in the wrong order	Validate `wet` and final one-solid topology
Feed is capped at journal	Cylinder was not overlapped and journal was not subtracted last	Fixed Boolean order
Dirty axial faces	Cutter caps coincide with bore caps	`delta` overrun
Inlet clips the context wall	Endpoint based only on `R_o(z_h)`	Use `R_o(z_h-r_h)+s_u`
Inlet points in the wrong direction	Incorrect local plane orientation	Unique face centre/area/normal/circle check
Zone names are swapped	Trusting Boolean return order	Centre and bounding-range classification
Face-count explosion	Sliver faces at intersections	`max_face_count` validation and loud failure

CAD Validation

Source: `docs/geometry/cad-validation.org`

Validation is part of construction

The script does not equate "OCCT returned a shape" with success. $\text{validate}_{\text{geometry}}()$ builds structured records containing a name, mandatory flag, measurement, target, tolerance, and PASS/FAIL status. Mandatory failures raise GeometryValidationError. Specific exceptions also distinguish invalid inputs, construction, Boolean, topology, export, and round-trip failures.

These checks are normal control flow, not bare Python `assert` statements, so `python -O` cannot disable them.

Topology checks

$_{\text{shaperecord}}()$ records solid counts, individual volumes, face counts, bounding boxes, and property-form `is_valid` and `is_manifold` results.

The mandatory topology contract requires:

- exactly one final full-film solid;
- one connected, valid, manifold `wet` union before journal subtraction;

- every resulting solid valid and manifold;
- each axial zone and context body to be exactly one solid;
- a positive face count no greater than `max_face_count`;
- positive feed-added volume in the same connected full-film solid.

The face-count ceiling detects a Boolean sliver-face explosion without loosening OCCT tolerances to conceal it.

Independent volume checks

The base film is compared with

$$V_{base} = \pi L(2R_m c + c^2)$$

at relative tolerance `1e-8`. This check is independent of the B-rep Boolean implementation.

For the port, the code compares

$$V_{film} - V_{base}$$

with

$$\text{Vol}(feed\ cylinder - bore\ blank).$$

Both must be positive and agree within relative `5e-5`. The implementation records that default OCCT tolerance can perturb the shared cone/cylinder intersection slightly; it does not introduce fuzzy healing. The simpler `pi*r_h^2*(t_w+s_u)` value is diagnostic only because a sloped wall and finite-radius hole do not have one universal passage length.

After zoning,

$$\frac{|V_A + V_{band} + V_B - V_{film}|}{V_{film}} \leq 10^{-8}.$$

Clearance checks

Five interior sections measure same-z radial minimum and maximum gaps to `0.0005 mm`. The sections are rotated by `-phi` before using their extrema, so the test is not hardcoded to the default `-Y` displacement.

Separately, the unique lateral bore and journal faces are queried for their closest points. Their minimum Euclidean distance is compared with `(c-e)cos(gamma)` to `0.0005 mm`. It is deliberately not compared with the radial target `c-e`. See Clearance mathematics.

Bounding box and inlet identity

The final-fluid bounds must reproduce `z=0` and `z=L` and the large-end bore envelope to `0.001 mm`. The allowed `+Y` feed changes only the positive `y` bound, which must equal `max(R_b(0),y_feed_end)`. A preceding radius guard proves that the feed cannot change `x_min`, `x_max`, or `y_min`.

OCCT bounding boxes may extend roughly `1e-7 mm` beyond nominal analytic coordinates because they include default topological tolerance. This is several orders below the `0.001 mm` bound gate and does not indicate that the modelling tolerance was relaxed.

The validator finds the inlet by geometry rather than face index. Exactly one face must have:

- planar geometry;
- area `pi*r_h^2` within `max(1e-7 mm^2, relative 1e-8)`;
- centre `(0,y_feed_end,z_h)` within `1e-6 mm`;
- outward normal parallel to `+Y` with dot product at least `1-1e-10`;
- exactly one circular boundary edge.

Face ordering and persistent naming are not assumed.

Main in-memory tolerances

Check	Acceptance
Base-film analytic volume	relative < 1e-8=
Feed Boolean identity	relative < 5e-5=
Five-station radial gaps	absolute < 0.0005 mm=
Minimum normal gap	absolute < 0.0005 mm=
Axial split conservation	relative < 1e-8=
Final bounds	absolute < 0.001 mm=
Inlet centre	distance < 1e-6 mm=

Interchange tolerances are separate; see Interchange.

Failure reporting and publication

The main flow in main() prints the specific exception, emits diagnostics for every shape that exists, writes an `overall: FAIL` `params.json`, and exits nonzero. Export is staged beside the requested output directory. A failed STEP audit cannot leave newly staged or stale program-generated STEP files looking accepted.

$_{\text{failurepayload}}$() records the exception type/message, inputs, resolved parameters, dependencies, validation records, shape diagnostics, and hashes of any trusted native fallback.

Test boundary and downstream checks

There is no standalone `tests/test_bearing_film.py` at present. The CAD command itself performs its mandatory geometry checks for every run. The native artifacts are then independently checked in Gmsh/OpenCASCADE by test$_{\text{nativebreppreflight}}$() for both strict parameter cases.

Downstream analytic meshes cross-check the same geometry contract, including no-port analytic geometry and ported inlet, radial-gap, and volume agreement. Those tests do not replace CAD validation; they verify that accepted CAD parameters and BREP survive the handoff into meshing.

CAD Acceptance Snapshot

Source: `docs/geometry/acceptance-results.org`

Evidence boundary

This snapshot preserves the two strict CAD acceptance runs produced with Python 3.12.12, build123d 0.11.1, and cadquery-ocp-novtk 7.9.3.1.1. The full machine-readable records remain:

- `out/strict_default/params.json`;
- `out/strict_case_e03_g20/params.json`.

Both runs contain 80 validation records: 42 in-memory geometry checks and 8 native-BREP round-trip checks pass; 3 of 30 STEP records fail on volume. The truthful result is `77/80 PASS` and `OVERALL: FAIL`, isolated to STEP exchange.

Cases and primary measurements

Quantity	Default	`epsilon=0.3`, `gamma=20 deg`
semi-cone angle	10 deg	20 deg
eccentricity	0.030 mm	0.015 mm
radial gap min/max	0.020 / 0.080 mm	0.035 / 0.065 mm
normal minimum target	0.019696155060 mm	0.032889241728 mm
OCCT normal minimum	0.019696155060 mm	0.032889241727 mm
feed inlet `y`	63.402653961 mm	63.777940469 mm
exact base-film volume	942.949034975 mm^3	942.949034975 mm^3
feed-added volume	167.922786887 mm^3	172.640532984 mm^3
total fluid volume	1110.871821862 mm^3	1115.589567959 mm^3

The five same-`z` stations in each case reproduce their radial targets to floating-point noise. Split-volume relative errors are `5.12e-15` and `3.14e-14`.

Native BREP round trip

File	Default relative error	20-degree relative error	Gate
`film_unsplit.brep`	`1.373e-13`	`6.563e-14`	`⩽ 1e-12`
`film_zones.brep`	`2.419e-13`	`9.844e-14`	`⩽ 1e-12`

Every imported solid retains the expected count, validity, and manifoldness.

STEP round trip

All STEP files use `PrecisionMode.GREATEST`.

File	Default relative error	20-degree relative error	Gate
`film_unsplit.step`	`2.800e-5`	`5.962e-5`	`⩽ 1e-6`
`hole_band.step`	`1.059e-4`	`2.229e-4`	`⩽ 1e-6`
`film_zones.step`	`2.800e-5`	`5.962e-5`	`⩽ 1e-6`
`ring_A.step`	`5.544e-13`	`4.532e-12`	`⩽ 1e-6`
`ring_B.step`	`7.577e-13`	`4.470e-12`	`⩽ 1e-6`
`context_assembly.step`	`3.219e-8`	`1.228e-7`	`⩽ 1e-6`

The absolute full-fluid losses are 0.0311089 mm^3 and 0.0665082 mm^3. The passing annular lands and failing feed-containing bodies localise the measured translation error to the cone/cylinder intersection rather than the base film. No STEP files are published in the strict output directories.

Interpretation

The native geometry is accepted for BREP-based Gmsh/OpenCASCADE work. STEP remains an engineering exchange target for external CAD tools, but these files do not satisfy the project's strict thin-film regression gate on the recorded software stack. The gate is not relaxed to convert a format-specific failure into a geometry PASS.

CAD Interchange: Native BREP and STEP

Source: `docs/geometry/interchange.org`

Two distinct acceptance questions

The project keeps these questions separate:

1. Is the in-memory/native OCCT geometry correct?
2. Does a neutral STEP translation preserve that geometry closely enough?

A native BREP PASS does not imply a STEP PASS, and a STEP failure does not by itself invalidate an already validated in-memory solid. This distinction is especially important where the thin annulus meets the radial cone/cylinder intersection.

The exporter helpers are $_{\text{exportbrepfile}}$() and $_{\text{exportstepfile}}$().

Native OCCT BREP

`film_unsplit.brep` stores the one complete fluid solid. `film_zones.brep` stores the three coincident axial solids. BREP stays within the OCCT topology model used by build123d and Gmsh's OpenCASCADE kernel, so it is the preferred open-source geometry handoff.

After export, $_{\text{roundtripall}}$() re-imports BREP and requires:

- expected solid count;
- every solid valid and manifold;
- total volume relative error < 1e-12=.

BREP should not be expected to carry the same XCAF assembly labels as STEP. The geometric solids and their validated counts are the contract.

STEP

STEP remains the exchange format for tools such as FreeCAD, SOLIDWORKS, and ANSYS. Every STEP export explicitly requests millimetres, p-curves, and the highest documented precision mode:

```
export_step(
    shape,
    path,
    unit=Unit.MM,
    write_pcurves=True,
    precision_mode=PrecisionMode.GREATEST,
)
```

The code does not request an undocumented AP schema. Re-import requires the expected solid count, valid/manifold solids, and volume relative error < 1e-6=. Labels are reported when present; missing labels are a warning because ring_A.step, hole_band.step, and ring_B.step are mandatory label-independent fallbacks on a fully passing STEP run.

Why p-curves and precision do not guarantee equivalence

P-curves provide a surface-parameter representation of trimming edges, and PrecisionMode.GREATEST asks the writer for its highest documented exchange precision. Neither option can guarantee that a receiving STEP importer reconstructs every cone/cylinder intersection identically. The only defensible acceptance test is re-import followed by topology and measurement checks.

The strict CAD acceptance snapshot records native BREP volume errors near numerical noise and larger STEP errors in bodies containing the feed intersection. Treat params.json from the actual run as the current evidence; the measured values are not a universal STEP tolerance claim.

Export set

export$_{all}$() stages:

Artifact	Contents
film_unsplit.step / .brep	One full-360 fluid solid
film_zones.step / .brep	Three coincident axial fluid bodies
ring_A.step, hole_band.step, ring_B.step	Label-independent zone files
context_assembly.step	Separate journal and bushing context
geometry_half_debug.step	Optional geometry-only half; forbidden for rotating CFD
film_thickness_map.png	Exact same-z radial-thickness diagnostic
params.json	Inputs, derived data, validation, versions, hashes, status
requirements.txt	Successful-run dependency pins

The context assembly is not fluid and the debug half is not a CFD symmetry model.

Transactional publication and strict failure

The main routine writes the batch to a temporary staging directory beside the requested output directory. Native BREP is round-tripped first and copied as a trusted fallback only after passing. STEP is then audited separately.

If STEP fails, $_{publishbrepfallback}$() retains only trustworthy native/diagnostic artifacts and $_{discardpublishedsteps}$() removes stale program-generated STEP files. The command exits nonzero and params.json says overall: FAIL. This prevents a previous STEP file from being mistaken for output of the failed strict run.

When rejected exchange files are needed for inspection, --retain-failed-step atomically writes the complete batch and REJECTED.json to the sibling <outdir>.rejected-step/ directory. The manifest records the strict round-trip measurements and hashes, sets solve_eligible: false, and marks the batch as diagnostic only. The trusted output directory remains STEP-free.

FreeCAD inspection

```
freecad out/strict_default/film_unsplit.brep
freecad out/strict_default/film_zones.brep
freecad out/strict_default.rejected-step/film_unsplit.step
```

The unsplit file should import as one solid; the zones file should contain three solids with coincident split faces. Visual inspection cannot verify a 20 micrometre gap; the numerical checks remain authoritative. Native BREP, not the rejected STEP, feeds the Gmsh preflight.

Meshing

Meshing Manual — Eccentric Conical Bearing

Source: `docs/meshing/index.org`

Purpose

This directory explains the meshing stages that sit between the validated CAD fluid volume and any later CFD case. The stages deliberately answer different questions:

1. Does Gmsh/OpenCASCADE import the native BREP without changing its geometry?
2. Can Gmsh create a sound first-order boundary triangulation on that BREP?
3. Can the no-port film be represented by an analytic, periodic Hex8 mesh?
4. Can the feed be reduced to a pressure patch while retaining the Hex8 film?
5. Can the real central feed be represented by one conformal, connected Prism6 mesh with an open film/tube junction?

No file in this directory defines viscosity, rotation speed, cavitation, turbulence, pressure boundary values, or a CFD solver.

Pipeline map

Stage	Input	What Gmsh creates	What Python creates	Principal result
BREP preflight	native unsplit/zoned BREP	no mesh	validation records	trusted OCC import and conformal zones
Surface smoke	fragmented millimetre BREP	2D Tri3 boundary mesh	classification and audits	boundary-mesh feasibility, no volume cells
Structured Hex8	`params.json`	no generated mesh	all nodes, Hex8 and Quad4 connectivity	all-Hex no-feed regression baseline
Surface-inlet Hex8	`params.json`	no generated mesh	inflated Hex8 film and pressure-patch partition	all-Hex reduced-domain hydrodynamic mesh
Layered Prism6	`params.json` plus native unsplit BREP	2D unwrapped Tri3 master only	all 3D nodes, Prism6 and boundary connectivity	full ported fluid mesh

The distinction between “Gmsh stores a mesh” and “Gmsh generates a mesh” is important. The two 3D pipelines register already-complete NumPy arrays as Gmsh discrete entities. Neither calls `gmsh.model.mesh.generate(3)`.

Invariants shared by the pipelines

1. Geometry and coordinates
   - The bearing axis is global `Z`.
   - The model is full 360 degrees. A diametral symmetry mesh is not a valid rotating-journal CFD replacement.
   - CAD and analytic construction parameters are millimetres.
   - Solver mesh coordinates are metres. The factor `0.001` is applied exactly once and recorded in reports/manifests.
   - Film thickness means same-`z` radial clearance along a bearing-centred ray, not shortest surface-normal distance.
   - The circumferential coordinate starts at the `-Y` minimum-gap ray. The final circumferential cell connects directly to index zero; no duplicate `2*pi` node or seam boundary is retained.

   See generate$_{\text{mesh}}$ and build$_{\text{prismmesh}}$.

2. Trust and provenance
   - Input paths, hashes, coordinate units, volumes, and bounding boxes are checked before downstream work.
   - Entity tags and return order are never treated as semantic names. Geometry, centre of mass, bounding boxes, support type, and adjacency classify bodies and faces.
   - Validation failures raise and publish a failure report. They do not relax OCC tolerances or silently substitute approximate geometry.

- Solver-eligible artifacts are staged and atomically replace the old output directory only after all mandatory checks pass.

The common implementation is in require, $\text{make}_{\text{stagingdirectory}}$, and $\text{atomic}_{\text{replacedirectory}}$.

Master data models

Type	Owner	Meaning
`CadReference`	BREP preflight	frozen CAD measurements used to classify and validate OCC entities
`VolumeRecord` / `SurfaceRecord`	BREP preflight	geometry-derived OCC inventory, independent of imported tag order
`MeshData`	no-port Hex8	immutable canonical points, Hex8, boundary quads, logical indices, metrics and metadata
`MasterMesh`	ported Prism6	immutable unwrapped `(u,z)` Tri3 mesh, disk mask, rim/seam/partition edges
`PrismMesh`	ported Prism6	immutable SI points, Prism6, external faces, mouth faces, fields and provenance

Source links:

- CadReference
- SurfaceRecord
- MeshData
- MasterMesh
- PrismMesh

Arrays in the two production mesh data models are made read-only. Canonical connectivity is one-based because the same arrays are registered with Gmsh; VTU writers subtract one when handing connectivity to `meshio`.

Choosing the mesh

Use the no-port structured Hex8 mesh as the regular `theta`, `z`, and through-gap regression reference.

Use the surface-inlet Hex8 mesh when the intended model applies supply pressure directly on the stationary bore and must exclude feed-passage head, wall friction, and jet dynamics. Its inlet is a mesh-aligned Quad4 patch rather than a resolved cylindrical drilling. It retains structured periodic connectivity but uses locally clustered, nonuniform theta/z coordinates around that patch.

Use the layered Prism6 mesh for the actual central feed. The circular mouth is naturally represented by an unstructured Tri3 master mesh. Sweeping that master creates layered prisms without forcing a fragile all-Hex block topology around a branching circular passage. The no-port baseline is retained rather than being weakened or repurposed.

Once a ported `mesh_arrays.npz` has passed, continue with Solver-neutral interchange to construct clean MSH/CGNS files. That stage transfers the existing mesh; it does not belong in the mesh-generation choice itself.

Implementation chains

Pipeline	Function chain	Regression
BREP preflight	$\text{load}_{\text{reference}} \rightarrow {}_{\text{runoccchecks}}$	native BREP test
Surface smoke	${}_{\text{runmesh}} \rightarrow \text{validate}_{\text{mesh}}$	surface-smoke integration
No-port Hex8	$\text{generate}_{\text{mesh}} \rightarrow \text{validate}_{\text{analyticmesh}} \rightarrow \text{add}_{\text{discretemodel}}$	analytic/topology test
Surface-inlet Hex8	$\text{build}_{\text{surfaceinletmesh}} \rightarrow \text{validate}_{\text{surfaceinletmesh}} \rightarrow \text{add}_{\text{discretemodel}}$	surface-inlet regression
Ported Prism6	$\text{rim}_{\text{coordinates}} \rightarrow \text{build}_{\text{mastermesh}} \rightarrow \text{build}_{\text{prismmesh}} \rightarrow \text{validate}_{\text{topology}}$	mouth/topology test

Native BREP Preflight in Gmsh/OpenCASCADE

Source: `docs/meshing/brep-preflight.org`

Meshing index · Next: surface smoke · Geometry manual · Operations manual

Question answered by this stage

The preflight proves that Gmsh's OpenCASCADE kernel sees the native OCCT BREP as the same geometry that the CAD program validated. It does not generate any mesh. Its canonical implementation is gmsh$_{\text{breppreflight}}$.py, entered through run$_{\text{preflight}}$.

The inputs are the one-solid fluid BREP, the three-zone BREP, and `params.json`. PreflightInputs defines their defaults. load$_{\text{reference}}$ converts the CAD record into an immutable CadReference.

Import contract and strict OCC settings

Before import, the code checks:

- each BREP SHA-256 agrees with `params.json`;
- the CAD-side BREP round-trip record reports the expected solid count, validity, manifold status, and essentially exact native volume;
- coordinate units are known.

Every OCC option that could heal, sew, simplify, or fuzzy-match geometry is set explicitly to zero. `Geometry.OCCScaling` is the only intentional transform. See configure$_{\text{occoptions}}$ and the `STRICT_OCC_OPTIONS` table near the top of the source.

This matters because a preflight is evidence, not a repair operation. A model that needs automatic fixing must fail here rather than appear trustworthy after an undocumented topology change.

Geometry-derived classification

Imported entity tags are incidental. The classifier uses geometric facts.

1. Volumes

   inventory$_{\text{volumes}}$ records OCC mass, centre of mass, and bounding box. Then classify$_{\text{volumes}}$ assigns `ring_A`, `hole_band`, and `ring_B` from centre-of-mass `z` and expected axial spans. It demands exactly one body in each interval and checks the per-zone native volumes.

2. Surfaces

   inventory$_{\text{surfaces}}$ adds support type, area, centre, bounding box, adjacent volumes, boundary-curve types, and OCC support properties. classify$_{\text{surfaces}}$ then identifies:

   - the two shared axial interfaces at `z1` and `z2` from plane position and two-volume adjacency;
   - axial outlets from plane position;
   - the pressure inlet from its centre and support plane;
   - the journal, bore, and feed walls from support axis, centre, and radius.

   The final boundary audit is validate$_{\text{boundaries}}$. The inlet normal is not inferred only from plane coefficients: an inside/outside probe verifies that the `-Y` side is fluid and the `+Y` side is exterior.

Fragmentation and conformal zone interfaces

The imported zone bodies are passed to `gmsh.model.occ.fragment`. Both return objects are used:

- output dimension/tags;
- source-to-output mapping.

validate$_{\text{fragmentmapping}}$ requires each source zone to map to exactly one final 3D body, requires the three mapped bodies to be distinct, and rejects any extra intersection/sliver volume.

After synchronization, validate$_{\text{zonemodel}}$ reclassifies the bodies and surfaces. Each internal axial plane must be one surface adjacent to exactly the two expected zones. That is the OCC topology needed for a later conformal mesh; three touching volumes without shared subshapes would not be sufficient.

The code also runs a non-production comparison with `Geometry.OCCBooleanSimplify=1`. Any topology difference is reported, while the published model remains the strict simplify-off result.

Disk round trips and unit scaling

The preflight writes `film_zones_fragmented.brep` in millimetres, clears to a fresh model, reimports the file with `Geometry.OCCScaling=1`, and repeats volume, surface, classification, adjacency, and topology checks.

It then creates an SI copy by importing the millimetre file with `Geometry.OCCScaling=0.001`. Length, area, and volume must scale as `1e-3`, `1e-6`, and `1e-9`. The SI BREP is itself written, reimported with scaling reset to one, and checked again. This proves that the file contains SI numeric coordinates and prevents accidental double scaling downstream.

The complete sequence is in $_{runocccheck s}$.

Independent OCP validation

validate$_{brepwithocp}$ reimports both Gmsh-written BREPs through build123d/OCP. It checks three valid, manifold solids, total and per-zone volumes, and maximum OCCT vertex/edge/face tolerances. Tolerances are converted back to millimetres and compared with the minimum radial gap, rather than accepted merely because OCC can load the file.

Outputs and failure semantics

Successful output includes:

- `film_zones_fragmented.brep` in millimetres;
- `film_zones_SI.brep` in metres;
- `brep_manifest.json` with explicit units and hashes;
- volume and surface CSV inventories;
- `preflight_report.json` and the Gmsh log.

The stage writes into a sibling staging directory. Only $_{publishpreflightsuccess}$ atomically publishes a successful bundle. The failure path publishes only a failure report/log bundle through $_{publishfailurebundle}$.

Test evidence

test$_{nativebreppreflight}$ runs both strict parameter cases. It asserts that no mesh was generated, the fragmented model has three bodies, SI area/volume scaling is correct, and all expected files exist. The same test immediately feeds the published BREP into the surface-smoke stage, making the BREP/report/hash contract executable.

Scope

Preflight establishes native-BREP import and shared OCC topology. It does not establish mesh feasibility, volume-cell quality, CFD boundary conditions, or an all-Hex decomposition.

Gmsh 2D Surface-Mesh Smoke Test

Source: `docs/meshing/surface-smoke.org`

Meshing index · Previous: BREP preflight · Topology and quality · Operations manual

Purpose and boundary

gmsh$_{surfacesmoke}$.py answers a narrow question: can Gmsh produce a valid first-order triangular mesh on every classified boundary and shared axial interface of the fragmented BREP?

It calls `gmsh.model.mesh.generate(2)` exactly once. It never calls `generate(3)`, and it asserts that the final model contains zero 3D elements. The output is therefore a boundary-mesh smoke test, not a CFD volume mesh.

Consuming the preflight contract

load$_{preflightcontract}$ requires:

- a preflight report with `overall: PASS`;
- a SHA-256 match for `film_zones_fragmented.brep`;
- an explicit millimetre coordinate contract;
- a manifest saying that the downstream scale-to-metres factor is `0.001`.

The BREP is imported with `Geometry.OCCScaling=0.001`. Geometry, nodes, and mesh sizes inside the Gmsh model are consequently SI. User-facing size arguments stay in millimetres and are converted once in configure$_{meshsizes}$.

Geometry and physical-group reuse

The script reuses the preflight's geometry classifiers instead of inventing a second tag convention. $_{runmesh}$ imports the BREP and calls validate$_{zonemodel}$ before any mesh is generated.

create$_{physicalgroups}$ then creates external diagnostic groups:

Group	Meaning
`journal_wall`	all three zoned journal surface entities
`stationary_wall`	bore and feed-passage wall entities
`pressure_feed`	remote circular inlet
`axial_end_z0` / `axial_end_zL`	axial fluid ends
`ring_A` / `hole_band` / `ring_B`	the three material volumes

The two axial split surfaces are also named `internal_interface_z1` and `internal_interface_z2` for inspection. Their manifest flag says `internal: true`. They are not future OpenFOAM wall patches; a conformal volume mesh must keep them internal.

Explicit mesh sizing

The default global length is 0.75 mm and the port length is 0.20 mm. Both are converted to metres before being passed to Gmsh. A Distance field samples the feed-cylinder surface and inlet/feed boundary curves. A Threshold field blends from the port size to the global size over the requested refinement radius.

Implicit point, curvature, and boundary-extension sizing are disabled. This makes the field, rather than hidden defaults, responsible for the size transition. See configure$_{meshsizes}$.

What is validated

collect$_{surfaceelements}$ walks each classified surface entity and accepts only first-order three-node triangles. validate$_{mesh}$ then checks:

- every classified surface has elements;
- element tags are unique across surfaces;
- every `minSICN` is finite and positive;
- every triangle has positive finite area;
- Gmsh reports no duplicate nodes;
- every 2D element belongs to the classified surface inventory;
- no 3D elements exist;
- total element count stays below the user gate;
- node bounding box is in metres;
- every physical group is nonempty;
- each axial interface is one meshed surface adjacent to two volumes;
- the inlet circle has at least 48 Line2 segments;
- triangulated inlet area is within 0.5 percent of the OCC area.

Quality statistics are deliberately descriptive: minimum, first percentile, fifth percentile, median, and mean `minSICN` are recorded per boundary group. The narrow annular faces can produce anisotropic triangles, so this stage uses strict positivity rather than an arbitrary high isotropic-quality threshold.

Outputs

export$_{surfacemeshes}$ writes:

- `surface_mesh.msh`: Gmsh 4.1 binary;
- `surface_mesh_ascii.msh`: Gmsh 2.2 ASCII;
- `surface_mesh.vtk`;
- `physical_groups.json`;
- `surface_quality.csv`;
- `surface_mesh_report.json` and `gmsh_surface_mesh.log`.

Publication follows the same staging/atomic-replacement discipline as the preflight. See $_{publishsmokesuccess}$ and $_{publishfailure}$.

PASS scope

PASS covers boundary triangulation, shared split entities, inlet resolution, naming, and SI scaling. It does not establish 3-D connectivity, through-gap layers, or the film/feed mouth.

The end-to-end regression is test$_{nativebreppreflight}$; the CLI path is run$_{surfacesmoke}$.

Analytic Full-360 No-Port Structured Hex8 Mesh

Source: `docs/meshing/structured-hex-no-port.org`

Meshing index · Ported Prism6 mesh · Topology and quality · Clearance mathematics · Operations manual

Role of the baseline

$\text{structured}_{\text{hexnoport}}$.py is the frozen all-Hex reference for the complete eccentric conical film without a feed drilling. It isolates the thin-film discretization from the branch topology of the port. The ported pipeline is separate; adding a fake cap or forcing a circular junction into this tensor-product topology would weaken the baseline.

Python/NumPy calculates every point, Hex8, boundary Quad4, and logical index. Gmsh stores, validates, displays, and exports the completed arrays. It does not generate any 2D or 3D element in this pipeline.

Analytic film mapping

The supported baseline fixes the journal centre to `(0,-e)`. Theta is measured from `-Y`:

```
d(theta) = (sin(theta), -cos(theta))
Rj(z)    = Rm + (L/2-z) tan(gamma)
Rb(z)    = Rj(z) + c
rho_j    = e cos(theta) + sqrt(Rj(z)^2 - e^2 sin(theta)^2)
rho      = rho_j + xi (Rb-rho_j)
P        = (rho sin(theta), -rho cos(theta), z)
```

The arrays use `theta_j` for `j=0..nTheta-1`, `z_k` for `k=0..nAxial`, and `xi_i` for `i=0..nGap`. There is no duplicate `theta=2*pi` node. $\text{generate}_{\text{mesh}}$ builds coordinates in millimetres conceptually and multiplies by `0.001` while populating the canonical SI point array.

The `xi` interpolation is bearing-centred radial interpolation at the same `z`. It is not a surface-normal offset. The extreme targets remain `c-e` and `c+e`; the normal distances of parallel cones would include `cos(gamma)` and are not used for layer placement.

Canonical MeshData and indexing

MeshData owns:

- `points_m: float64[N,3]`;
- `hexes: uint64[M,8]`, one-based;
- four outward `boundary_quads` arrays;
- logical `(theta, axial, gap)` cell indices;
- one-based node/cell tags;
- cell centres and metric arrays;
- immutable metadata recording units, resolution, and solve eligibility.

The node tag function is node_{tag}. The last circumferential cell uses `(j+1) mod nTheta`, so its far face references the original index-zero nodes directly. Periodicity is therefore connectivity, not a solver boundary condition.

Each Hex8 is ordered bottom-to-top in `xi` with the two circumferential and two axial directions arranged for positive Jacobians. The boundary quads are ordered outward for journal, stationary bore, `z=0`, and `z=L`.

Analytic and topology validation

$\text{validate}_{\text{analyticmesh}}$ checks exact counts and canonical dtypes/shapes, finite values, no duplicate nodes/cells, cone residuals, ray intersection, monotone `xi` layers, cardinal gaps, positive signed and Gauss volumes, face-pyramid positivity, and quality gates.

$\text{validate}_{\text{topology}}$ canonicalizes all six faces of every hex and proves:

- every face has one or two incident cells;
- one-incident-cell faces equal the disjoint union of the four boundary groups;
- internal owner pairs exactly equal the expected tensor-product neighbours;
- there is one connected region;
- last-to-first theta faces match and are absent from boundary patches.

$\text{validate}_{\text{boundaryorientation}}$ checks each face both against its owner-cell centre and against the analytic journal/bore/end normal.

Metrics

compute$_{\text{custommetrics}}$ works in chunks and records:

- signed face-pyramid volume and cell centre;
- independent 2x2x2 Gauss volume and minimum determinant;
- aspect ratio;
- maximum face non-orthogonality;
- maximum skewness;
- minimum face-pyramid volume.

Gmsh independently supplies `minSICN`, `minDetJac`, and element volume in add$_{\text{gmshqualitymetrics}}$. Gmsh and canonical total volumes must agree. A small positive `minSICN` is reported but is not by itself a rejection condition: resolving a 20–80 μm gap with long circumferential/axial cells is intentionally anisotropic.

Registering a completed mesh with Gmsh

add$_{\text{discretemodel}}$ creates:

- four closed discrete curve entities for the journal/bore rings at both axial ends;
- four discrete boundary surfaces with signed curve boundaries;
- one discrete volume bounded by those surfaces.

It bulk-adds nodes to the volume, Line2 elements to the ring curves, Quad4 elements to boundary surfaces, and Hex8 elements to the volume. It verifies Gmsh element types 1, 3, and 5, then calls `reclassifyNodes`. Stable physical IDs are `101–104` for boundaries and `201` for `fluid`. Ring physical groups are diagnostic and keep Line2 elements in MSH2 while `Mesh.SaveAll` remains off.

This discrete topology tells Gmsh how the supplied mesh entities relate. It is not a request for Gmsh to regenerate the mesh.

Exports and round trips

Every `nGap` case writes:

- `structured_hex.msh` (Gmsh 4.1 binary);
- `structured_hex_openfoam.msh` (Gmsh 2.2 ASCII);
- `volume_hex.vtu` and `boundary_quads.vtu`;
- `mesh_arrays.npz`, the exact canonical provenance copy;
- ElementData `.pos` files and reports.

validate$_{\text{gmshroundtrip}}$ reopens each MSH and compares physical IDs/entity membership, SI coordinates, oriented Line2/Quad4/Hex8 connectivity, type counts, bounding box, determinant, and volume. validate$_{\text{vturoundtrip}}$ and validate$_{\text{mesharraysroundtrip}}$ require exact VTU/NPZ coordinates, connectivity, fields, and patch IDs.

Visualization artifacts are not solver meshes

write$_{\text{visualizationmodels}}$ creates two deliberately separate views:

- an exact-coordinate cutaway with an approximately 60-degree cell-aligned wedge omitted;
- a gap-exaggerated mesh that holds the journal wall fixed and scales only the through-gap displacement.

Both are named and manifested as non-solver artifacts. The exaggerated VTU stores `solve_eligible=0`, `distorted_geometry=1`, and the gap scale. True validation always uses the canonical mesh, never the exaggerated coordinates.

Convergence and optional OpenFOAM audit

theta$_{\text{convergence}}$ requires monotonically decreasing faceted-volume error with circumferential refinement. write$_{\text{convergence}}$ also proves that subdividing the same linear through-gap geometry does not change total faceted volume.

If available, `gmshToFoam` and `checkMesh` are audited by audit$_{\text{openfoam}}$. Auto mode records a skip when they are absent; required mode fails. This is only mesh conversion/inspection, not CFD execution.

Regression links

- Geometry, orientation, counts, and seam: test$_{\text{analyticgeometryorientationcountsandperiodicseam}}$
- Gap subdivision and theta convergence: test$_{\text{gapsubdivisioninvarianceandthetaconvergence}}$
- Gmsh/VTU/NPZ and non-solver markers for both strict cases: test$_{\text{alltruescaleroundtripsandvisualizationmarkers}}$

- Atomic failure behavior: test$_{\text{failureatomicallyremovesstagedsolveroutputs}}$

The top-level driver is run$_{\text{structured}}$.

Structured Hex8 Film with a Surface Pressure Inlet

Source: `docs/meshing/structured-hex-surface-inlet.org`

No-port Hex8 baseline · Full-passage Prism6 branch · Topology and quality · Clearance mathematics

Model boundary

structured$_{\text{hexsurfaceinlet}}$.py constructs the full-360 eccentric conical film as first-order Hex8 cells. It does not include the radial drilling, bushing-wall passage, or external inlet stub in the fluid volume. Instead, a set of outer-film Quad4 faces on the `+Y` stationary bushing bore is named `pressure_feed`.

This is the production branch for a film-only hydrodynamic study in which the supply condition is applied directly at the bore opening. It deliberately excludes feed-passage hydrostatic head, wall friction, developing pipe flow, and inlet-jet dynamics. Those effects belong to the retained full-passage Prism6 branch.

The pressure patch is on the stationary bushing bore, not on the journal. The journal is the moving inner wall and remains geometrically and topologically continuous.

Fluid control volume, not metal assembly

The solver mesh represents only the lubricant. Its annular shape therefore surrounds an empty core: in the physical bearing that core is occupied by the journal, but journal metal is not part of the fluid cell zone. Filling the core with cells would model the journal as lubricant.

A skin or wireframe rendering of the mesh shows the one-owner exterior faces, even though the interior contains Hex8 volume cells. The visible surfaces have these meanings:

- the inner annular skin is `journal_wall`;
- the outer annular skin is `stationary_wall` except where it is partitioned as `pressure_feed`;
- the two annular end faces are `axial_end_z0` and `axial_end_zL`.

The pressure inlet is not represented by deleting faces. `pressure_feed` faces remain the exterior faces of their adjacent Hex8 cells and become an inlet when the solver assigns the pressure boundary condition to that named group. This is the finite-volume representation of a surface opening; it is not a resolved drilling, recess, or fluid tube.

Journal and bushing solids may be exported separately for visual context. They must not be fused with this mesh or imported as additional fluid regions.

Analytic Hex8 film

The physical nodes use the same cone/ray law and periodic tensor connectivity as the validated no-port baseline. Theta is measured from the `-Y` minimum-clearance ray:

```
d(theta) = (sin(theta), -cos(theta))
Rj(z)    = Rm + (L/2-z) tan(gamma)
Rb(z)    = Rj(z) + c

q        = ex sin(theta) - ey cos(theta)
rho_j    = q + sqrt(Rj(z)^2 - ex^2 - ey^2 + q^2)
rho      = rho_j + xi [Rb(z)-rho_j]
P        = (rho sin(theta), -rho cos(theta), z)
```

The canonical arrays contain no duplicate `theta=2*pi` nodes. The final circumferential Hex8 block references the index-zero nodes directly, so the full model has no seam patch. Geometry is evaluated from millimetre inputs and converted to metres exactly once for mesh export.

The surface-inlet branch remaps the theta and z coordinates before evaluating this law, inflates the xi coordinates, and reclassifies selected `xi=1` boundary faces. It does not remove nodes, alter logical Hex8 connectivity, or create a recess into the film.

Cubic theta/z inlet clustering

The logical grid remains tensor-product, but its physical theta and z coordinates are nonuniform. On either side of the feed-centre coordinate, normalize distance from the centre to `d in [0,1]` and apply

```
w(d) = (1-s) d + s d^3.
```

Theta is anchored at `0, pi, 2*pi` and z at `0, z_h, L`. The default `s=0.82` clusters lines smoothly about `theta=pi` (the `+Y` feed direction) and `z=z_h`. `s=0` recovers uniform spacing; validation requires `0 < s < 1=`, strict monotonicity, and exact endpoint/feed-centre anchoring.

Every theta line still spans the full axial interval and every z line spans the full circumference. This preserves one conforming structured block, periodicity, cell count, and all-Hex8 connectivity without hanging nodes. Clustering improves the footprint by redistributing the existing lines rather than globally multiplying the mesh.

Symmetric 5:1 through-gap inflation

The `xi` coordinates use the same symmetric geometric distribution at every `(theta,z)` location. With `--gap-inflation-ratio 5`, the largest central film cell is five times the thickness of either wall-adjacent cell. The fractions are positive, sum to one, and mirror about `xi=0.5`.

Both walls are therefore resolved:

- `xi=0`: moving `journal_wall`;
- `xi=1`: stationary bore, partitioned into `stationary_wall` and `pressure_feed`.

Inflation changes only interior layer positions. The analytic journal and bore surfaces, local radial gap, periodic connectivity, and continuous film volume are unchanged. This is same-`z` radial layering, not a conical surface-normal offset.

Mesh-aligned pressure patch

The nominal feed is a 4 mm diameter circle centred at `+Y` and `z=z_h`. A logically tensor-product Hex8 mesh cannot represent that circle exactly while keeping every external face a complete Quad4. The implementation therefore selects whole clustered outer-wall faces whose projected face centres satisfy the nominal circular footprint:

```
x_c^2 + (z_c-z_h)^2 ⩽ r_h^2,    y_c > 0.
```

The resulting boundary is a fine mesh-aligned staircase approximation. It is a pressure boundary patch, not a resolved cylindrical hole. Theta/z clustering or increasing `n_theta` and `n_axial` refines its boundary; increasing `n_gap` affects only the through-film layers.

The report records at least:

- selected Quad4 count;
- projected `x-z` area and relative error from `pi r_h^2`;
- actual three-dimensional Quad4 area seen by the solver;
- area-weighted centroid and offset from `(0,+Rb(z_h),z_h)`;
- equivalent projected diameter `sqrt(4 A_projected/pi)`;
- circumferential and axial resolution at the feed;
- the maximum distance of the mesh-aligned boundary from the nominal circle.

These metrics describe the actual solver boundary. The implementation must not label it an exact circular aperture. For the default parameter case at `n_theta=256, n_axial=96, s=0.82`, the accepted mesh has 390 connected inlet faces, `12.5106696912 mm^2` projected area, `0.443254%` absolute relative area error, and a `3.991125 mm` projected equivalent diameter. Its maximum rim radius error is `0.141383 mm`. The second accepted cone case has 392 faces, `0.138729%` area error, and `0.139494 mm` maximum rim error.

The production gates are 1% projected-area error and `0.16 mm` maximum rim error. Area agreement alone cannot bound the staircase, so both tests are mandatory. The generated report remains authoritative for a particular run.

At the default feed, adjacent cells are approximately `0.253195 deg` circumferentially, `0.221176 mm` in local bore arc length, and `0.112722 mm` axially. Their global width ratios are about 14.56 in theta and 14.36 in z. The largest consecutive-cell growth factors are about 1.029 in theta and 1.080 in z: the mapping changes spacing smoothly rather than creating an abrupt local-block transition. Pushing `s` toward one would further coarsen the remote bearing, so inlet-shape convergence should use `n_theta=≠=n_axial` studies rather than unbounded clustering strength.

Why the journal remains continuous

The pressure patch is produced by partitioning the original stationary-bore Quad4 set:

```
original stationary bore
    = stationary_wall union pressure_feed
stationary_wall intersect pressure_feed
    = empty
```

No journal face is removed or relabelled. The `journal_wall` therefore contains one Quad4 for every circumferential and axial film cell, including the region radially opposite the pressure patch. There is no tube cell, film/tube owner-neighbour interface, mouth cap, or internal feed boundary in this model.

Physical groups

ID	Name	Role
101	`journal_wall`	complete moving journal surface
102	`stationary_wall`	stationary bushing bore outside the inlet patch
103	`axial_end_z0`	axial outlet candidate at `z=0`
104	`axial_end_zL`	axial outlet candidate at `z=L`
106	`pressure_feed`	mesh-aligned pressure-inlet patch on the `+Y` bore
201	`fluid`	one connected Hex8 film region

There is no `feed_tube_wall`, `feed_mouth`, `mouth_cap`, circumferential seam, or `defaultFaces`. Patch names describe intended roles; this stage does not assign pressure values, journal speed, lubricant properties, or solver boundary conditions.

Validation contract

The implementation follows the topology and round-trip strategy of the no-port Hex8 baseline and adds inlet partition checks:

- exact point, Hex8, and Quad4 counts;
- one connected periodic Hex8 region with no duplicate seam nodes or faces;
- every volume face has one or two owners and none has more than two;
- the five boundary groups are pairwise disjoint and equal the complete one-owner face set;
- `stationary_wall union pressure_feed` exactly reproduces the no-port outer bore boundary;
- stored theta and z coordinates are strictly monotonic, match the canonical nodes, and are anchored at their endpoints and feed centre;
- local and global theta/z spacing ranges and ratios are reported;
- every inlet face lies on the analytic bore, faces outward, and is centred around the `+Y` feed location;
- the journal boundary is complete and disjoint from `pressure_feed`;
- analytic journal/bore residuals and minimum/maximum radial gaps pass;
- all Hex8 volumes, Jacobian determinants, and face pyramids are finite and positive;
- non-orthogonality, skewness, and Gmsh quality records pass the existing Hex8 gates;
- canonical cell volume converges to `pi L (2 c Rm + c^2)` and is invariant under gap subdivision;
- MSH 4.1 and MSH 2.2 reproduce coordinates, connectivity, physical names, IDs, and membership; VTU reproduces coordinates, connectivity, every numeric field, and physical IDs, with names carried by `physical_groups.json`; NPZ reproduces every canonical array exactly.

The native ported BREP contains the wall drilling and external stub, so its total volume is not the volume target for this reduced film-only model. The analytic annular volume and exact cone equations are the appropriate geometry contracts.

Relationship to the other mesh branches

Branch	Volume cells	Feed representation	Use
No-port Hex8	Hex8	none	frozen geometry, periodicity, quality, and convergence regression
Surface-inlet Hex8	Hex8	pressure patch on stationary bore	current film-only hydrodynamic model
Full-passage Prism6	Prism6	resolved drilling, wall passage, stub, and remote inlet disk	studies that retain feed-passage hydraulics

The new branch does not replace or weaken either validated baseline. It reuses tensor-product annular connectivity with nonuniform theta/z coordinates while changing the physical boundary partition required by the reduced-domain study.

Run and inspect

Default production family:

```
uv run python meshing/structured_hex_surface_inlet.py \
  --params out/strict_default/params.json \
  --outdir out/structured_surface_inlet_default \
  --n-theta 256 --n-axial 96 \
  --gap-levels 4 8 12 --preview-ngap 8 \
  --gap-inflation-ratio 5 \
  --inlet-cluster-strength 0.82 \
  --max-projected-area-relative-error 0.01 \
  --max-inlet-rim-error-mm 0.16 \
  --openfoam skip --no-gui
```

Gmsh:

```
uv run gmsh \
  out/structured_surface_inlet_default/nGap_08/structured_hex.msh

uv run gmsh \
  out/structured_surface_inlet_default/nGap_08/viz/pressure_feed_only.msh
```

ParaView on Wayland:

```
QT_QPA_PLATFORM=xcb paraview \
  out/structured_surface_inlet_default/nGap_08/volume_hex.vtu

QT_QPA_PLATFORM=xcb paraview \
  out/structured_surface_inlet_default/nGap_08/viz/pressure_feed_only.vtu
```

For the full mesh, use `Surface With Edges` and color by `gap_layer_index` or mesh quality. To inspect patch membership in context, open `boundary_quads.vtu` and color by `patch_id`; ID 106 is `pressure_feed`. The compact `pressure_feed_only.vtu` contains only inlet points and faces, so ParaView frames the near-circular patch directly. The authoritative status remains `manifest.json` and `mesh_report.json`; an interactive rendering is not a topology or clearance measurement.

Conformal Central-Feed Layered Prism6 Mesh

Source: `docs/meshing/layered-prism-central-feed.org`

Meshing index · No-port Hex8 baseline · Topology and quality · Clearance mathematics · Operations manual

Why Prism6 is the production ported topology

The real lubricant domain branches from a 20–80 µm conical film into a circular radial feed passage. A tensor-product all-Hex mesh is natural for the no-port film but not for this circular junction. The robust ported pipeline keeps a Tri3 master surface and explicitly sweeps it into first-order Prism6 cells.

This choice preserves controlled layers through the film and a conformal open feed without tetrahedra, hanging nodes, or a contrived hexahedral block split. The validated all-Hex case remains unchanged as the reference described in the no-port baseline.

The developed conical-surface idea has literature context in the project paper index, but the port, periodic-node collapse, and explicit Prism6 topology are implementation decisions made and validated in this repository. The cited paper solves a developed-surface Reynolds-equation FEM model; it does not resolve the three-dimensional feed passage.

The implementation is layered$_{prismcentralfeed}$.py. run$_{ported}$ is the top-level driver.

Input and BREP contracts

load$_{contract}$ reads resolved geometry from `params.json`, including the actual `y_feed_end`. It requires a PASS preflight report, millimetre BREP units, and one SHA shared by the current BREP, CAD record, and preflight record. Native volume, bounding box, and inlet placement must agree.

validate$_{nativebrep}$ independently imports the unsplit BREP with build123d and requires one valid, manifold solid with the recorded volume and six-coordinate bounding box.

The parameter contract accepts either `hole_radius` or `hole_diameter`. The radius branch is tested separately because `dict.get(key, expression)` evaluates the default expression eagerly in Python; using it here previously accessed a missing `hole_diameter` even when `hole_radius` existed. The implementation now branches on key presence before reading either value. See the radius/diameter contract regression.

The current BREP surface classifier explicitly rejects `eccentricity_ratio=0`. In a concentric case both cones share an axis, so the centre-distance heuristic cannot distinguish journal from bore. Silent misclassification is worse than a documented unsupported case; support must use residuals against both surface equations before this guard is removed.

The BREP is a validation reference, not the source of volume cells. No STEP, STL, or CAD volume meshing participates in the analytic construction.

Film geometry

For general journal offset `(ex,ey)` and theta measured from `-Y`:

```
d(theta) = (sin(theta), -cos(theta))
q        = ex sin(theta) - ey cos(theta)
rho_j    = q + sqrt(Rj(z)^2 - ex^2 - ey^2 + q^2)
rho_b    = Rb(z)
P(theta,z,xi) = J(theta,z) + xi [B(theta,z)-J(theta,z)]
```

PortedParams implements `Rj`, `Rb`, and the ray/journal intersection. Layer interpolation is the same-`z` radial clearance convention used by the CAD and no-port mesh. It is not shortest surface-normal clearance.

Construction is conceptually millimetric. The complete point array is multiplied by `0.001` once in $\text{build}_{\text{prismmesh}}$. All solver exports and cell volumes are therefore SI.

The unwrapped Tri3 master

1. Domain and axial partitions

   $\text{build}_{\text{mastermesh}}$ creates a Gmsh geometric model in the plane

   ```
   u = Rm theta,    0 ⩽ u ⩽ 2 pi Rm,    0 ⩽ z ⩽ L.
   ```

   Horizontal lines at `z1` and `z2` split the rectangle into `ring_A`, `hole_band`, and `ring_B`. The middle band contains two surfaces that share the same rim curves: the bore remainder outside the mouth and the feed disk inside it. The disk is intentionally meshed; it will become the internal film/tube interface.

2. Exact rim nodes and linear faceting

   $\text{rim}_{\text{coordinates}}$ places each rim vertex on both analytic supports:

   ```
   x = rh cos(alpha)
   z = zh + rh sin(alpha)
   y = +sqrt(Rb(z)^2 - x^2)
   theta = atan2(x,-y) mod 2 pi
   u = Rm theta.
   ```

   Each rim curve is forced to one Line2 between its intended endpoints. The vertices are exact cylinder/bore intersections, but the first-order edge between them is a chord. The report separates nodal residual from the between-node sagitta `rh[1-cos(pi/Nrim)]` and regular-polygon area

   $$A_N = \frac{1}{2} N_{rim} r_h^2 \sin\left(\frac{2\pi}{N_{rim}}\right).$$

   Increasing `--rim-segments` reduces chord and polygon-area error while increasing master triangles, feed prisms, memory, and format size.

3. Periodic seam

   The feed is centred near `theta=pi`, away from the unwrapped seam. Gmsh's periodic-curve API constrains the three right boundary curves to the three left curves by a translation of `2*pi*Rm`. After `generate(2)`, the code reads the slave/master node correspondence, replaces every slave tag by its master tag, compacts the node array, and maps `u` back into `[0,2*pi*Rm)`.

   This is the only mesh generation Gmsh performs in the ported pipeline: it creates the 2D first-order Tri3 master. The extracted MasterMesh stores points, consistently oriented triangles, disk mask, axial-zone field, rim loop, periodic seam edges, split edges, axial-end edges, and metadata.

The master edge census requires the collapsed seam and `z1/z2` lines to have two triangle owners. Only the `z=0` and `z=L` edges may remain external.

Film sweep

Every master triangle, including every disk triangle, is copied through the gap. Adjacent copies form a Prism6. The default `--gap-inflation-ratio 5` uses symmetric geometric cell fractions: the two wall-adjacent cells are smallest and the largest central cell is five times their thickness. The fractions remain positive, sum to one, and mirror about `xi=0.5`. Ratio `1` restores uniform `xi=i/nGap` spacing; one or two gap cells cannot express non-uniform two-sided inflation and therefore remain uniform.

This is wall-normal inflation across the lubricant film. It clusters nodes at both `journal_wall` and `bushing_bore_wall` without changing connectivity or the exact journal/bore coordinates. It does not create radial boundary-layer rings around `feed_tube_wall`; that requires a separately graded feed-disk master triangulation.

Triangle orientation is made consistent in the `(u,z)` master before any volume cell is constructed; volume cells are not "repaired" later with arbitrary per-cell node swaps. The coordinate construction is in symmetric$_{\text{gapcoordinates}}$().

At `xi=0`, reversing the master triangle produces the outward `journal_wall`. Because the disk was not removed, the journal contains one boundary triangle for every master triangle and remains continuous underneath the feed.

At `xi=1`:

- outside-disk triangles form `bushing_bore_wall`;
- disk triangles are not boundaries;
- those exact disk node tags are the feed tube's base.

Feed-tube sweep

For every feed-disk node, the tube retains `x` and `z`. Its bore coordinate is `+sqrt(Rb(z)^2-x^2)`, and its `y` coordinate interpolates to the recorded `y_feed_end` through `tube_layers` values of `eta`. The current grading map is `s(eta)=eta^tube_grading`; endpoints, strict monotonicity, and positive layer lengths are checked and the minimum/maximum sizes are reported.

At `eta=0`, build$_{\text{prismmesh}}$ returns the already-existing outer-film disk tags. It does not create and merge coincident nodes. Higher `eta` layers add new nodes. Sweeping disk Tri3s creates feed Prism6s; sweeping rim edges creates `feed_tube_wall`; and the final disk creates `pressure_feed` with outward `+Y` orientation.

Why the mouth is open, not capped

Every mouth key has exactly two incident cells—one film Prism6 and one feed Prism6—using identical node tags, and appears in no external boundary group. It is a shared feed-film interface and an internal owner-neighbour face, not a cap. validate$_{\text{topology}}$ also requires one connected cell graph and a path from `pressure_feed` into the film.

The focused regression is test$_{\text{mouthnodesfacesandcontinuousjournal}}$.

Canonical PrismMesh and fields

PrismMesh holds immutable:

- SI `float64` points and one-based `uint64` Prism6 connectivity;
- three Tri3 boundary groups and three Quad4 boundary groups;
- internal mouth triangles;
- node/cell tags and cell centres;
- film/feed counts, master/disk counts, and metadata;
- cell fields listed below.

Field	Meaning
`region_id`	0 film, 1 feed tube
`axial_zone_id`	0 ring$_A$, 1 hole$_{\text{band}}$/feed, 2 ring$_B$
`gap_layer_index`	film layer 0..nGap-1; -1 in feed
`gap_um`	exact same-z radial film gap; zero in feed
`theta_deg`	bearing-frame circumferential location
`minSICN` / `minDetJac` / `volume_m3` / `aspect_ratio`	quality and measure fields
`solve_eligible` / `distorted_geometry`	canonical safety flags

Physical groups

ID	Name	Role
101	`journal_wall`	moving wall, continuous under feed
102	`bushing_bore_wall`	stationary bore outside mouth
103	`axial_end_z0`	axial end
104	`axial_end_zL`	axial end
105	`feed_tube_wall`	stationary cylindrical feed wall
106	`pressure_feed`	remote inlet disk
201	`fluid`	one connected volume

There is no solver patch named `feed_mouth`, `mouth_cap`, `internal_feed`, or `defaultFaces`. The mouth may appear only in explicitly diagnostic files.

Discrete Gmsh model and validation

add$_{\text{discreteprismmodel}}$ creates six discrete surfaces and one discrete volume, bulk-adds all nodes, then adds Tri3, Quad4, and Prism6 elements. It verifies Gmsh types 2, 3, and 6 and creates the stable physical groups. It never calls `generate(3)`.

The validation stack includes:

- analytic cone, bore, cylinder, inlet, gap, and SI checks in validate$_{\text{geometry}}$;
- incidence/graph/boundary/seam/split checks in validate$_{\text{topology}}$;
- exact face counts for all six patches and equality of their sum with the complete external-face census;
- local outward winding for every external Tri3 and Quad4 in validate$_{\text{externalfaceorientation}}$;
- positive custom integration-point Jacobians and volumes;
- independent Gmsh `minSICN`, `minDetJac`, and volume from add$_{\text{gmshquality}}$;
- oriented exterior-boundary volume versus cell-sum volume;
- linear mesh volume versus native BREP;
- closest-point nodal probes against all six native BREP surfaces in validate$_{\text{nodesagainstbrep}}$.

The closest-point check measures nodal placement. It does not erase or hide the separate chord/sagitta error between first-order rim nodes.

Formats and publication

Each gap level writes Gmsh 4.1 binary, Gmsh 2.2 ASCII, VTU volume/boundary files, canonical NPZ arrays, physical-group metadata, quality CSV, and reports. Round trips are implemented in validate$_{\text{gmshroundtrip}}$, validate$_{\text{vturoundtrip}}$, and validate$_{\text{npzroundtrip}}$.

write$_{\text{visualizations}}$ creates exact-coordinate diagnostic subsets marked `solve_eligible=0`. The shared mouth diagnostic defaults to translucent wireframe so it cannot be mistaken for a cap. Diagnostic MSH files are reopened and checked by validate$_{\text{visualizationmsh}}$.

All gap levels are built under one staging directory. Only a complete family with invariant volume is atomically published. On failure, $_{\text{publishfailure}}$ replaces stale solver artifacts with `failure_report.json` and preserves logs by relative gap-level path.

Optional GUI launch occurs after the atomic replacement and outside the canonical failure/publication scope. open$_{\text{optionalgui}}$ turns FLTK/Wayland errors into warnings; a display failure cannot downgrade or delete an already validated mesh family.

The solver-neutral continuation from `mesh_arrays.npz` is documented in Solver-neutral interchange.

Extension boundary

A future quad-dominant branch could recombine suitable master triangles into a Hex8/Prism6 hybrid. It must preserve the same mouth node identity, two-incident cell interface proof, connected cell graph, and external physical boundaries; it is not required by the current robust Prism6 pipeline.

Regression links

- Analytic rim and closed loop: test$_{\text{analyticmouthrimandcloseddegreetwoloop}}$
- Symmetric film inflation: test$_{\text{symmetricthroughgapinflation}}$
- Periodic seam and split conformity: test$_{\text{periodicseamandaxialpartitionsareinternal}}$
- One region and complete boundaries: test$_{\text{onecomponentandcompletedisjointboundaries}}$

- Jacobians and Gmsh quality: test$_{\text{positiveprismjacobiansandgmshquality}}$
- Geometry and volume: test$_{\text{inletgeometryradialgapsandvolumeagreement}}$
- Both strict cases and all main formats: test$_{\text{allroundtripsandbothstrictcases}}$
- Diagnostic Gmsh files: test$_{\text{diagnosticmshfilesopeningmsh}}$
- Atomic failure cleanup: test$_{\text{failureatomicallyremovesstagedsolveroutputs}}$
- Every external face locally outward and reversed-face rejection: orientation regression
- Concentric-case guard: concentric rejection regression
- Post-publication GUI isolation: GUI warning-only regression

Topology, Quality, Round Trips, and Publication

Source: *`docs/meshing/topology-and-quality.org`*

Meshing index · Hex8 pipeline · Prism6 pipeline · CAD validation · Outputs and reports

Why validation is layered

The repository keeps four claims separate:

1. Native BREP geometry is valid and imports faithfully.
2. BREP surfaces can be triangulated by Gmsh.
3. Canonical 3D arrays have correct analytic geometry and topology.
4. Exported files reproduce those arrays and names.

A PASS at an earlier stage does not imply a PASS later. Conversely, the known strict STEP exchange failure does not invalidate native BREP or analytic mesh results. The stage map is in the meshing index.

Canonical face census

Connectivity is the source of truth for volume-mesh topology.

For each volume cell, local faces are extracted and converted to canonical unordered node keys. A Prism6 key begins with the face arity before its sorted node tags, so a three-node face can never collide with a four-node face. Incidence counts then mean:

Incident cells	Interpretation
1	exterior boundary face
2	conformal internal face shared by two cells
0	missing/disconnected declaration
>2	non-manifold overlap

Hex8 implementation: validate$_{\text{topology}}$. Prism6 helpers and implementation: $_{\text{facecensus}}$ and validate$_{\text{topology}}$.

Boundary completeness

The declared boundary groups must be pairwise disjoint. Their canonical keys must equal the complete set of one-incident-cell volume faces. This one equality detects several common failures at once:

- a missing patch face;
- an accidental internal cap exported as a wall;
- the same face assigned to two patches;
- an undeclared `defaultFaces`-like remainder;
- a hanging/nonmatching interface that appears as exterior faces.

Boundary winding is checked separately. For Hex8, each area vector must point away from the incident cell and agree with the analytic cone/end direction. For Prism6, validate$_{\text{externalfaceorientation}}$ requires every external Tri3 and Quad4 normal to point from its sole incident cell centroid toward the face centroid. The signed exterior-boundary volume remains a global conservation check, but it cannot hide locally reversed faces.

Connectivity and periodicity

1. Hex8 tensor-product graph

   The no-port mesh computes the complete expected neighbour list from logical `(theta,z,xi)` indices. Actual two-incident-cell face pairs must match it exactly. The final theta block must share its circumferential face with block zero, and those seam keys must not occur in any external patch.

Relevant functions: $\text{expected}_{\text{neighbourpairs}}$ and $\text{validate}_{\text{topology}}$.

2. Prism6 generic graph

   The ported mesh builds cell adjacency from every two-incident-cell triangular and quadrilateral face. A union-find pass in $_{\text{componentcount}}$ must return one connected component. Extruded master edges prove that the collapsed theta seam and the `z1/z2` partitions remain internal owner-neighbour faces through every gap layer.

   The pressure-inlet owners must be feed cells in the same component as film cells. Thus "the inlet exists" and "the inlet communicates with the film" are different checks, and both are required.

The shared-mouth owner-neighbour proof

The film/tube mouth is the most important local topology in the ported mesh. Each mouth Tri3 must:

- be found in the volume-face census;
- have exactly two incident cells;
- have one incident cell with `region_id` 0 and one with `region_id` 1;
- use the same node tags on both cells;
- be absent from all one-incident-cell boundary groups.

This proves an open, conformal fluid connection. The triangle is a shared internal owner-neighbour face, not a cap. A cap would have one incident cell and would appear in an external boundary group. The code and focused test are:

- Prism $\text{validate}_{\text{topology}}$
- $\text{test}_{\text{mouthnodesfacesandcontinuousjournal}}$

The same validation proves the journal remains continuous: `journal_wall` contains one triangle for every complete master triangle, including the feed disk footprint.

Geometry metrics and quality gates

1. Surface smoke Tri3

   The smoke stage computes Euclidean triangle area and asks Gmsh for `minSICN`. Every area and `minSICN` must be finite and positive. It reports distribution statistics rather than imposing a high isotropic threshold. See $\text{validate}_{\text{mesh}}$.

2. Hex8

   $\text{compute}_{\text{custommetrics}}$ uses oriented face area vectors and a 2x2x2 Gauss rule. Hard gates are:

   - positive face-based signed volume;
   - positive Gauss volume and minimum determinant;
   - agreement between the two total-volume calculations;
   - positive minimum face-pyramid volume;
   - maximum non-orthogonality at most 45 degrees;
   - maximum skewness at most 4;
   - finite Gmsh `minSICN`;
   - positive Gmsh `minDetJac` and element volume.

   Aspect ratio is reported, not capped, because the film is intentionally thin.

3. Prism6

   $_{\text{prismgaussmetrics}}$ evaluates the Jacobian at three triangle points and two through-prism Gauss points. Every custom determinant and integrated volume must be positive. Gmsh independently supplies `minSICN`, `minDetJac`, and volume for every Prism6.

   Quality statistics include minimum, p01, p05, median, p95, and maximum for:

   - all cells;
   - film prisms;
   - feed-tube prisms;
   - cells touching mouth nodes.

   Again, `minSICN` must be positive but need not exceed 0.1. A thin layered film can be valid and strongly anisotropic.

Volume comparisons

Different comparisons answer different questions:

Comparison	Purpose
canonical cell sum vs Gmsh element sum	independent implementation check
canonical cell sum vs oriented exterior boundary	connectivity/orientation conservation
linear mesh vs analytic no-port volume	circumferential faceting convergence
linear ported mesh vs native BREP	complete curved-domain approximation error
gap-level family volumes	subdivision invariance

The ported mesh also separates node-to-BREP closest-point residuals from rim sagitta. Closest point verifies analytic vertices; sagitta quantifies the linear chord between exact vertices. One must not be used to disguise the other.

Gmsh discrete entities

In the analytic 3D pipelines, Gmsh discrete entities attach topology and names to an already-complete mesh:

- `addDiscreteEntity` creates curves/surfaces/volume;
- `addNodes` receives canonical point tags and SI coordinates;
- `addElementsByType` receives Line2/Quad4/Hex8 or Tri3/Quad4/Prism6 arrays;
- `reclassifyNodes` assigns nodes to the lowest-dimensional discrete entities;
- stable physical groups are then added.

Round-trip strategy

1. Gmsh MSH

   Both production pipelines write Gmsh 4.1 binary and OpenFOAM-compatible Gmsh 2.2 ASCII. Fresh models reopen both files. Node tags may be renumbered, so coordinates establish a canonical node mapping. Oriented connectivity is then compared after geometric mapping, not by assuming imported tags/order.

   The audit checks element types/counts, physical IDs, names, entity membership, coordinates, connectivity, SI bounding box, and positive determinants/volumes. The ported audit also reconstructs mouth ownership and verifies the `region_id` ElementData view.

2. VTU

   `meshio` rereads the volume and boundary VTUs. Coordinates, cell type, connectivity, field lengths/values, and patch IDs must match canonical arrays. Prism6 is represented by meshio/VTK's `wedge` cell name.

3. NPZ

   The NPZ archive is the label-independent provenance copy. It is reloaded with `allow_pickle=False` and every array plus JSON metadata must be exactly equal.

4. Diagnostic formats

   Visualization-only meshes are validated separately and carry non-solver markers. In the ported pipeline, the binary Gmsh 2.2 cutaway and feed-boundary files are reopened and their element counts and `solve_eligible` flag are checked. In the Hex8 pipeline, cutaway connectivity and exaggerated-file markers are round-tripped independently.

Atomic publication

Each top-level run creates a sibling staging directory. All requested gap levels, format round trips, plots, reports, and optional external audits are completed there. A successful stage replaces the old output directory using `os.replace`. If replacement fails, the previous directory is restored.

Failure behavior is tested after solver-like artifacts already exist in the staging tree:

- Hex8 atomic failure test
- Prism6 atomic failure test

The published failure directory contains no stale solve-eligible MSH, VTU, or NPZ. The ported path retains diagnostic logs under their relative gap-level paths.

Report semantics

`overall` is stage-scoped. Interpret it with the recorded units, topology counts, quality minima/distributions, round-trip records, manifest, and OpenFOAM status; `SKIPPED` is valid when external tools are unavailable.

Solver-neutral interchange

Solver-Neutral Interchange

Source: `docs/interchange/index.org`

Question answered by this branch

The ported mesh already exists before this stage starts. Interchange asks a narrower question than meshing:

> Can the exact validated node coordinates, Prism6 connectivity, six external patches, and internal feed-film connection survive transfer into clean Gmsh and CGNS files without introducing a solver-specific model?

The canonical input is `mesh_arrays.npz` plus its manifest and report. STEP, BREP, and the older Gmsh files are evidence, not reconstruction inputs.

Current evidence boundary

Case	Static round trips	Fluent import	Permitted conclusion
`ported_prism_default/nGap_04`	MSH 4.1, MSH 2.2, and CGNS PASS	not run	`STATICALLY_VALIDATED_NOT_IMPORTED`
`ported_prism_default/nGap_08`	MSH 4.1, MSH 2.2, and CGNS PASS	not run	`STATICALLY_VALIDATED_NOT_IMPORTED`
20-degree, eccentricity-0.3 family	no retained interchange run	not run	path-driven CLI support; run before making a transfer claim

A successful CGNS write or round trip is not evidence that Fluent accepted the mesh. Only a live Fluent audit may produce `FLUENT_IMPORT_PASS`.

Reading map

Chapter	Main question
Canonical source contract	Why does interchange start from NPZ arrays, and what must be present before export?
Clean discrete reconstruction	How are the same arrays registered as one volume and six boundary entities without remeshing?
Static topology and geometry audit	Which connectivity, manifoldness, orientation, scale, and volume claims are proved before writing?
Round-trip validation	How are files compared when a reader renumbers nodes, cells, entities, and physical IDs?
Formats and CGNS structure	What do the three files contain, and why is CGNS not just another extension?
Fluent import-only audit	What must a live proprietary import prove, and what does the automation deliberately refuse to do?
Commands, reports, and Workbench handoff	How are cases exported, inspected, reported, and promoted after a live import?

Implementation entry points

- InterchangeInputs
- $\text{load}_{\text{canonicalcase}}$
- $\text{audit}_{\text{canonical}}$
- $\text{write}_{\text{cleanmeshes}}$
- $\text{audit}_{\text{roundtrip}}$
- $\text{run}_{\text{interchange}}$
- $\text{run}_{\text{fluentimportaudit}}$
- interchange regression tests

Scope boundary

This stage contains no material model, journal speed, supply pressure, turbulence, cavitation, thermal model, initialization, iteration, or result calculation. Generic patch roles are retained only so a later solver setup can make explicit physics decisions. It also makes no commitment to Fluent or OpenFOAM as the final solver.

Canonical Interchange Source

Source: `docs/interchange/canonical-source.org`

Why interchange starts from arrays

The mesh generator has already answered the geometry and meshing questions. Reconstructing the volume from BREP at this point would create a second mesh, not convert the validated one. Reading the existing `ported_prism.msh` would also make a format writer/reader the authority and could preserve an earlier file's views or entity layout.

The authoritative interchange source is therefore:

File	Role
`mesh_arrays.npz`	exact coordinates, cell connectivity, external faces, mouth faces, fields, and tags
`manifest.json`	stage status, units, cell type, counts, and solver-eligibility contract
`mesh_report.json`	volume, BREP error, boundary census, quality, and prior round-trip evidence
`physical_groups.json`	expected volume/boundary names and forbidden patch names

$\mathrm{load}_{\mathrm{canonicalcase}}$ requires all four. A valid NPZ beside a failed manifest is rejected; files cannot be mixed between cases merely because their array shapes are plausible.

In-memory contract

CanonicalCase binds the case path, reconstructed immutable `PrismMesh`, source manifest, and source report. CanonicalAudit adds the verified summary plus face-to-cell incidence needed by later round-trip orientation checks.

The relevant NPZ arrays are:

Array family	Shape/meaning
`points_m`	`N x 3` float64 coordinates already in metres
`prisms`	`C x 6` one-based Prism6 node tags
`boundary_tri_*`	ordered Tri3 faces for journal, bore, and pressure inlet
`boundary_quad_*`	ordered Quad4 faces for both axial ends and tube wall
`mouth_triangles`	shared feed-film faces; diagnostic topology, never a BC
`node_tags` / `cell_tags`	stable canonical identifiers
`cell_centres_m`	owner-centroid reference for local face orientation
`field_region_id`	film `0`, feed `1`
`field_volume_m3`	generator's independently checked cell volumes
`metadata_json`	unit, layering, region, and construction provenance

The loader accepts only the exact three Tri3 patch names and three Quad4 patch names. It checks float64 coordinates, six-node cells, one-based contiguous node tags, unique cell tags, valid node references, and the fields required by the interchange audit.

Why one-based tags remain in the canonical arrays

Gmsh and most solver formats use positive node identifiers; NumPy indexing is zero-based. The generator deliberately stores solver-facing connectivity as one-based tags and subtracts one only at coordinate lookup sites:

```
face_points = mesh.points_m[faces.astype(np.int64) - 1]
```

Keeping the conversion visible is less error-prone than letting each writer guess whether an array contains row indices or mesh tags. It also lets the same arrays pass directly to `gmsh.model.mesh.addElementsByType`.

Source checks before format construction

$\text{audit}_{\text{canonical}}$ does not assume that a previously published PASS remains sufficient. It rechecks:

- manifest/report PASS state, metre units, and Prism6 type;
- node, cell, mouth, and six patch counts against the NPZ;
- absence of duplicate coordinates and duplicate cells;
- positive recomputed Prism6 volume and Jacobian samples;
- agreement with the stored total volume and source report;
- the established native-BREP relative-volume envelope;
- complete face incidence, connectivity, manifoldness, and orientation.

This repetition is intentional. It catches later manual file replacement, schema drift, or an exporter being pointed at an incomplete case directory.

Hash provenance

The published `file_hashes.json` records SHA-256 for the canonical NPZ, source manifest, source mesh report, and each exported mesh. The hashes prove which source files participated in one export; they do not by themselves prove geometric correctness.

The exporter does not copy the old MSH files, so equality of their hashes is neither expected nor desirable. It constructs a clean model from the same arrays and validates semantic equivalence.

Selecting cases

Case selection is path-based rather than encoded as a geometry mode:

```
uv run python interchange/export_solver_neutral.py \
  --case-dir out/ported_prism_default/nGap_04 \
  --outdir out/interchange_default/nGap_04 \
  --fluent skip

uv run python interchange/export_solver_neutral.py \
  --case-dir out/ported_prism_e03_g20/nGap_04 \
  --outdir out/interchange_e03_g20/nGap_04 \
  --fluent skip
```

This is enough to support `nGap_04`, `nGap_08`, and the 20-degree/eccentricity 0.3 family without adding format-specific geometry switches. The case directory already contains the resolved geometry and mesh identity.

Regression boundary

The loader and canonical audit are exercised through the clean mini-mesh fixture in $\text{clean}_{\text{export}}$. Negative/reversed cells, missing patches, duplicate cells, and malformed group sets have focused rejection tests in $\text{test}_{\text{solverneutralinterchange}}$.py.

Clean Discrete Mesh Reconstruction

Source: `docs/interchange/discrete-model.org`

Reconstruction is not remeshing

$\text{write}_{\text{cleanmeshes}}$ creates a new Gmsh model container but does not generate a node or cell. It reuses $\text{add}_{\text{discreteprismmodel}}$, which registers completed arrays on discrete entities.

The distinction is important:

- `gmsh.model.mesh.generate(3)` would create a new mesh and violate identity;
- `addNodes` and `addElementsByType` serialize connectivity already proved by the generator;
- Gmsh supplies entity/group semantics and official writers, not meshing policy.

Entity model

The clean model contains exactly:

Dimension	Entity/group	Elements
3	`fluid`	all linear Prism6 cells, CGNS `PENTA_6`

Dimension	Entity/group	Elements
2	`journal_wall`	Tri3
2	`bushing_bore_wall`	Tri3
2	`pressure_feed`	Tri3
2	`axial_end_z0`	Quad4
2	`axial_end_zL`	Quad4
2	`feed_tube_wall`	Quad4

The Gmsh element type checks require `2` for Tri3, `3` for Quad4, and `6` for Prism6. These are first-order elements; there are no high-order nodes to project or reinterpret.

Registration order

The implementation performs four distinct operations:

1. create six discrete surface entities and one discrete volume;
2. add the unique canonical node set once on the volume entity;
3. add external surface elements and volume cells with separate element tags;
4. reclassify nodes, then attach stable physical groups and names.

A simplified extract is:

```
gmsh.model.mesh.addNodes(3, VOLUME_ENTITY, mesh.node_tags, mesh.points_m.ravel())
gmsh.model.mesh.addElementsByType(
    VOLUME_ENTITY, prism_type, mesh.cell_tags, mesh.prisms.ravel()
)
gmsh.model.mesh.reclassifyNodes()
gmsh.model.addPhysicalGroup(3, [VOLUME_ENTITY], tag=201, name="fluid")
```

Surface element tags start after the canonical cell tags so node, cell, and surface identifiers remain globally unique inside Gmsh. Entity tags and physical IDs are stable local interface values, but later round trips never assume another format preserves their numeric values.

Why the mouth is omitted

The shared feed-film interface is already represented by the face relation between two Prism6 cells. Registering its Tri3s as a surface entity would turn an internal owner-neighbour face into a named boundary candidate.

Therefore `mouth_triangles` are used only for validation. They are absent from `SURFACE_ENTITIES`, physical groups, `zones.csv`, and exported boundary sections. The journal triangles remain continuous beneath the disk; omission of a mouth BC does not omit journal geometry.

This design is proved in the ported-mesh chapter and rechecked in the interchange topology audit.

Why reuse the generator's registration function

The element ordering and physical-group contract already live in one tested function. Reimplementing them in the exporter would create two mappings that could diverge. Reuse stops at the discrete registration boundary: the exporter deliberately does not add the generator's ElementData views or copy its MSH files.

Immediately after registration the exporter asserts:

```
len(gmsh.view.getTags()) == 0
```

This is the clean-output distinction. The production visualization MSH is allowed to contain quality fields; the solver-neutral MSH/CGNS files contain only mesh and group definitions.

Physical groups versus elementary entities

Elementary entities organize elements inside the Gmsh model. Physical groups carry engineering names across output formats. The exporter needs both:

- separate surface entities prevent patch elements from being merged before naming;
- physical names become MSH physical names and CGNS boundary/family records;
- the single volume group prevents multiple unintended fluid zones.

Gmsh writes only elements belonging to physical groups when `Mesh.SaveAll` is zero. That behavior is useful here: every intended element is explicitly in one of the seven groups, so an unclassified element cannot silently leak into the file.

Rejected alternatives

1. Copy or rename an existing MSH

   This would preserve post-processing views and would not prove that a clean model can be reconstructed from the canonical source. Renaming a Gmsh MSH to a Fluent MSH is not conversion; the formats merely share an extension.

2. Write CGNS with generic HDF5 calls

   CGNS is a documented hierarchy and API contract, not "HDF5 with arrays in it." A hand-built approximation could look plausible to `h5ls` while omitting `Elements_t`, `ZoneBC_t`, family references, or required ranges. This project uses Gmsh's compiled official CGNS writer instead.

3. Use meshio for CGNS

   meshio remains useful for VTU diagnostics, but the selected CGNS path must preserve the six named boundary memberships in the file itself. The Gmsh writer/reader path already integrates physical groups and is validated by the same API used for the MSH round trips; another dependency adds no value here.

Static Topology and Geometry Audit

Source: `docs/interchange/static-topology-and-geometry.org`

Audit order

$\text{audit}_{\text{canonical}}$ runs before any writer is trusted. Its checks are ordered from cheap schema failures to connectivity and numerical geometry:

1. exact source counts and group names;
2. duplicate nodes/cells and cell validity;
3. complete face incidence and external shell manifoldness;
4. shared-mouth and global cell-graph connectivity;
5. local outward orientation for every external face;
6. bounding box, metre scale, cell volume, and BREP-relative error.

The output is a compact `CanonicalAudit` summary plus arrays mapping each declared patch face to its sole incident cell. Later file round trips reuse those incidence relations after mapping cells back to canonical signatures.

Cell uniqueness and validity

A duplicate Prism6 can occupy the same six nodes with a different local ordering. The duplicate key is therefore the sorted six-node set, not the raw row:

```
cell_keys = _structured_rows(np.sort(mesh.prisms, axis=1))
len(np.unique(cell_keys)) == len(mesh.prisms)
```

This test is topological. Positive signed volume and positive sampled Jacobian determinants are separate geometric tests performed by $_{\text{prismmetrics}}$. Neither can replace the other: two duplicate valid cells still overlap, and a unique cell can still be inverted.

Complete face census

For every Prism6, two triangular and three quadrilateral local faces are extracted using the canonical local-face tables. Faces are sorted only for identity; their stored order remains available for normal checks.

$_{\text{facecensus}}$ returns each unique face, its incidence count, and the first/second incident cell indices.

Incident cells	Required interpretation
1	external boundary face
2	conformal internal owner-neighbour face
0	impossible for a face extracted from a cell; indicates a declaration error elsewhere
>2	non-manifold overlap

The union of all six declared patches must equal the complete one-incident-cell face set. This catches missing patch elements, duplicate patch assignment, and `defaultFaces`-style leftovers through one exact set equality.

External shell edges

Every edge of the closed external surface must occur in exactly two external faces, regardless of whether those faces are Tri3 or Quad4. The audit extracts all three or four edges per patch face, sorts their endpoint tags, and requires an incidence count of two.

This does not assert that a single named patch is closed; most patches have legitimate rims shared with another patch. It asserts that the union of all six patches is a closed two-manifold shell.

Shared feed-film interface

Each `mouth_triangles` row is looked up in the triangular volume-face census. It must have exactly two incident cells, with sorted `region_id` pair `[0,1]`:

```
film Prism6 ---- shared Tri3 ---- feed Prism6
 region 0       no BC entity       region 1
```

The same face key must be absent from the external triangle set. These checks prove an internal owner-neighbour face, not a boundary condition or zone.

Implementation and focused regression:

- $\text{audit}_{\text{canonical}}$
- mouth/path test
- generator mouth test

Cell-graph connectivity and feed paths

Every two-incident-cell face contributes an undirected cell-graph edge. $_{\text{componentcount}}$ uses union-find to require one connected 3-D region.

One component alone is necessary but the audit also checks the intended local route:

- every `pressure_feed` face is incident to a feed cell;
- mouth pairs join feed and film cells;
- pressure-feed incident cells, both axial-end incident cells, and both sides of every mouth face have the same union-find root.

This distinguishes "the imported mesh is connected somewhere" from "the external feed actually communicates through the mouth to the film and both axial exits."

Local boundary orientation

The generator originally checked the pressure inlet explicitly and compared total cell volume with the signed exterior-boundary volume. The hardened contract now checks every external Tri3 and Quad4 independently in $\text{validate}_{\text{externalfaceorientation}}$.

For face `f` and its sole incident cell `c`:

```
n_f  = area vector from the stored face-node order
x_f  = face centroid
x_c  = cell centroid
require dot(n_f / |n_f|, x_f - x_c) > 0
```

For a Quad4 the area vector is the sum of the two oriented triangles `(0,1,2)` and `(0,2,3)`. The owner direction is local, so one reversed face cannot be hidden by cancellation elsewhere on the closed surface.

The audit records the minimum positive projection and minimum face area per patch. Reversing one journal triangle is a mandatory failure in the negative/reversed regression.

Scale, bounding box, and volume

Coordinates must already be float64 metres. No export scaling is applied. The audit requires:

- NPZ metadata says `coordinate_unit: m`;
- generator provenance records one earlier `0.001` conversion from CAD mm;
- recomputed bounding box equals the canonical source;

- integrated Prism6 volume agrees with the stored per-cell sum and source `mesh_report.json` under the configured relative tolerance;
- the source linear-mesh/native-BREP error stays within `5e-4`, the established curved-facet envelope;
- every signed cell volume and sampled Jacobian remains positive.

The BREP comparison is not recomputed from CAD during interchange. Its value is read from the source mesh report and checked against the accepted envelope, because recreating CAD would violate the mesh-identity boundary.

What this audit does not prove

Static topology and geometry do not prove that Fluent's importer will retain the same zones, element types, scale, or connectivity. That is a separate runtime boundary in the live Fluent audit. They also do not establish CFD suitability under a particular turbulence, cavitation, or transient model.

Format Round-Trip Validation

Source: *`docs/interchange/round-trip-validation.org`*

Why numeric tags cannot be compared directly

MSH and CGNS readers may renumber nodes, elements, elementary entities, and physical IDs while preserving the same mesh. A correct audit must tolerate renumbering without tolerating changed geometry or membership.

$\text{audit}_{\text{roundtrip}}$ therefore reconstructs canonical signatures in three steps:

1. map imported nodes to canonical nodes by coordinates;
2. map imported cells/faces by unordered canonical node sets;
3. check geometry and stored face orientation using the imported ordered rows.

Coordinate mapping

$_{\text{canonicalizecoordinates}}$ lexicographically sorts imported and source `(x,y,z)` arrays. The source audit has already rejected coincident duplicate coordinates, so corresponding sorted rows define an unambiguous node map.

The maximum componentwise error must not exceed `--coordinate-tolerance` (default `1e-14 m`). The smallest nominal film gap is `20 micrometres = 2e-5 m`, so the default gate is nine orders of magnitude smaller than that physical length scale. It is a round-trip precision gate, not a mesh-size tolerance.

The resulting lookup maps imported node tags back to canonical one-based node tags. All later connectivity comparisons operate in the canonical tag space.

Connectivity signatures

$_{\text{signaturemapping}}$ sorts node tags within each cell or face only to establish identity. It requires equal array shape, every expected signature exactly once, and no missing or duplicate mapped member.

For Prism6 cells, the imported ordered rows are retained after identity mapping and passed to the same Jacobian/volume calculation. For external faces, the imported ordering is used for the local outward-normal check. This separation allows harmless element reordering but rejects a node-order change that inverts a cell or face.

Exact group membership

The reopened model must expose exactly seven physical names:

```
fluid
journal_wall
bushing_bore_wall
axial_end_z0
axial_end_zL
feed_tube_wall
pressure_feed
```

`fluid` must be the only 3-D entity/group. Each boundary name must map to one 2-D entity. The element rows collected from each entity must match the canonical patch signatures exactly. A correct total boundary count with one merged or renamed patch is a failure.

$\text{validate}_{\text{groupnames}}$ also rejects all forbidden mouth/default names. In the CGNS path its error is annotated `GEOMETRY_ONLY_NOT_FLUENT_READY` to make clear that surviving coordinates without BC membership are insufficient.

Reconstructing the internal mouth after export

The mouth is not an exported surface section, so it cannot be checked by looking for a group. Instead, the audit maps every imported Prism6 to its canonical cell and verifies that each canonical mouth node triple is contained in the two mapped incident cells recorded by `CanonicalAudit`.

The absence of a mouth physical group plus preservation of both incident cells is the interchange form of the owner-neighbour proof.

Geometry checks on reopened files

Every reopened format must reproduce:

- point, Prism6, and per-patch element counts;
- exactly one 3-D fluid zone;
- `PENTA_6` volume and `TRI_3=/=QUAD_4` boundary element types;
- bounding box within coordinate tolerance;
- total volume within relative tolerance;
- positive per-cell signed volumes and Jacobian samples;
- positive local outward projection on every external face;
- no solution fields or Gmsh post-processing views.

The report stores the maximum coordinate error, minimum signed volume, minimum Jacobian, and minimum outward projection per patch rather than only a boolean result.

Format-specific interpretation

The same audit function reopens all three outputs through Gmsh. For native MSH this exercises the corresponding MSH reader. For CGNS it exercises Gmsh's CGNS reader, including the zone/BC structures emitted by its writer. This is an independent serialization round trip but still not a Fluent import.

The production results recorded on 2026-07-16 were:

Format	nGap$_{04}$ max coordinate error	nGap$_{08}$ max coordinate error
Gmsh 4.1 binary	0	0
Gmsh 2.2 ASCII	approximately `6.94e-18 m`	approximately `6.94e-18 m`
CGNS	0	0

Current values belong in `interchange_report.json`; this table records the acceptance run, not a permanent promise about future writers.

Regression links

- three-format count/unit/connectivity test
- exact physical-group test
- exported orientation test
- CGNS name-loss rejection

Gmsh and CGNS Interchange Formats

Source: `docs/interchange/formats-and-cgns.org`

Three outputs, one discrete model

$\text{write}_{\text{cleanmeshes}}$ writes all three files before clearing the model:

File	Representation	Intended use
`bearing_prism_gmsh41.msh`	Gmsh MSH 4.1 binary	compact canonical inspection/interchange
`bearing_prism_gmsh22_ascii.msh`	Gmsh MSH 2.2 ASCII	conservative legacy/tool compatibility and readable physical names
`bearing_prism.cgns`	HDF5-backed CGNS unstructured mesh	Fluent/import-neutral transfer candidate

The content comes from one registered model; the writers do not independently derive boundaries or cells.

Clean MSH files

The generator's `ported_prism.msh` contains ElementData views such as `region_id`, `minSICN`, and `gap_um`. Those are useful for inspection but are not part of a minimal solver-neutral mesh.

The interchange MSH files contain only:

- one unique coordinate set;
- Prism6 volume elements;
- Tri3 and Quad4 external faces;
- one named volume group and six named surface groups.

`Mesh.SaveAll` remains zero. Because every intended element belongs to a physical group, this prevents unclassified elements from being exported. The MSH 2.2 writer is explicitly ASCII; the 4.1 writer is binary.

Gmsh MSH and Fluent MSH are different formats

The extension `.msh` identifies unrelated format families in Gmsh and Fluent. Renaming `bearing_prism_gmsh41.msh` to another filename does not convert its syntax, zone records, or connectivity representation into Fluent's native format.

For the Prism6 full-passage branch, the first Fluent-native file remains `bearing_prism_imported.msh.h5`, written only after a live CGNS import audit passes. See Fluent import-only audit.

The reduced surface-inlet Hex8 branch has a narrower direct route: fluent$_{legacymsh}$.py writes Fluent legacy ASCII directly from the validated canonical arrays. It does not convert a Gmsh file. It reconstructs every Quad4 face from Hex8 connectivity, assigns owner/neighbour cells, requires complete and disjoint external boundary coverage, verifies the Fluent right-hand face orientation toward `c0`, writes SI coordinates, and reparses the result before atomic publication.

This writer intentionally supports only the existing first-order Hex8 surface-inlet mesh. It is not a general Fluent writer.

CGNS structures used here

CGNS represents an unstructured zone through a hierarchy rather than a flat element table. The relevant concepts are:

- an unstructured `Zone_t` with grid coordinates;
- `Elements_t` sections for `PENTA_6`, `TRI_3`, and `QUAD_4` connectivity;
- =ZoneBC$_t$=/family associations carrying the six boundary memberships;
- `GridLocation=FaceCenter` for BC ranges that reference surface elements;
- a family/group representing the single `fluid` volume;
- no `FlowSolution_t` data.

The CGNS SIDS describe `Zone_t` and `Elements_t` in the hierarchy definition and the grid/element definition. Those structures are why an HDF5 sidecar or arbitrary group layout is not a CGNS implementation.

Writer selection

The pinned Gmsh 4.15.2 build reports `Cgns` in `General.BuildInfo`. The exporter refuses CGNS output if that capability is absent and calls `gmsh.write(...cgns)` only after building the exact physical-group model.

Gmsh 4.15.2 writes BC ranges over `TRI_3=/=QUAD_4` sections with `GridLocation=CellCenter`. That describes volume-cell centres, not the referenced surface elements, and `cgnscheck` reports those ranges as having invalid dimension. cgns$_{compat}$.py therefore performs one narrow post-write correction: only those discovered surface-BC location datasets are changed to `FaceCenter`. CGNS defines `FaceCenter` for boundary faces in the boundary-condition SIDS.

The correction is deliberately not a second mesh writer. It:

- discovers the zone, element sections, and BC ranges from CGNS labels;
- rejects an unexpected layout or location value;
- writes through a same-directory temporary file and atomically replaces it;
- hashes every other dataset before and after;
- preserves the CGNS version, coordinates, connectivity, ranges, and names;
- is idempotent and covered by a regression test.

Reasons for using this writer:

- it is a maintained CGNS-library integration, not custom HDF5;
- it maps Gmsh entities and physical groups into CGNS sections/BCs;

- the same process can immediately reopen the file and query those definitions;
- no parallel element-order mapping is needed.

`h5py` is used only for the standards correction and its invariant check. Generic hand-written HDF5-to-CGNS output remains rejected.

The Gmsh reference documents CGNS as a supported mesh format and physical groups as the meaningful entity collections used during export: Gmsh 4.15.2 manual.

Boundary names must exist in CGNS itself

A `zones.csv` sidecar is useful for humans, but Fluent does not import that file. After reopening CGNS, the audit queries physical names and entity memberships from the CGNS reader and requires all six exact patch names plus `fluid`.

If names or memberships are lost, the run fails with `GEOMETRY_ONLY_NOT_FLUENT_READY`. It does not publish a sidecar claiming that the missing definitions "should" have been present.

Observed Fluent 26.1 CGNS failure

Correcting the surface ranges to `GridLocation=FaceCenter` is a standards conformance fix, not proof that a proprietary reader will accept the file. Fluent 26.1 Student was tested with both corrected HDF5-backed and ADF-backed CGNS variants. Both processes terminated in `CeetronSAM.dll` before Fluent reached `Mesh Check`.

Because changing the CGNS storage backend did not change the failure, the surface-inlet handoff no longer uses CGNS as its critical path. Read `bearing_surface_inlet_hex_fluent.msh` with **File → Read → Mesh**. This invokes Fluent's native mesh reader rather than **File → Import → CGNS → Mesh** and its Ceetron/VKI dependency.

Coordinates and units

The coordinate arrays passed to the writer are already numeric SI values in metres; the exporter applies scale `1.0`. Reopened coordinates and bounding box must match the canonical arrays under `1e-14 m` by default, which detects both `0.001` and `1000` mistakes immediately.

Gmsh's current CGNS output does not declare a length unit that its reader uses for rescaling; the reader reports that the length unit is undefined and leaves the numbers unchanged. Therefore the metre contract is established by:

1. the canonical NPZ/report unit declaration;
2. no scaling during export;
3. exact numeric coordinate/bounding-box round trip;
4. the interchange manifest/report supplied with the file.

This is preferable to pretending that an absent CGNS dimensional-units node was written. A live Fluent import must independently confirm the same metre extents.

Linear precision and the thin gap

All coordinates remain float64 in memory and CGNS. A zero-error CGNS round trip was observed for both production cases. The audit tolerance is many orders below the 20–80 micrometre film thickness, so a float32-like precision loss or unit conversion would fail before solver import.

The elements remain linear. Exact analytic rim nodes are connected by chords; CGNS does not restore curved geometry between those nodes. Rim sagitta and linear-mesh/BREP volume error remain the appropriate approximation measures.

No solution state

No pressure, velocity, material, model, initialization, or iteration data is attached. The CGNS file is a mesh with zone definitions, not a Fluent case or a CFD solution. The `solution_fields: 0` report field and empty Gmsh view set make this boundary machine-checkable.

Fluent Import-Only Audit

Source: `docs/interchange/fluent-import-audit.org`

Why a second audit is mandatory

The static CGNS round trip proves what Gmsh wrote and can read. Fluent has its own importer, zone mapping, element interpretation, license boundary, and native mesh representation. Therefore `FLUENT_IMPORT_PASS` is reserved for $run_{fluentimportaudit}$ executing against a real Fluent process.

The 2026-07-16 acceptance environment had no PyFluent/Fluent installation, so the retained production outputs are `STATICALLY_VALIDATED_NOT_IMPORTED`. No parser fixture, mocked session, or CGNS round trip can promote that state.

CLI modes and license behavior

The exporter exposes three policies:

Mode	Behavior
`--fluent skip`	never imports PyFluent and never requests an Ansys license
`--fluent auto`	runs only when PyFluent can be imported; launch/license failure becomes an explicit `NOT_RUN`
`--fluent required`	exits nonzero unless a real import returns PASS

$\text{fluent}_{\text{capability}}$ can detect the Python package without consuming a license. A license is known to be usable only when `launch_fluent` succeeds; package presence alone is not reported as availability.

The required/auto distinction is enforced by $\text{enforce}_{\text{fluentmode}}$ and tested without a proprietary runtime in the unavailable-runtime regression.

Live session configuration

The automated audit launches Fluent with:

```
session = pyfluent.launch_fluent(
    mode=pyfluent.FluentMode.MESHING,
    dimension=pyfluent.Dimension.THREE,
    precision=pyfluent.Precision.DOUBLE,
    processor_count=1,
    ui_mode="gui" if gui else "no_gui_or_graphics",
)
```

Meshing/import mode is sufficient and avoids entering a solve workflow. Three dimensions and double precision are explicit rather than inherited from local defaults. One process is enough for an import audit; parallel performance is outside this stage.

PyFluent's launcher and meshing APIs are documented in its launch guide and meshing import API.

Import and checks

The session performs only:

```
session.tui.file.import_.cgns_vol_mesh(str(cgns.resolve()))
session.tui.mesh.check_quality_level(1)
session.tui.mesh.check_mesh()
session.tui.mesh.check_quality()
```

The transcript is captured from before import through the mesh/quality checks. No code switches to solution mode, initializes a field, assigns boundary values, or calls an iteration command.

The live data-model queries then obtain:

- all boundary and cell-zone IDs and names;
- face count for each required boundary;
- cell count and cell-zone shape/type;
- mesh-check transcript metrics and warnings.

Mandatory comparisons

The Fluent result must match the canonical NPZ-derived summary, not merely the CGNS reader result:

Fluent observation	Required result
cell zones	exactly one `fluid` zone
cell shape	Prism/wedge only
boundary zones	exactly the six required names
boundary counts	exact canonical count for every patch

Fluent observation	Required result
nodes, faces, cells	exact canonical totals
bounding box	metre-scale canonical extents
total volume	canonical volume within strict tolerance
minimum cell volume	positive
maximum skewness	finite and in Fluent's valid range
connected regions	one
mesh check	no invalid-connectivity, negative-volume, or zero-volume failure
feed mouth	absent from the boundary-zone set

Missing or merged patches, unknown/default zones, an unexpected fluid split, or incomplete transcript metrics are hard failures.

Transcript parsing is deliberately conservative

$\text{parse}_{\text{fluenttranscript}}$ extracts counts, extents, volume statistics, skewness, and region count from the captured Fluent transcript. Fluent wording can vary by release, so the audit does not substitute a guessed value when a metric cannot be parsed. The `transcript_metrics_complete` check fails and the native mesh is not written.

When a new Fluent release changes output text, update the parser against a real saved transcript and retain that transcript as evidence. Do not relax the required-metric set merely to obtain a PASS.

Guarded native save

`bearing_prism_imported.msh.h5` is written only inside the branch where every mandatory check is true. This ordering makes the native Fluent mesh a promotion artifact rather than an import attempt:

```
if report["status"] == "PASS":
    session.tui.file.write_mesh(str(native_path.resolve()))
```

An optional import-only `.cas.h5` may be saved manually after the same audit, but it must remain explicitly marked as having no material model, boundary values, initialization, or solution.

Manual Fluent audit

When automation is unavailable, the equivalent import-only review is:

1. launch Fluent in 3-D, double precision;
2. import `bearing_prism.cgns` through **File** > **Import** > **CGNS** > **Mesh** rather than **Mesh & Data**;
3. run **Mesh** > **Check** at maximum verbosity;
4. require one `fluid` zone containing only wedge/Prism6 cells and exactly the six canonical external zones with report-matching counts;
5. reject any mouth, default, unknown, or merged boundary zone;
6. compare nodes, faces, cells, bounds, volume, minimum cell volume, skewness, and connected-region count with `interchange_report.json`;
7. inspect the full mesh, the feed boundaries, the bore opening, and an `x=0` clipped section through the film/feed junction;
8. save `bearing_prism_imported.msh.h5` only after all checks pass.

Generic roles may then be assigned without values—walls, `pressure-inlet`, and =pressure-outlet=—but that is outside the interchange acceptance state.

Manual visual proof

When Fluent is available, retain screenshots or a documented display record of:

1. the complete ported volume mesh;
2. `pressure_feed` and `feed_tube_wall` only;
3. `bushing_bore_wall` only, with the circular opening visible;
4. an `x=0` section through tube and film;
5. a clipped cell view through the shared feed-film junction;
6. the boundary-zone list, demonstrating that no mouth zone exists.

Visual proof supports the count audit but does not replace it. No Fluent screenshots were produced in the 2026-07-16 acceptance run because Fluent was unavailable.

Journal boundary

$\text{fluent}_{\text{importcheck}}$.jou is a portable console import/check aid. It deliberately does not save a native mesh: a static journal cannot reliably interpret every release-specific transcript and decide whether all canonical comparisons passed. The guarded save belongs in the PyFluent audit or a documented manual review.

Separately marked integration test

$\text{test}_{\text{realfluentimportwhenavailable}}$ is marked `integration`. It skips when PyFluent or a license is unavailable; a skip is not converted into an import pass.

Interchange Commands and Report Contracts

Source: *`docs/interchange/operations-and-reports.org`*

Production exports

Default four-layer film mesh:

```
uv run python interchange/export_solver_neutral.py \
  --case-dir out/ported_prism_default/nGap_04 \
  --outdir out/interchange_default/nGap_04 \
  --fluent skip
```

Default eight-layer film mesh:

```
uv run python interchange/export_solver_neutral.py \
  --case-dir out/ported_prism_default/nGap_08 \
  --outdir out/interchange_default/nGap_08 \
  --fluent skip
```

The 20-degree/eccentricity-0.3 family is discovered in this repository as `out/ported_prism_e03_g20/`. Select it by path:

```
uv run python interchange/export_solver_neutral.py \
  --case-dir out/ported_prism_e03_g20/nGap_04 \
  --outdir out/interchange_e03_g20/nGap_04 \
  --fluent skip
```

The output directory must not already exist unless `--overwrite` is explicit. Replacement remains atomic: all files and round trips complete in a sibling staging directory before the destination changes.

Tolerance options

Option	Default	Meaning
`--coordinate-tolerance`	`1e-14 m`	maximum componentwise coordinate error after a format round trip
`--volume-relative-tolerance`	`1e-12`	total exported/reopened volume error relative to the canonical cell sum

These are serialization tolerances, not meshing parameters. They do not move nodes, heal cells, or change the already-established BREP/faceted-volume envelope.

Output directory

File	Contract
`bearing_prism_gmsh41.msh`	clean MSH 4.1 binary, no views
`bearing_prism_gmsh22_ascii.msh`	clean MSH 2.2 ASCII, no views
`bearing_prism.cgns`	one unstructured fluid zone, six BC groups, no solution fields
`interchange_report.json`	source audit, three round trips, tolerances, provenance, and Fluent result
`interchange_manifest.json`	concise unit/zone/file/readiness contract
`zones.csv`	zone dimension, element type, count, role, and external flag

File	Contract
`file_hashes.json`	source and exported SHA-256 records
`gmsh_interchange.log`	writer/reader messages, including CGNS unit handling
`README_OPEN_ME_FIRST.txt`	generated handoff paths and current promotion boundary

When a live audit passes, the same directory additionally contains `fluent_import_transcript.txt`, `fluent_import_report.json`, and `bearing_prism_imported.msh.h5`.

Status model

Only three overall states are valid:

Overall	Meaning
`STATIC_PASS_FLUENT_NOT_RUN`	canonical and three format audits passed; no live Fluent acceptance claim
`FLUENT_IMPORT_PASS`	static audits plus a real Fluent import/check/query/save passed
`FAIL`	a mandatory static or live-required check failed; no successful promotion

For a static-only output, `readiness` is `STATICALLY_VALIDATED_NOT_IMPORTED` and `fluent_ready=false`. A CGNS file's mere existence never changes those fields.

Useful report queries:

```
jq '{overall,readiness,static_status,fluent_import_status}' \
  out/interchange_default/nGap_04/interchange_report.json

jq '.source | {points,prism6_cells,patch_counts,mouth_internal_faces,bounding_box_m,volume_m3}' \
  out/interchange_default/nGap_04/interchange_report.json

jq '.exports[] | {format,coordinate_max_error_m,minimum_signed_volume_m3,physical_groups}' \
  out/interchange_default/nGap_04/interchange_report.json
```

Live Fluent command

When PyFluent and the required Ansys/VKI license are available:

```
uv run python interchange/fluent_import_check.py \
  --interchange-dir out/interchange_default/nGap_04
```

Use `--gui` only when the visual proof is being collected. The automation still performs the same count and transcript gates; GUI availability is not a PASS criterion.

Inspection inputs

Gmsh may open either clean MSH file:

```
uv run gmsh out/interchange_default/nGap_04/bearing_prism_gmsh41.msh
uv run gmsh out/interchange_default/nGap_04/bearing_prism_gmsh22_ascii.msh
```

The CGNS import candidate for Fluent is:

```
out/interchange_default/nGap_04/bearing_prism.cgns
```

The older `ported_prism.msh` remains useful for Gmsh ElementData inspection, but it is not the clean solver-neutral handoff artifact.

Workbench boundary

Workbench Meshing is not expected to edit a Gmsh volume mesh. The intended route for the resolved full-passage Prism6 model is:

1. import `bearing_prism.cgns` in standalone Fluent or a Fluent Setup session;
2. complete the live audit in Fluent import-only audit;
3. save `bearing_prism_imported.msh.h5` only after PASS;
4. use that Fluent-native mesh, or an explicitly import-only `.cas.h5`, in the Workbench release's supported Fluent import path.

If Ansys Meshing must generate or edit the mesh, return to the separate BREP/CAD route and accept that the result will be a newly generated mesh, not this validated ported Prism6 connectivity.

For the reduced surface-inlet Hex8 model, generate the native Fluent file from the canonical arrays:

```
uv run python interchange/fluent_legacy_msh.py \
  --npz out/structured_surface_inlet_default/nGap_08/mesh_arrays.npz \
  --out out/ansys_surface_inlet_default/bearing_surface_inlet_hex_fluent.msh \
  --report out/ansys_surface_inlet_default/fluent_native_validation.json
```

In Fluent, read `bearing_surface_inlet_hex_fluent.msh` through **File** → **Read** → **Mesh**. Run **Mesh Check** and require one `fluid` Hex8 zone plus `journal_wall`, `stationary_wall`, `pressure_feed`, `axial_end_z0`, and `axial_end_zL`. The optional `VISUAL_CONTEXT_ONLY_context_assembly.step` is metal context for Workbench/SpaceClaim inspection; it must not enter the Fluent fluid zone.

The corrected HDF5 and ADF CGNS variants both terminated Fluent 26.1 Student inside `CeetronSAM.dll` before `Mesh Check`. They remain diagnostic artifacts, but are not the current execution path. The native ASCII file bypasses Ceetron/VKI and preserves the canonical Hex8 nodes and connectivity. Save a native `.msh.h5` only after **Mesh Check** passes, then use that file for parallel runs.

The current reduced-model file is `STATICALLY_VALIDATED_NATIVE_MSH_FLUENT_NOT_RUN`. Its five external boundary groups differ intentionally from the full-passage model's six: there is no `feed_tube_wall` because the feed passage is outside the reduced domain.

Acceptance evidence from 2026-07-16

Case	Points	Prism6	Mouth internal faces	Static status	Fluent
`nGap_04`	339358	552560	1732	PASS in all three formats	$\mathrm{NOT}_{\mathrm{RUN}}$
`nGap_08`	575094	1021984	1732	PASS in all three formats	$\mathrm{NOT}_{\mathrm{RUN}}$

Patch counts and numerical details should be read from each current `interchange_report.json` rather than copied into downstream solver setup.

Tests

```
uv run pytest -q tests/test_solver_neutral_interchange.py
uv run pytest -q
```

The acceptance run completed with 32 passing tests and one skipped real-Fluent integration test. The skip records unavailable proprietary infrastructure; it is not evidence about Fluent's importer.

Operations

Operations Map

Source: `docs/operations/index.org`

Commands assume the repository root as the working directory.

Running and testing stage order, default and second-case commands, and regression tests.
Visualization FreeCAD, Gmsh, and ParaView inspection without confusing diagnostic subsets with solver meshes.
Outputs and reports artifact contracts, units, hashes, stage statuses, and atomic failure behavior.
OpenFOAM mesh handoff optional `gmshToFoam` and `checkMesh` integration in the canonical mesh-generation CLIs.
OpenFOAM CFD handoff accepted S100 import and commissioning evidence, rejected diagnostics, low-speed continuation, and the current guarded stop.
Solver-neutral and Fluent handoff clean MSH/CGNS exports, static reports, live-import promotion, and Workbench boundary.
Code-reading guide shortest route from entry points through geometry, meshing, validation, export, and tests.

Reference material is indexed separately in References. Mesh-design details live in Meshing; transfer semantics live in Solver-neutral interchange.

Code-Reading Guide

Source: `docs/operations/code-reading-guide.org`

Common control pattern

All pipelines follow this shape:

```
inputs = parse_args()
contract = load_and_validate_inputs(inputs)
stage = make_staging_directory(outdir)
artifact = construct(contract)
validate(artifact)
export_to_stage(artifact)
round_trip_and_compare()
atomic_replace_directory(stage, outdir)
```

Exceptions stop publication. Mesh generators publish their failure contract; `run_interchange` discards the failed staging directory and leaves an existing published output untouched. The concrete entry points are `run_preflight`, `run_surface_smoke`, `run_structured`, `run_ported`, and `run_interchange`.

CAD path

1. InputParams and ResolvedParams
2. resolve$_{params}$ and validate$_{params}$
3. make$_{boreblank}$, make$_{journal}$, and make$_{feedcylinder}$
4. make$_{fullfilm}$ and split$_{axialzones}$
5. validate$_{geometry}$
6. export$_{all}$ and $_{roundtripall}$
7. main

`validate_geometry` mixes same-`z` radial and surface-normal clearance checks; their distinct targets are derived in Clearance mathematics.

Native-BREP assurance path

1. CadReference
2. configure$_{occoptions}$
3. classify$_{volumes}$
4. classify$_{surfaces}$
5. validate$_{fragmentmapping}$
6. validate$_{boundaries}$
7. $_{runocchecks}$
8. run$_{preflight}$

The classifiers derive meaning from geometry rather than entity tags. The preflight tests define the production-case regression contract.

No-port Hex8 reference

1. BearingParams
2. MeshData
3. node$_{tag}$
4. generate$_{mesh}$
5. validate$_{topology}$
6. validate$_{analyticmesh}$
7. add$_{discretemodel}$
8. the three round trips, beginning with validate$_{gmshroundtrip}$
9. the Hex8 tests

Connectivity is expressed in logical circumferential, axial, and gap indices `(j,k,i)`, including the last-to-first periodic cell.

Central-feed Prism6 path

1. Contract and analytic mouth
    - PortedParams
    - load$_{contract}$
    - validate$_{nativebrep}$
    - rim$_{coordinates}$

2. Two-dimensional topology

    - MasterMesh
    - build$_{\text{mastermesh}}$

    The feed-disk mask determines whether each outer film face becomes bore wall or internal mouth.

3. Three-dimensional construction

    - PrismMesh
    - build$_{\text{prismmesh}}$
    - $_{\text{prismgaussmetrics}}$

    A film triangle swept across one gap interval and a disk triangle swept across one tube interval both produce Prism6 cells. The final film layer and first tube layer reuse node tags at the mouth; nothing is merged afterward.

4. Proof and serialization

    - $_{\text{facecensus}}$
    - validate$_{\text{topology}}$
    - validate$_{\text{geometry}}$
    - add$_{\text{discreteprismmodel}}$
    - generate$_{\text{gapcase}}$
    - the Prism6 tests

Solver-neutral interchange path

1. Source and pre-write audit

    (a) InterchangeInputs
    (b) load$_{\text{canonicalcase}}$
    (c) audit$_{\text{canonical}}$

    The exporter rebuilds `PrismMesh` from NPZ arrays but does not rebuild geometry or cells. `audit_canonical` derives face incidence, patch owners, cell-graph connectivity, and the geometry summary used by every later comparison.

2. Clean model and format writes

    (d) write$_{\text{cleanmeshes}}$
    (e) reused add$_{\text{discreteprismmodel}}$

    Read these together: the first controls clean/no-view writer state; the second owns the single tested Tri3/Quad4/Prism6 entity registration.

3. Renumbering-independent round trips

    (f) $_{\text{canonicalizecoordinates}}$
    (g) $_{\text{signaturemapping}}$
    (h) audit$_{\text{roundtrip}}$

    Node tags are reconstructed from coordinates, then cell/face membership from canonical node signatures. Imported ordering is still used for signed-volume and outward-normal checks.

4. Optional live Fluent boundary

    (i) fluent$_{\text{capability}}$
    (j) parse$_{\text{fluenttranscript}}$
    (k) run$_{\text{fluentimportaudit}}$
    (l) interchange tests

    The native Fluent mesh save occurs at the end of `run_fluent_import_audit` and only inside the all-checks-PASS branch.

Running and Testing

Source: `docs/operations/running-and-testing.org`

Environment

```
uv sync --locked
```

Python selection and dependency pins are in `.python-version`, `pyproject.toml`, `uv.lock`, and `requirements.txt`.

All geometry source dimensions are millimetres. The volume-mesh pipelines convert solver coordinates to metres exactly once.

Stage order and status semantics

The stages have independent status records. A failure in one exchange format does not rewrite the result of another stage.

Stage	Canonical command	Current recorded status	Meaning
CAD construction	`bearing_film.py`	`FAIL` at strict STEP gate	Native geometry and BREP checks pass; STEP volume round trip exceeds `1e-6`.
Native BREP preflight	`gmsh_brep_preflight.py`	`PASS`	One unsplit OCC volume and three conformal fragmented zones survive import and SI scaling.
Boundary smoke mesh	`gmsh_surface_smoke.py`	`PASS`	Valid Tri3 surface mesh only; zero 3-D elements.
No-port Hex8 baseline	`structured_hex_no_port.py`	`PASS`	Full-360 analytic structured mesh with no feed passage.
Ported Prism6 mesh	`layered_prism_central_feed.py`	`PASS`	Full-360 conformal film plus open central feed passage.
Solver-neutral export	`export_solver_neutral.py`	`STATIC_PASS_FLUENT_NOT_RUN`	Clean MSH/CGNS round trips match canonical nGap_{04} and nGap_{08} arrays.
Canonical Fluent import audit	`fluent_import_check.py`	`NOT_ACCEPTED`	The earlier S100 Fluent solve diverged and does not establish `FLUENT_IMPORT_PASS`.
OpenFOAM audit in production mesh reports	optional	`SKIPPED`	These mesh-generation runs used `--openfoam skip`; the status is scoped to those reports.
Downstream S100 OpenFOAM import	handoff import procedure	`ACCEPTED`	`gmshToFoam`, patch correction, and standard `checkMesh` accepted the 294,912-cell Hex8 mesh.
Downstream OpenFOAM solution study	CFD handoff	accepted liquid branch through 20 rpm	The first 20-to-50-rpm update stopped at 20.2 rpm on `p_min < pSat`; no two-phase point is accepted.
Native Reynolds/JFO application	`bash open-foam/check_jfoBearingFoam.sh`	`UNVALIDATED_CANDIDATE`	The zero-speed check and high-speed continuation pass internal gates; external JFO benchmark and three-grid checks remain mandatory.

The strict CAD command is therefore expected to return nonzero on the currently recorded build. Inspect its `params.json` rather than changing the tolerance or using the rejected STEP. The trusted native BREP files remain published for the downstream open-source path.

The commands below rebuild the canonical CAD and mesh contracts. They do not replay the later solver study. OpenFOAM case construction, replay, checkpoint, and guarded-ramp commands live in the replay handoff and the guarded-ramp handoff.

Generate the default geometry contracts

1. CAD and native BREP

   ```
   uv run python bearing_film.py \
     --outdir out/strict_default \
     --retain-failed-step
   ```

   The current strict report records the STEP failure and retains: `film_unsplit.brep`, `film_zones.brep`, `film_thickness_map.png`, `requirements.txt`, and `params.json`. The nonzero exit is part of the STEP acceptance contract. Rejected exchange files are isolated in `out/strict_default.rejected-step/` with `solve_eligible: false`; they are for inspection, not downstream meshing.

The second parameter case is generated independently:

```
uv run python bearing_film.py \
  --eccentricity-ratio 0.3 \
  --semicone-angle-deg 20 \
  --outdir out/strict_case_e03_g20
```

2. Native BREP preflight

   Default paths are encoded in the CLI, so the default case is:

```
uv run python meshing/gmsh_brep_preflight.py --no-gui
```

   Second case:

```
uv run python meshing/gmsh_brep_preflight.py \
  --unsplit out/strict_case_e03_g20/film_unsplit.brep \
  --zones out/strict_case_e03_g20/film_zones.brep \
  --params out/strict_case_e03_g20/params.json \
  --outdir out/gmsh_preflight_e03_g20 \
  --no-gui
```

   Downstream consumers require `overall: PASS` and verify the recorded BREP SHA-256.

3. Two-dimensional surface smoke test

```
uv run python meshing/gmsh_surface_smoke.py --no-gui

uv run python meshing/gmsh_surface_smoke.py \
  --brep out/gmsh_preflight_e03_g20/film_zones_fragmented.brep \
  --preflight-report out/gmsh_preflight_e03_g20/preflight_report.json \
  --outdir out/gmsh_surface_smoke_e03_g20 \
  --no-gui
```

   This stage deliberately calls only `gmsh.model.mesh.generate(2)`. It proves boundary classification and shared interfaces, not volume-mesh quality.

Generate volume meshes

1. Frozen no-port Hex8 reference

   Default case:

```
uv run python meshing/structured_hex_no_port.py \
  --params out/strict_default/params.json \
  --outdir out/structured_no_port_default \
  --n-theta 256 --n-axial 96 \
  --gap-levels 4 8 12 --preview-ngap 8 \
  --openfoam skip --no-gui
```

   Second case:

```
uv run python meshing/structured_hex_no_port.py \
  --params out/strict_case_e03_g20/params.json \
  --outdir out/structured_no_port_e03_g20 \
  --n-theta 256 --n-axial 96 \
  --gap-levels 4 8 12 --preview-ngap 8 \
  --openfoam skip --no-gui
```

   This is a regression baseline without the feed passage. It must not be used as the ported CFD domain.

2. Film-only Hex8 mesh with surface pressure inlet

   This reduced-domain model applies the supply condition directly to a mesh-aligned patch on the stationary bore. It retains only annular-film Hex8 cells, uses symmetric 5:1 through-gap inflation, clusters theta/z lines smoothly around the feed, and contains no feed-passage volume:

```
uv run python meshing/structured_hex_surface_inlet.py \
  --params out/strict_default/params.json \
  --outdir out/structured_surface_inlet_default \
  --n-theta 256 --n-axial 96 \
```

```
  --gap-levels 4 8 12 --preview-ngap 8 \
  --gap-inflation-ratio 5 \
  --inlet-cluster-strength 0.82 \
  --openfoam skip --no-gui
```

The second parameter case changes only `--params` and `--outdir`:

```
uv run python meshing/structured_hex_surface_inlet.py \
  --params out/strict_case_e03_g20/params.json \
  --outdir out/structured_surface_inlet_e03_g20 \
  --n-theta 256 --n-axial 96 \
  --gap-levels 4 8 12 --preview-ngap 8 \
  --gap-inflation-ratio 5 \
  --inlet-cluster-strength 0.82 \
  --openfoam skip --no-gui
```

The `pressure_feed` group consists of complete outer Quad4 faces. The report must pass its projected-area, connected-rim, outward-normal, and boundary partition checks. The `0.82` setting is cubic coordinate clustering about `theta=pi` and `z=z_h`; theta/z spacing is intentionally nonuniform and remains conforming. See Structured Hex8 film with a surface pressure inlet for the approximation and physics boundary.

3. Conformal central-feed Prism6 production mesh

   Default case:

```
uv run python meshing/layered_prism_central_feed.py \
  --params out/strict_default/params.json \
  --brep out/strict_default/film_unsplit.brep \
  --preflight out/gmsh_preflight/preflight_report.json \
  --outdir out/ported_prism_default \
  --n-theta 256 --n-axial 96 \
  --gap-levels 4 8 12 --preview-ngap 8 \
  --gap-inflation-ratio 5 \
  --rim-segments 128 --tube-layers 48 --tube-grading 1.0 \
  --openfoam skip --no-gui
```

   Second case:

```
uv run python meshing/layered_prism_central_feed.py \
  --params out/strict_case_e03_g20/params.json \
  --brep out/strict_case_e03_g20/film_unsplit.brep \
  --preflight out/gmsh_preflight_e03_g20/preflight_report.json \
  --outdir out/ported_prism_e03_g20 \
  --n-theta 256 --n-axial 96 \
  --gap-levels 4 8 12 --preview-ngap 8 \
  --gap-inflation-ratio 5 \
  --rim-segments 128 --tube-layers 48 --tube-grading 1.0 \
  --openfoam skip --no-gui
```

   Gmsh generates the two-dimensional master Tri3 mesh. Python constructs the three-dimensional Prism6 connectivity; the program never asks Gmsh to generate the 3-D volume mesh. See `build_master_mesh`, `build_prism_mesh`, and `validate_topology`.

   `--gap-inflation-ratio 5` is the ratio of the largest central film-cell thickness to either wall-adjacent cell thickness. It is applied symmetrically at the journal and bushing; use `1` only when reproducing uniform through-gap spacing.

Export solver-neutral MSH and CGNS

The four-layer and eight-layer production cases are independent export runs:

```
uv run python interchange/export_solver_neutral.py \
  --case-dir out/ported_prism_default/nGap_04 \
  --outdir out/interchange_default/nGap_04 \
  --fluent skip

uv run python interchange/export_solver_neutral.py \
  --case-dir out/ported_prism_default/nGap_08 \
```

```
  --outdir out/interchange_default/nGap_08 \
  --fluent skip
```

Use `--fluent auto` to attempt a live import only when PyFluent and a license are available; use `required` when absence or import failure must make the command nonzero. Full semantics are in Interchange commands and reports.

Tests

Run the complete regression set:

```
uv run pytest -q
```

Run focused suites while developing one stage:

```
uv run pytest -q tests/test_gmsh_brep_preflight.py
uv run pytest -q tests/test_structured_hex_no_port.py
uv run pytest -q tests/test_layered_prism_central_feed.py
uv run pytest -q tests/test_solver_neutral_interchange.py
```

The BREP test also runs the surface-smoke stage. The no-port suite is frozen regression coverage and must stay green when the ported pipeline changes.

Report queries

```
uv run python meshing/layered_prism_central_feed.py --help
jq -r '.overall' out/gmsh_preflight/preflight_report.json
jq -r '.overall' out/gmsh_surface_smoke/surface_mesh_report.json
jq -r '.overall' out/structured_no_port_default/run_report.json
jq -r '.overall' out/ported_prism_default/run_report.json
jq -r '.overall' out/interchange_default/nGap_04/interchange_report.json
jq -r '.cases[] | [.n_gap,.total_prisms,.topology_status,.overall] | @tsv' \
  out/ported_prism_default/run_report.json
```

Outputs and Reports

Source: `docs/operations/outputs-and-reports.org`

Evidence hierarchy

For each stage, trust evidence in this order:

1. The stage's `overall` field and mandatory audit/check records.
2. SHA-256 and unit contracts in its manifest/report.
3. Exact array or geometry round-trip results.
4. CSV summaries and logs.
5. Screenshots and interactive inspection.

Status is stage-local: the strict CAD STEP audit, native BREP preflight, surface mesh, volume meshes, neutral transfer, and proprietary import report independently.

CAD contract: `out/strict_default/`

File	Contract
`params.json`	Inputs, resolved values, measurements, hashes, validation records, and overall status.
`film_unsplit.brep`	One native OCCT fluid solid, in millimetres.
`film_zones.brep`	Three native OCCT solids: `ring_A`, `hole_band`, and `ring_B`.
`film_thickness_map.png`	Same-z radial-clearance diagnostic; not pressure or CFD output.
`requirements.txt`	Package versions recorded by the CAD run.

The current `params.json` says `overall: FAIL` because STEP round-trip errors for the full film and feed-containing band exceed `1e-6`. Its native BREP round trips pass at approximately numerical-noise scale. The strict directory therefore publishes trusted BREP artifacts and does not present the rejected STEP as accepted production geometry.

Inspect the distinction directly:

```
jq '{overall,error,brep:.measurements.brep_round_trip,step:.measurements.step_round_trip}' \
  out/strict_default/params.json
sha256sum out/strict_default/film_unsplit.brep \
  out/strict_default/film_zones.brep
```

The export and audit contract is implemented in `export_all`.

Native BREP preflight: `out/gmsh_preflight/`

File	Contract
`preflight_report.json`	`overall: PASS`, validation records, CAD reference, OCC diagnostics, unit scaling, and output hashes.
`brep_manifest.json`	Explicit coordinate unit and scale-to-SI contract for each BREP.
`film_zones_fragmented.brep`	Three conformal OCC volumes in millimetres.
`film_zones_SI.brep`	Independently exported and re-imported SI copy in metres.
`surfaces.csv`	OCC surface inventory, areas, centres, bounds, and adjacency.
`volumes.csv`	OCC volume inventory and geometric classification.
`gmsh_preflight.log`	Gmsh OCC import, fragmentation, export, and re-import log.

The fragmented BREP is the canonical input to the surface-smoke stage. The ported analytic mesh uses `film_unsplit.brep` as an independent validation reference and verifies its hash through the preflight report.

Surface smoke: `out/gmsh_surface_smoke/`

File	Contract
`surface_mesh.msh`	Gmsh 4.1 binary first-order Tri3 boundary mesh.
`surface_mesh_ascii.msh`	Gmsh 2.2 ASCII surface mesh.
`surface_mesh.vtk`	Visualization interchange.
`surface_mesh_report.json`	PASS/FAIL records, SI bounds, quality, areas, and `volume_mesh_generated: false`.
`physical_groups.json`	Geometrically classified boundary and volume groups.
`surface_quality.csv`	Per-boundary mesh counts, areas, and quality statistics.
`gmsh_surface_mesh.log`	Gmsh 2-D meshing log.

This directory contains no valid 3-D solver volume mesh.

No-port Hex8 family

Root directory `out/structured_no_port_default/`:

`run_report.json` run inputs and one summary per gap level;
`convergence_report.json` and `convergence.csv` gap-level and circumferential faceting convergence.

Each `nGap_XX/` directory contains:

File	Contract
`structured_hex.msh`	Gmsh 4.1 binary, true-scale full Hex8 mesh.
`structured_hex_openfoam.msh`	Gmsh 2.2 ASCII conversion input.
`volume_hex.vtu`	ParaView Hex8 volume mesh and cell fields.
`boundary_quads.vtu`	External Quad4 faces and integer patch IDs.
`mesh_arrays.npz`	Canonical NumPy coordinates/connectivity for exact comparison.
`manifest.json`	`overall: PASS`, `solve_eligible: true`, SI unit, and exact counts.
`mesh_report.json`	Analytic, topology, quality, format-round-trip, and optional OpenFOAM records.
`gmsh_structured.log`	Discrete-model registration and round-trip log.
`views/*.pos`	One Gmsh ElementData view per quality/geometry field.
`images/*.png`	Static true-scale and clearly marked exaggerated diagnostics.
`viz/*`	Exact subsets and gap-exaggerated visualization-only files.

There is deliberately no feed boundary in this baseline.

Surface-inlet Hex8 family

Root directory `out/structured_surface_inlet_default/`. Its root reports record invariance across `nGap`; each `nGap_XX/` contains the same canonical Hex8/Quad4 file set as the no-port family plus:

File	Contract
`physical_groups.json`	Five external boundary groups and one fluid volume; no feed-tube group.
`images/pressure_feed_footprint.png`	Intended 4 mm circle overlaid with the selected bore Quad4 faces.
`viz/pressure_feed_only.msh` / `.vtu`	Lightweight boundary-only inlet inspection.
`viz/cutaway_exact.msh` / `.vtu`	Exact-coordinate diagnostic subset; never a solver input.

The solver-eligible volume contains only Hex8 annular-film cells. `pressure_feed` is a partition of the original stationary bore boundary, not a cylindrical fluid passage. `mesh_report.json` records its projected area, actual three-dimensional area, centroid, equivalent diameter, rim topology, normal direction, and resolution-dependent footprint error. The report and NPZ metadata also record `theta_edge_coordinates_rad`, `z_node_coordinates_mm`, cluster strength/power, local spacing at the feed, global spacing extrema, and width ratios. These arrays are part of the canonical nonuniform-grid contract, not visualization settings.

1. Solver mesh and visual context are separate artifacts

   An Ansys handoff may include both a CGNS mesh and a context STEP. They serve different purposes:

Artifact	Contents	Use
CGNS surface-inlet mesh	one Hex8 lubricant cell zone and its named boundary-face zones	import into Fluent
`VISUAL_CONTEXT_ONLY_context_assembly.step`	journal and illustrative bushing metal solids	inspect the bearing arrangement in Workbench, SpaceClaim, or FreeCAD

   The context STEP is not a second representation of the fluid and must not be included in the Fluent fluid cell zone. Conversely, the apparently hollow centre of the CGNS mesh is correct: the journal occupies that space in the physical assembly. `pressure_feed` is the inlet boundary on the bore skin; there is deliberately no feed-tube cell zone in this reduced model.

   The current default `nGap_08` Ansys handoff is `out/bearing_surface_inlet_hex_ANSYS_HANDOFF.zip`. It contains:

File	Role
`bearing_surface_inlet_hex.cgns`	solver mesh: 223488 nodes, 196608 Hex8 cells, and 53248 external Quad4 faces
`VISUAL_CONTEXT_ONLY_context_assembly.step`	separate two-solid metal context
`ANSYS_IMPORT.txt`	import procedure, counts, status, and CGNS hash
`zones.csv`	exact physical-zone inventory
`pressure_feed_footprint.png`	nominal circle versus the 390-face inlet patch

   This bundle was assembled from the validated surface-inlet case; it is not yet produced by a repository export entry point. Its status is `STATICALLY_VALIDATED_NOT_IMPORTED`. The Gmsh/CGNS round trip is evidence for static transfer, not a substitute for Fluent's **Mesh Check**.

Ported Prism6 family

Root directory `out/ported_prism_default/`:

`run_report.json` one result per `nGap`, including cell counts, minimum quality, native-BREP volume error, inlet error, topology, and OpenFOAM status;

`convergence_report.json` and `convergence.csv` invariance under film-layer subdivision.

Each `nGap_XX/` directory contains:

File	Contract
ported_prism.msh	Gmsh 4.1 binary canonical Prism6 mesh and ElementData.
ported_prism_openfoam.msh	Gmsh 2.2 ASCII conversion input with exterior Tri3/Quad4 faces.
volume_prism.vtu	Float64 ParaView Prism6 mesh and requested cell fields.
boundary_faces.vtu	Six external physical boundaries and patch IDs.
mesh_arrays.npz	Canonical nodes, Prism6 connectivity, mouth faces, fields, and metadata.
physical_groups.json	Stable physical IDs, wall roles, and forbidden-patch list.
surface_quality.csv	Face counts and areas by boundary.
manifest.json	overall: PASS, solve_eligible: true, distorted_geometry: false, and SI units.
mesh_report.json	Complete topology, geometry, quality, BREP, round-trip, visualization, and OpenFOAM records.
gmsh_ported.log	Master/discrete-model and round-trip log.
viz/*	Exact-coordinate or diagnostic subsets, all non-solver artifacts.

Implementation links:

- no-port MSH round trip and VTU export
- ported MSH round trip, VTU round trip, and NPZ round trip

Solver-neutral interchange

Root directory for the default family is out/interchange_default/. Each nGap_XX/ directory is derived from the corresponding validated ported NPZ, not from CAD or a prior MSH.

File	Contract
bearing_prism_gmsh41.msh	clean MSH 4.1 binary with no ElementData views
bearing_prism_gmsh22_ascii.msh	clean MSH 2.2 ASCII with physical names
bearing_prism.cgns	unstructured PENTA_6 fluid zone plus six exact boundary definitions
interchange_report.json	source audit, three format audits, provenance, tolerances, and Fluent state
interchange_manifest.json	concise status, zone, mouth, unit, and readiness contract
zones.csv	exact element/face count per named zone and intended role
file_hashes.json	SHA-256 for source contract files and exported meshes
gmsh_interchange.log	official writer/reader transcript
README_OPEN_ME_FIRST.txt	generated path/status handoff

Static outputs have overall: STATIC_PASS_FLUENT_NOT_RUN and readiness: STATICALLY_VALIDATED_NOT_IMPORTED. A live PASS additionally creates fluent_import_transcript.txt, fluent_import_report.json, and bearing_prism_imported.msh.h5.

Implementation and schema details:

- Interchange commands and reports
- $\text{run}_{\text{interchange}}$
- $_{\text{writezones}}$

Units and fields

Quantity	Unit
CAD and native BREP coordinates	mm
User-facing mesh-size CLI arguments	mm
Canonical volume-mesh coordinates	m
Surface areas in solver reports	m^2
Cell and total mesh volumes	m^3
gap_um field	micrometres

The ported mesh exports region_id, axial_zone_id, gap_layer_index, gap_um, theta_deg, minSICN, minDetJac, volume_m3, aspect_ratio, solve_eligible, and distorted_geometry.

Solver eligibility and diagnostic files

Only a full true-scale mesh whose manifest says `overall: PASS` and `solve_eligible: true` is a conversion candidate. The following are never solver inputs:

- any file under a `viz/` directory unless a report explicitly promotes it (the current reports do not);
- `mouth_interface_DIAGNOSTIC_ONLY.vtu`;
- no-port `gap_x100_VISUALIZATION_ONLY_DO_NOT_SOLVE.*`;
- surface-smoke outputs;
- rejected STEP files.

For solver-neutral output, static PASS is an import candidate but not a Fluent acceptance claim. Only `FLUENT_IMPORT_PASS` may promote a native Fluent `.msh.h5`.

Atomic publication and failure evidence

All production scripts write to staging directories and replace the target only after mandatory checks pass. The mesh generators replace a failing case with `failure_report.json` plus diagnostics and no solver-eligible MSH, VTU, or NPZ. The interchange exporter publishes nothing from a failed attempt and leaves an older completed destination untouched. Tolerances are never weakened automatically.

See ported failure publication, no-port failure publication, and the induced-failure regression.

Visualization

Source: `docs/operations/visualization.org`

Visualization is an inspection layer; solver eligibility comes from `manifest.json` and `mesh_report.json`.

CAD and native BREP

Native OCCT files for the strict default case:

`out/strict_default/film_unsplit.brep` one connected full fluid solid;
`out/strict_default/film_zones.brep` three coincident axial bodies.

The journal and bushing are context solids in non-strict/legacy STEP exports, not part of the CFD fluid. Close surfaces can produce z-fighting stripes in a viewer; hide context bodies before diagnosing grooves or discontinuities.

The optional browser preview is provided by `bearing_film.py --preview`:

```
uv run python -m ocp_vscode
uv run python bearing_film.py --outdir out/preview --preview
```

The strict STEP audit can still make the CAD command exit nonzero after the preview is shown. That is an exchange-format result, not a viewer failure.

The thickness map uses bearing-frame angle on the horizontal axis, axial coordinate `z` on the vertical axis, and exact same-`z` radial thickness as colour. For the default case the eccentricity/minimum-gap meridian is 270 degrees and the `+Y` feed/maximum-gap direction is 90 degrees. Slight axial variation away from those extrema comes from the changing conical journal radius in the exact square-root intersection expression; the map is not a pressure or CFD result.

1. Artix Fontconfig diagnostics

   The OCP wheel can load an older bundled Fontconfig library against newer Artix XML configuration, producing `xsi:nil` and generic-family warnings. These are cosmetic when B-rep construction, in-memory validation, and native round trips pass. They are not grounds for modifying `/etc/fonts`; a compatible OCP/ Fontconfig build is the appropriate eventual fix.

Gmsh geometry and surface smoke

Fragmented millimetre BREP after preflight:

```
uv run python meshing/gmsh_brep_preflight.py --gui
```

Validated surface-only mesh:

```
uv run gmsh out/gmsh_surface_smoke/surface_mesh.msh
```

The surface-smoke file contains Tri3 boundary elements and no 3-D volume elements. It is not a solver volume mesh.

No-port Hex8 visualization

Exact full mesh and exact cutaway:

```
uv run gmsh out/structured_no_port_default/nGap_08/structured_hex.msh
uv run gmsh out/structured_no_port_default/nGap_08/viz/cutaway_exact.msh
```

Quality view:

```
uv run gmsh \
  out/structured_no_port_default/nGap_08/viz/cutaway_minSICN.pos
```

The `.pos` file embeds the exact cutaway cells together with the `minSICN` view, so it does not need the `.msh` as a second positional argument. This is the reliable inspection command when loading both files together produces an empty Gmsh window.

Gap-exaggerated files under `viz/` are permanently marked `VISUALIZATION_ONLY_DO_NOT_SOLVE`. They are distorted geometry and must never be converted to OpenFOAM:

```
uv run gmsh \
  out/structured_no_port_default/nGap_08/viz/gap_x100_VISUALIZATION_ONLY_DO_NOT_SOLVE.msh
```

The generator-side GUI modes are `exact`, `cutaway`, `quality`, and `exaggerated`; see `open_gui`.

Surface-inlet Hex8 visualization

The inlet footprint alone is the fastest Gmsh check:

```
uv run gmsh \
  -setnumber Mesh.SurfaceFaces 1 \
  -setnumber Mesh.SurfaceEdges 1 \
  out/structured_surface_inlet_default/nGap_08/viz/pressure_feed_only.msh
```

The exact-coordinate volume cutaway shows the 5:1 gap layers without the feed column:

```
uv run gmsh \
  -setnumber Mesh.VolumeFaces 1 \
  -setnumber Mesh.VolumeEdges 1 \
  out/structured_surface_inlet_default/nGap_08/viz/cutaway_exact.msh
```

ParaView is generally more responsive for the same cutaway:

```
QT_QPA_PLATFORM=xcb paraview \
  out/structured_surface_inlet_default/nGap_08/viz/cutaway_exact.vtu
```

The inlet is visible as boundary faces because a finite-volume pressure opening must still close the cell-volume topology. Its `pressure_feed` tag, not deletion of those faces, makes it an inlet. The mesh has no cells outside the bore and no cells between the inlet patch and journal.

For patch membership in ParaView, open `boundary_quads.vtu`, select **Surface With Edges**, and color by `patch_id`. Physical ID 106 is `pressure_feed`. The footprint PNG compares the whole-face approximation with the nominal 4 mm projected circle. For a patch-only view that fills the camera, open `viz/pressure_feed_only.vtu`; it contains a compact point set.

The visibly denser theta/z edges around the inlet are intentional cubic clustering in the exact mesh coordinates. They are neither duplicated cells nor gap exaggeration. The logical mesh remains one conforming periodic Hex8 block.

1. Interpreting the apparently hollow full mesh

   Gmsh and ParaView normally draw a volume mesh through its exterior skin. For this model the skin is annular, so the central void is expected: it is the space occupied by journal metal in the assembled bearing, not a missing part of the lubricant mesh. Volumetric status is established by the `fluid` Hex8 element count and connectivity in `mesh_report.json`, not by opaque screen filling.

   The physical groups identify what is visible:

Group	Visible role
`journal_wall`	inner annular fluid boundary against the moving journal
`stationary_wall`	outer annular fluid boundary against the bushing bore
`pressure_feed`	small near-circular subset of the stationary bore
`axial_end_z0`, `axial_end_zL`	annular end boundaries
`fluid`	Hex8 cells between the inner and outer skins

Group	Visible role

Viewer colours are not part of the mesh contract and may change between imports. Use the physical-group names or `patch_id` rather than identifying a boundary by colour.

The metal journal and bushing belong in a separate context STEP. Viewing that STEP answers an assembly question; viewing the CGNS/MSH/VTU answers a fluid mesh question. Do not combine the context solids with the solver's `fluid` zone merely to make the viewport look filled.

For the current default Ansys handoff, the two views are:

```
uv run gmsh \
  out/ansys_surface_inlet_default/bearing_surface_inlet_hex.cgns

freecad \
  out/ansys_surface_inlet_default/VISUAL_CONTEXT_ONLY_context_assembly.step
```

Ported Prism6 visualization

Canonical solver-eligible mesh:

```
uv run gmsh out/ported_prism_default/nGap_08/ported_prism.msh
```

Exact-coordinate feed subsets:

```
uv run gmsh out/ported_prism_default/nGap_08/viz/feed_cutaway_exact.msh
uv run gmsh out/ported_prism_default/nGap_08/viz/feed_boundary_only.msh
```

The cutaway must show all gap layers, continuous film below the feed, the open tube-to-film connection, and the pressure-inlet disk. Its cells retain their true coordinates but it is a subset, so `solve_eligible` is false.

For the default 5:1 film inflation, color the cutaway by `gap_layer_index` and use `Surface With Edges`. The `nGap_08` normalized cell-thickness fractions increase from `0.047019` at either wall to `0.235095` at the two central cells, then mirror exactly. The authoritative coordinates and fractions are stored under `film_inflation` in that case's `manifest.json`.

The internal mouth can be inspected in:

```
out/ported_prism_default/nGap_08/viz/mouth_interface_DIAGNOSTIC_ONLY.vtu
```

It records `solve_eligible=0` and `diagnostic_only=1` and is displayed as a translucent red wireframe by default. The canonical solver boundary model does not contain a mouth patch or cap. Visualization creation is implemented in `write_visualizations`.

1. Gmsh responsiveness and the apparently missing hole

   Use `nGap_04` for interactive inspection:

   ```
   # Fast orientation check: external feed tube, rim, and inlet only.
   uv run gmsh \
     out/ported_prism_default/nGap_04/viz/feed_boundary_only.msh

   # Exact cell cutaway; suppress dense edge drawing for the first view.
   uv run gmsh \
     -setnumber Mesh.SurfaceFaces 1 \
     -setnumber Mesh.SurfaceEdges 0 \
     -setnumber Mesh.VolumeFaces 1 \
     -setnumber Mesh.VolumeEdges 0 \
     out/ported_prism_default/nGap_04/viz/feed_cutaway_exact.msh
   ```

   At the recorded production resolution, the default `nGap_04` cutaway is about 90 MB and contains more than half a million prisms; `nGap_12` is roughly 237 MB with about 1.49 million prisms. A blank or unresponsive Gmsh window while the larger file is parsed/rendered is therefore not evidence of missing geometry. Use `nGap_04` for interactive topology study and the higher levels for convergence/quality evidence.

   Quality `.pos` views multiply render load. Establish a responsive plain-mesh view first, then enable one post-processing field at a time.

   The film-to-tube mouth is also **not an exterior hole edge**. In the connected fluid mesh it is an internal shared surface with two incident cells. A skin-only renderer correctly omits it; drawing an exposed circular cap there would represent the wrong topology. To see the open passage, use the exact cutaway and rotate toward the feed, or use the much lighter =x=0 ParaView section below. The red mouth diagnostic shows the internal shared triangles, not a wall patch.

ParaView on Artix Linux and Wayland

Use XCB to avoid ParaView's Wayland/OpenGL warnings:

```
QT_QPA_PLATFORM=xcb paraview \
  out/ported_prism_default/nGap_08/volume_prism.vtu
```

The generated launcher is equivalent and relocatable:

```
out/ported_prism_default/nGap_08/viz/launch_paraview_xcb.sh
```

Inspect with **Surface With Edges**; color by `region_id`, `gap_layer_index`, `gap_um`, or `minSICN`. An `x=0` Clip exposes the feed-axis section; **Crinkle Clip** preserves whole cells, and **Shrink** separates individual prisms.

Useful direct views:

```
QT_QPA_PLATFORM=xcb paraview \
  out/ported_prism_default/nGap_08/viz/feed_cutaway_exact.vtu
QT_QPA_PLATFORM=xcb paraview \
  out/ported_prism_default/nGap_08/viz/x0_section.vtu
```

If `pvpython` is installed, the generated helper loads the full mesh, colors by region, and creates the `x=0` clip using paths relative to itself:

```
pvpython out/ported_prism_default/nGap_08/viz/open_in_paraview.py
```

Absence of `pvpython` is a recorded visualization skip, not a mesh failure.

Solver-neutral and Fluent inspection

The clean interchange MSH has no ElementData views and is useful when the task is to inspect only the transferred mesh/entity model:

```
uv run gmsh out/interchange_default/nGap_04/bearing_prism_gmsh41.msh
```

When Fluent is available, the required visual evidence is not a generic isometric screenshot. It must isolate the complete volume, feed inlet/tube, bore opening, `x=0` connection, and a clipped junction while also recording the boundary-zone list. The exact display set and its relationship to the count audit are in the Fluent visual-proof contract.

Validation boundary

A screenshot cannot resolve a 20 micrometre minimum gap or prove conformal ownership. Those claims come from analytic residuals, the generic Prism6 face census, positive Jacobians, format round trips, and the native-BREP checks in `validate_topology` and `validate_geometry`.

OpenFOAM Mesh Handoff

Source: `docs/operations/openfoam-handoff.org`

This stage audits mesh conversion only. It does not create CFD boundary conditions, material properties, journal rotation, cavitation, turbulence, or a solver run.

This is a boundary of the canonical **mesh-generation stage**, not the current boundary of the whole repository. The later S100 import, solver commissioning, low-speed continuation, and guarded saturation-threshold campaign are recorded separately in the OpenFOAM CFD handoff.

Modes

Both volume-mesh CLIs support:

`--openfoam auto` run conversion and `checkMesh` when both utilities are found; otherwise record `SKIPPED` without failing mesh generation.
`--openfoam required` absence of either utility, a conversion warning, or a failed audit is fatal.
`--openfoam skip` do not attempt conversion; record `SKIPPED`.

The current production reports were generated with `--openfoam skip`, so their OpenFOAM status is intentionally `SKIPPED`. That does not weaken the native mesh PASS, but those reports are not the later S100 import evidence. The S100 study performed a separate accepted conversion and standard `checkMesh` audit.

Check availability first:

```
command -v gmshToFoam
command -v checkMesh
command -v foamVersion || true
```

Canonical conversion inputs

Use only the Gmsh 2.2 ASCII files written for conversion:

- no-port reference: `nGap_XX/structured_hex_openfoam.msh`;
- central-feed production mesh: `nGap_XX/ported_prism_openfoam.msh`.

Do not convert the Gmsh 4.1 binary visualization file, VTU, surface-smoke mesh, cutaway, section, mouth diagnostic, or exaggerated visualization.

The MSH2 files include exterior surface elements aligned with volume-cell faces and stable physical groups. `Mesh.SaveAll` is not enabled because it can discard physical definitions in MSH2.

Audited command

```
uv run python meshing/layered_prism_central_feed.py \
  --params out/strict_default/params.json \
  --brep out/strict_default/film_unsplit.brep \
  --preflight out/gmsh_preflight/preflight_report.json \
  --outdir out/ported_prism_default \
  --openfoam required --no-gui
```

For a machine where OpenFOAM is optional, replace `required` with `auto`. The implementation captures executable paths, help/version output, stdout, stderr, conversion logs, `checkMesh` logs, cell count, connected-region count, and patch inventory in `mesh_report.json`.

The corresponding code is:

- ported Prism6 `audit_openfoam`;
- no-port Hex8 `audit_openfoam`;
- auto-skip regression.

Underlying utilities

```
gmshToFoam -case CASE ported_prism_openfoam.msh
checkMesh -case CASE -allTopology -allGeometry
```

Required audit evidence includes:

- `gmshToFoam` return code zero;
- no unhandled, inverted, undefined, unmatched, fatal, or dropped-element diagnostic;
- `checkMesh` return code zero and an explicit `Mesh OK.` line;
- expected cell count and exactly one connected fluid region;
- no negative-volume, illegal, or inverted cells;
- exact patch set and no `defaultFaces`.

If the installed `gmshToFoam` cannot convert Prism6 cells, `required` mode fails clearly. The pipeline must not tetrahedralize or omit cells as a fallback.

Expected physical groups

1. No-port Hex8 reference

ID	Name	Role
101	`journal_wall`	moving wall in a future CFD case
102	`stationary_wall`	stationary bore wall
103	`axial_end_z0`	axial opening
104	`axial_end_zL`	axial opening

ID	Name	Role
201	`fluid`	one volume region, not a boundary patch

There is no `pressure_feed` because the reference geometry has no feed port.

2. Central-feed Prism6 production mesh

ID	Name	Role
101	`journal_wall`	moving wall; continuous under the feed
102	`bushing_bore_wall`	stationary bore outside the mouth
103	`axial_end_z0`	axial opening
104	`axial_end_zL`	axial opening
105	`feed_tube_wall`	stationary feed wall
106	`pressure_feed`	external inlet disk
201	`fluid`	one connected volume region, not a patch

The feed mouth is a shared internal owner-neighbour interface with two incident cells. It must not appear as `feed_mouth`, `mouth_cap`, `internal_feed`, or `defaultFaces`. There must be no symmetry, cyclic, or circumferential seam patch.

Reading the result

```
jq '.openfoam' out/ported_prism_default/nGap_08/mesh_report.json
jq '.gmsh.physical_groups' out/ported_prism_default/nGap_08/mesh_report.json
find out/ported_prism_default/nGap_08 -maxdepth 1 \
  -name 'openfoam_*.log' -print
```

A `PASS` here means format conversion and mesh topology/geometry passed the installed OpenFOAM utilities. It still does not mean that a physically valid CFD problem has been configured or solved. Solver evidence must instead meet the acceptance boundaries stated in the CFD handoff.

External documentation

- OpenFOAM Foundation source guide: `gmshToFoam` directory reference
- Gmsh 4.15.2 reference: mesh formats and physical groups

Concepts

Project Glossary

Source: `docs/concepts/glossary.org`

Geometry representation

1. B-rep

   A boundary representation stores a solid through analytic or parametric faces, their trimming edges, vertices, orientation, and adjacency. It is not merely a surface triangle collection. OCCT is the B-rep kernel beneath build123d and Gmsh's OpenCASCADE interface in this project.

   See CAD construction.

2. Topology versus geometry

   Geometry answers where a curve or surface lies. Topology answers which vertices bound an edge, which edges bound a face, and which faces bound a solid. Two coincident faces can be geometrically identical but topologically unshared; a conformal mesh needs the appropriate shared topology or shared nodes.

3. Manifold solid

   Locally, a manifold solid boundary separates one inside from one outside. A non-manifold edge or face has an ambiguous number of incident regions and is a bad fluid-volume contract.

4. p-curve

   A parametric curve describing a trimming edge in the two-dimensional parameter space of a surface. STEP can carry p-curves to help receiving kernels rebuild trimmed faces. They improve interchange information but do not guarantee an identical re-import.

5. Boolean fragment versus fuse

   A fuse replaces overlapping solids by their union. A fragment operation splits intersecting entities while retaining the material pieces and establishing shared interfaces. The CAD film uses fuse/subtract; the three imported axial zones use Gmsh OCC fragment to enforce conformal shared faces.

Gmsh entity layers

1. OCC entity

   A CAD-kernel vertex, curve, surface, or volume imported or constructed through `gmsh.model.occ`. It has a dimension and a model-local tag. Tags are not stable semantic identifiers, so this project classifies OCC entities by geometry and adjacency.

2. Discrete entity

   A Gmsh model entity whose geometry is represented directly by an existing mesh. The analytic Hex8 and Prism6 pipelines register completed nodes and elements on discrete curves, surfaces, and volumes. Gmsh is not asked to generate those 3-D cells.

3. Physical group

   A named collection of model entities written into mesh formats. It is the semantic bridge to solver regions and patches. A physical group is not itself a boundary condition; OpenFOAM dictionaries later assign physical behaviour to the resulting patch name.

4. Periodic seam

   The duplicate boundary of the unwrapped master domain at `theta=0` and `theta=2*pi`. Periodic node identification collapses the slave representation onto the master, so the physical full-360 mesh has no external seam patch.

Mesh topology

1. Conformal interface

   Both sides use the same face connectivity and node tags. There are no gaps, overlaps, hanging nodes, or coincident-but-independent faces. At the feed mouth the film and tube share each Tri3 exactly.

2. Face incidence, owner, and neighbour

   A volume cell is incident to each of its faces. A face census gives:

   - one incident cell: exterior boundary face; that cell is the owner;
   - two incident cells: internal face; finite-volume terminology calls them owner and neighbour after choosing an ordering;
   - more than two incident cells: non-manifold mesh topology.

   The shared feed-film mouth is therefore an internal owner-neighbour face and the numerical expression of an open, continuous fluid passage—not a wall. See the ported mesh.

3. CGNS zone and boundary condition

   A CGNS `Zone_t` contains one structured or unstructured grid and its coordinate and element sections. Boundary memberships live in =ZoneBC$_t$=/family structures; a JSON sidecar cannot replace those definitions for an importer. See Formats and CGNS.

4. PENTA$_6$

   The CGNS name for a linear six-node pentahedral/wedge cell. It is the format equivalent of this repository's Prism6, subject to node-order validation during round trip and live import.

5. Hex8

   A first-order hexahedron with eight corner nodes. The no-port baseline follows logical circumferential, axial, and gap directions and therefore admits a structured Hex8 layout.

6. Tri3 and Prism6

   A Tri3 is a first-order triangle. Sweeping its three nodes between two layers creates a six-node wedge/prism (Prism6). The ported mesh uses a common Tri3 master topology for the film and feed disk, then creates Prism6 cells explicitly.

7. Boundary element

   A Tri3 or Quad4 explicitly written on an exterior volume face so an exchange tool can recover a named patch. An internal face such as the feed mouth must not be written as a solver boundary element.

Geometry approximation in a linear mesh

1. Nodal exactness

   A boundary node can satisfy the analytic cylinder or cone equation to floating point tolerance. This says nothing about the straight edge between two nodes.

2. Chord sagitta

   The maximum distance between a circular arc and its straight chord. For feed radius `r_h` and `N` equal rim segments,

   $$s = r_h \left(1 - \cos\frac{\pi}{N}\right).$$

   The ported mesh reports both nodal residual and sagitta because they measure different approximation errors.

3. Faceted volume error

   The relative difference between the volume enclosed by linear mesh faces/cells and the analytic or native-BREP volume. Circumferential and circular-boundary refinement should reduce it even when every cell is already topologically valid.

Cell validity and quality

1. Signed volume

   An oriented cell volume. A negative result indicates reversed connectivity; zero indicates degeneration. Positivity is mandatory.

2. Jacobian determinant

   The determinant of the mapping from a reference element to the physical cell. It must remain positive at the checked integration points. A positive total volume alone cannot exclude a locally inverted mapping.

3. `minDetJac`

   Gmsh's minimum sampled Jacobian determinant quality. This project requires it to be finite and positive and cross-checks it with custom calculations.

4. `minSICN`

   Gmsh's signed inverse condition number metric. Positive values indicate valid orientation. The thin-film elements are intentionally highly anisotropic, so the metric can be small; the code reports its distribution rather than imposing an inappropriate isotropic threshold such as `0.1`.

5. Non-orthogonality

   For a finite-volume face, the angle between the face-area vector and the vector joining adjacent cell centres. Large values weaken the direct normal-flux approximation and increase correction requirements.

6. Skewness

   A measure of how far the face geometry is displaced from an ideal interpolation location between neighbouring cells. It is distinct from aspect ratio.

7. Face-pyramid volume

   The oriented volume of the pyramid from a cell centre to one of its faces. A positive minimum helps detect concave or inverted local cell geometry.

Artifact eligibility

1. Solve-eligible artifact

   A true-coordinate complete mesh that passed all mandatory checks and may enter the next solver-conversion stage. Cutaways, internal-interface views, and any distorted visualization carry `solve_eligible=0` and are never substitutes for the canonical mesh.

References

Reference Map

Source: `docs/references/index.org`

Literature the conical-bearing comparison and primary JFO verification sources, with the narrow claims each supports.
Software pinned environment, official manuals, and tool roles.
Provenance data lineage, hashes, stage status, and external links.

Literature

Source: `docs/references/literature.org`

Project literature index

The parent BTP survey is maintained in `docs/papers_sorted/index.org`. This project uses the local conical-bearing paper for a narrow model comparison and primary lubrication papers for JFO code verification.

Gangrade, Phalle, and Mantha

Ajay Kumar Gangrade, Vikas M. Phalle, and S. S. Mantha, "Performance Analysis of a Conical Hydrodynamic Journal Bearing," *Iranian Journal of Science and Technology, Transactions of Mechanical Engineering*, volume 43, pages 559–573; published online in 2018.

- DOI: 10.1007/s40997-018-0217-2
- Local author copy: Performance Analysis of a Conical Hydrodynamic Journal Bearing.pdf

1. Relevance to this project

   The paper analyzes the lubricant clearance using a modified Reynolds equation in a conical/spherical-coordinate formulation and solves the resulting thin-film problem by finite elements on a developed conical surface. It motivates the periodic circumferential/axial master coordinates and the thin-film context. It does not validate this project's three-dimensional feed, native-BREP assurance, or Tri3-to-Prism6 construction; those are local design decisions implemented in the local master-surface construction and explicit Prism6 construction.

JFO and complementarity verification

- Elrod, "A Cavitation Algorithm," *Journal of Lubrication Technology* 103 (1981), 350–354: 10.1115/1.3251669.
- Giacopini et al., "A Mass-Conserving Complementarity Formulation to Study Lubricant Films in the Presence of Cavitation," *Journal of Tribology* 132 (2010), 041702: 10.1115/1.4002215.
- Miraskari et al., "A Robust Modification to the Universal Cavitation Algorithm in Journal Bearings," *Journal of Tribology* 139 (2017), 031703: 10.1115/1.4034244.

The local automated test checks broad graph-read bands around the 60-cell sinusoidal slider peaks and rupture/reformation locations shown by Miraskari et al. Their compressible Elrod calculation uses a 100 GPa bulk modulus and is compared with Giacopini's incompressible LCP; the local test directly assembles that incompressible limit. This is a coarse check of the shared active-set kernel, not a digitized pressure-profile comparison or validation of the conical model. Exact inputs and results are recorded in JFO validation and Fluent path.

Manufactured solution and grid convergence

Gravenkamp, Pfeil, and Codina publish a Reynolds–Elrod manufactured solution and its forcing data:

- paper: 10.1016/j.cma.2023.116488;
- code/data: 10.5281/zenodo.8315789.

Reusing it requires a separate diagnostic mode with the published smooth regularization and forcing. The paper's second-order result applies to its stabilized linear finite elements; its artificial-diffusion result is first order, which is the relevant initial expectation for the current upwind finite volume scheme. The local observed order must still be measured. Grid convergence must then follow the procedure of Celik et al., 10.1115/1.2960953, after the physical feed footprint is made consistent across refinements.

Clearance terminology

The project's same-`z` radial-clearance definition and its distinction from surface-normal distance are derived in Clearance mathematics.

Software References

Source: `docs/references/software.org`

Reproducible Python environment

The authoritative dependency declarations are `pyproject.toml`, `uv.lock`, and `requirements.txt`; the environment is managed with uv.

Component	Declared version	Role	Official reference
Python	`⩾3.10,<3.14`; local selection 3.12	Runtime	Python documentation
build123d	0.11.1	Parametric OCCT BREP CAD, booleans, BREP/STEP import and export	build123d 0.11.1 documentation
Gmsh	4.15.2	OCC preflight, 2-D master/smoke meshing, discrete mesh registration, quality, MSH and CGNS I/O	Gmsh 4.15.2 reference manual
Matplotlib	3.10.7	Static geometry and quality diagnostics	Matplotlib documentation
meshio	5.3.5	Float64 VTU export and round-trip import	meshio repository
ocp-vscode	4.0.0	Optional browser CAD preview	OCP CAD Viewer repository
pytest	9.1.1	Development regression tests	pytest documentation

NumPy is imported directly by both volume-mesh implementations but is currently resolved transitively rather than declared as a top-level dependency.

CAD and geometry kernel

build123d provides Python access to Open Cascade Technology (OCCT). The production geometry is exact BREP; STL, tessellated offsets, and mesh-derived CAD are not used. The CAD uses documented shape validity/manifold properties and writes native BREP as the trusted open-source interchange.

Relevant local code:

- BREP/STEP export;
- Gmsh OCC import and preflight;
- independent ported-mesh BREP check.

FreeCAD is an external manual viewer for BREP/STEP inspection. It is not a runtime dependency or an automated acceptance oracle in this repository. Official site: FreeCAD.

Gmsh contracts

The project pins Gmsh 4.15.2. The official 4.15.2 manual documents the Python API, OpenCASCADE kernel, physical groups, discrete entities, periodic curves, quality queries, and MSH formats used here:

- HTML manual: https://gmsh.info/doc/texinfo/
- PDF manual: https://gmsh.info/doc/texinfo/gmsh.pdf

Gmsh generates 2-D triangles for the smoke and ported master-surface stages. The no-port Hex8 and ported Prism6 volume connectivity are calculated in NumPy and registered as complete discrete meshes; neither pipeline calls Gmsh's 3-D mesh generator.

The solver-neutral exporter additionally requires `Cgns` in Gmsh's compiled `General.BuildInfo`. It uses `gmsh.write` and `gmsh.open` for the CGNS round trip and checks exact physical names/memberships after reopening. See Formats and CGNS.

CGNS and Fluent

CGNS is the neutral unstructured-mesh representation used for the optional Fluent handoff. The relevant standard structures are documented by the CGNS SIDS:

- zone and hierarchy structures;
- coordinates and element sections.

PyFluent is intentionally optional and is not added to `pyproject.toml`. A machine without Ansys should still export and statically validate all neutral files without installing proprietary tooling. When installed externally, the audit uses official APIs for:

- 3-D/double-precision launch;
- CGNS volume-mesh import;
- mesh and quality checks.

Ansys's Fluent user guide documents the GUI route as **File** > **Import** > **CGNS** > **Mesh** and recommends verifying cell and boundary zones after third-party import: Fluent file import.

CGNS import may require the Ansys/VKI license. Package detection is therefore not equivalent to a usable runtime; only a successful live launch and import can produce `FLUENT_IMPORT_PASS`.

Visualization

Gmsh opens `.msh` and `.pos` files. ParaView opens the float64 `.vtu` files; official documentation is at docs.paraview.org. On Artix Linux/Wayland the project uses `QT_QPA_PLATFORM=xcb` to avoid known display-path issues. PyVista is not required.

OpenFOAM

OpenFOAM is not a Python dependency. The mesh-generation scripts discover `gmshToFoam` and `checkMesh` only for the optional conversion audit. The downstream S100 study used a local OpenFOAM 14 source build; exact source and ThirdParty revisions, environment setup, and build evidence are recorded in the OpenFOAM build handoff.

The OpenFOAM Foundation source guide describes `gmshToFoam` as reading Gmsh `.msh` files:

- OpenFOAM Foundation `gmshToFoam` directory reference

Because distributions and utility capabilities vary, reports record executable paths and runtime version/help output. Absence is `SKIPPED` in `auto` mode and fatal in `required` mode.

Provenance and Evidence Boundaries

Source: `docs/references/provenance.org`

Scope

This record covers the reproducible CAD, Gmsh preflight, surface-smoke, structured Hex8, central-feed Prism6, solver-neutral interchange, and downstream OpenFOAM evidence in this repository. It records where claims come from and prevents screenshots, rejected formats, or an unavailable conversation link from becoming unstated technical evidence.

Data lineage

```
bearing_film.py + CLI inputs
  → params.json + film_unsplit.brep + film_zones.brep (mm)
  → gmsh_brep_preflight.py
       → validated hashes + fragmented BREP (mm) + checked SI BREP (m)
       → gmsh_surface_smoke.py → 2-D Tri3 smoke mesh (m)
  → structured_hex_no_port.py → analytic no-port Hex8 reference (m)
  → layered_prism_central_feed.py
       → analytic film/tube Prism6 mesh (m)
       → node and volume comparison against film_unsplit.brep
       → mesh_arrays.npz + manifest/report
            → export_solver_neutral.py
```

```
                 → clean MSH 4.1 + MSH 2.2 + CGNS (m)
                 → static format round trips
                 → optional live Fluent import/check

validated S100 surface-inlet Hex8 mesh
  → gmshToFoam + patch correction + checkMesh
  → zero-flow solver proof
  → isolated pressure-fed and zero-pressure-bias OpenFOAM cases
  → accepted checkpoints + rejected diagnostics + guarded ramp state
```

The ported mesh reads `resolved_parameters` from `params.json`, including the actual `y_feed_end`. It verifies the native BREP SHA-256 against both the CAD record and preflight record before meshing. It does not hardcode the default inlet coordinate.

Contract implementation:

- ported input/hash contract;
- native BREP validity, volume, and bounds;
- mesh-node closest-point checks;
- open-mouth and one-region topology proof.

Current recorded status boundary

Evidence	Status	Permitted conclusion
Native build123d/OCCT geometry and BREP round trip	PASS checks inside strict CAD report	Native geometry is suitable as the downstream reference.
STEP exchange at relative volume tolerance `1e-6`	FAIL	Do not mesh or promote the rejected STEP.
BREP import, fragmentation, disk re-import, unit scaling	PASS	Gmsh OCC sees the expected native geometry and shared interfaces.
2-D surface smoke mesh	PASS	Boundary meshing/classification works; this says nothing about a 3-D volume mesh.
No-port analytic Hex8 family	PASS	Frozen full-360 no-feed regression reference.
Central-feed analytic Prism6 family	PASS	Conformal full-360 ported volume mesh passes its recorded topology, geometry, quality, and format gates.
Solver-neutral MSH/CGNS for default nGap_{04} and nGap_{08}	STATIC PASS	Canonical mesh, exact six boundary memberships, metre scale, and internal mouth survive Gmsh/CGNS round trips.
Canonical Fluent import audit	NOT ACCEPTED	The earlier S100 Fluent solve diverged; it is corroborating failure evidence, not `FLUENT_IMPORT_PASS`.
OpenFOAM conversion in canonical production mesh reports	SKIPPED	Those reports used `--openfoam skip`; this remains true for that artifact family.
Separate S100 OpenFOAM import	ACCEPTED	`gmshToFoam`, patch correction, and standard `checkMesh` accepted 294,912 Hex8 cells and the intended patches.
Zero-flow and 0-rpm pressure-fed commissioning	ACCEPTED	Solver plumbing is proven; the 0.5 MPa case is pressure-driven, not pure hydrodynamic operation.
200-rpm single-liquid run and exploratory pressure-fed cavitation runs	REJECTED	Retain as failure evidence; do not report as bearing performance.
Zero-bias OpenFOAM continuation	ACCEPTED THROUGH 20 RPM	Checkpoints remain bounded, conservative, and single-liquid through the accepted 20-rpm hold.
First 20-to-50-rpm update	$\mathrm{SATURATION}_{\mathrm{THRESHOLD}}$ at 20.2 rpm	The configured `pSat` threshold was crossed before cavity evolution; no two-phase result is accepted.

Evidence	Status	Permitted conclusion
Native Reynolds/JFO continuation	UNVALIDATED CANDIDATE	The active-set kernel passes a coarse published graph-consistency check; the conical feed representation, native assembly, grid convergence, and physical cavity interpretation remain unvalidated.

The canonical mesh reports and the downstream CFD cases are separate evidence families. A skipped optional audit in an older mesh report does not erase the later S100 import, and a successful S100 import does not retroactively promote the canonical Fluent interchange audit.

Hash and unit provenance

Use the reports rather than copying hashes into prose:

```
jq '.sha256' out/strict_default/params.json
jq '.outputs' out/gmsh_preflight/preflight_report.json
jq '{overall,coordinate_unit,counts,solve_eligible,distorted_geometry}' \
  out/ported_prism_default/nGap_08/manifest.json
jq '.files' out/ported_prism_default/nGap_08/mesh_report.json
sha256sum out/ported_prism_default/nGap_08/ported_prism.msh \
  out/ported_prism_default/nGap_08/mesh_arrays.npz
jq '{overall,readiness,source,exports,fluent}' \
  out/interchange_default/nGap_04/interchange_report.json
jq '.' out/interchange_default/nGap_04/file_hashes.json
```

CAD/native-BREP coordinates are millimetres. Solver mesh coordinates are metres. Every mesh report records the conversion factor and bounding box so a second application of `0.001` is detectable.

External references consulted

- build123d official documentation: build123d 0.11.1 documentation;
- Gmsh official documentation: Gmsh 4.15.2 reference manual;
- CGNS SIDS: hierarchy and grid/element structures;
- Fluent CGNS import: Ansys Fluent file import;
- PyFluent launch/import APIs: launch guide;
- OpenFOAM Foundation source guide: `gmshToFoam` reference;
- publisher DOI record for the local paper: 10.1007/s40997-018-0217-2.
- published complementarity benchmark: Miraskari et al. 2017 and Giacopini et al. 2010;
- Reynolds–Elrod manufactured solution and open forcing data: Gravenkamp et al. 2024 and Zenodo 8315789;
- primary grid-convergence procedure: Celik et al. 2008;
- Ansys Fluent theory for liquid–vapour cavitation and operating versus absolute pressure.

These references explain software APIs or literature context; local validation records establish this project's numerical acceptance.

Conversational provenance

Project ChatGPT conversation — unavailable when these docs were written; no engineering claims were derived from it.

Maintenance

Maintaining the Org Documentation

Source: `docs/contributing.org`

Maintenance contract

These files are a shared engineering notebook, not generated API prose. A useful change should make a physical assumption, algorithm, trust boundary, or failure mode easier to inspect alongside the code.

Keep documents focused. Add a new file when a topic has its own construction method or validation contract; do not grow `index.org` into the manual itself.

File and link conventions

- Use lowercase hyphenated filenames.
- Give every file `#+TITLE`, `#+STARTUP`, and `#+OPTIONS` headers.
- Use relative `file:` links so the tree remains relocatable.
- Prefer a source anchor over a bare filename:

```
[[file:../bearing_film.py::def make_full_film][make_full_film]]
```

- Link a test when describing a regression contract.
- Link a generated report only as evidence; reports are not source files.
- Use `mm` for CAD derivations and explicitly mark the one `0.001` conversion into solver-space metres.

Claim discipline

Evidence requirements by claim type:

Kind	Required support
geometry definition	formula plus construction-function link
implementation behavior	exact source-function link
mandatory invariant	validator and test link
current run status	named JSON report and date/case
literature statement	paper citation and precise scope
future proposal	label as proposed; do not write as implemented

Do not transfer claims between pipeline stages. A native-BREP PASS is not a STEP PASS; a topology PASS is not a quality PASS; a `checkMesh` PASS is not a validated CFD model. Likewise, writing and reopening CGNS proves a static interchange round trip, not Fluent acceptance. Use `STATICALLY_VALIDATED_NOT_IMPORTED` until the live checks in the Fluent import audit pass against the canonical source.

Decision changes

For a geometry-meaning or topology change:

1. add or update an entry in the decision register;
2. update the detailed subsystem chapter;
3. link the implementation and regression test;
4. update the trust-state table only after a recorded run;
5. preserve rejected alternatives and consequences so later readers understand why the code is shaped this way.

Literature and conversational sources

Papers motivate physics and numerical approaches, but they do not prove this implementation. State exactly which part of a paper is being used. Project conversation links belong in provenance; when their content is unavailable, record that fact rather than reconstructing it from memory.

Documentation check

```
emacs --batch -Q --eval '(require (quote org))' \
  --eval '(setq org-confirm-babel-evaluate nil)' \
  --eval '(mapc (lambda (f) (with-current-buffer (find-file-noselect f) (org-lint))) (directory-files-recursively "do
```

`org-lint` checks structure; source-anchor resolution remains a separate check.

Updating commands and output layouts

CLI defaults are defined in each `parse_args` function, and output names are defined in the corresponding export/generation function. Verify both before editing command examples. Do not document a manually created diagnostic file as a reproducible pipeline output unless the source now creates it.

Style

Prefer equations, small code excerpts, and explicit failure examples over decorative prose. Explain why a line or invariant exists; readers can inspect obvious Python mechanics in the linked source. Keep visualization subsets clearly marked non-solver and never use their coordinates as validation data.

OpenFOAM CFD handoff

OpenFOAM Bearing CFD Handoff

Source: `docs/handoff/00-index.org` *Updated: [2026-07-29 Wed]*

Next: OpenFOAM build

Purpose

This is the complete handoff for the OpenFOAM work started on 2026-07-25. It is split into small files so that a result, command, assumption, or failure can be opened directly without searching one very large document.

The short conclusion is:

Stage	Status	What it proved
OpenFOAM 14 source build	accepted	The solver and conversion tools run locally
S100 mesh import	accepted	The validated Hex8 mesh and five named boundaries survive conversion
Zero-flow case	accepted	Fields, patches, numerics, and solver plumbing are correct
0 rpm, 0.5 MPa feed	accepted pressure-only commissioning case	A mass-balanced stationary pressure-driven state converges; it is not pure hydrodynamic
200 rpm single-liquid case	rejected	It predicts pressure below absolute vacuum; the physical model is incomplete
Pressure-fed Schnerr-Sauer smoke test	exploratory and rejected	Mapping time 285 improved initialization, but phase and mass balance did not converge
0 rpm, zero bias, cavitation active	accepted equilibrium	The exact zero-flow field survives the phase model without using time 285
1 rpm, zero bias	accepted continuation checkpoint	Rotation alone creates a bounded hydrodynamic field; no vapour forms at this low speed
5 rpm, zero bias	accepted continuation checkpoint	Pressure, flow balance, and loads settle; pressure remains positive and no vapour forms
10, 12, 14, and 15 rpm, zero bias	accepted continuation checkpoints	The low-speed liquid branch remains bounded, balanced, and above saturation
20 rpm, zero bias	accepted continuation checkpoint	The liquid branch settles at `p_min=1836.89 Pa` with clean conservation and `alpha.oil=1`
First 20-to-50-rpm ramp update	guarded stop	At 20.2 rpm, `p_min=-0.177 Pa < pSat`; this brackets the schedule-dependent saturation threshold but is not yet a cavity solution
Direct 10 to 15 rpm restart	rejected diagnostic	The discontinuous wall-speed update briefly crosses `pSat` and triggers a runaway phase/pressure feedback
Native 10 to 15 rpm `omega` ramp	accepted diagnostic	The same target speed is reached with Schnerr-Sauer active, `alpha.oil` equal to one, and clean continuity
Native Reynolds/JFO midsurface model through 2000 rpm	**unvalidated candidate**	Its active-set kernel passes a coarse published graph-consistency check, but the full conical model has a grid-dependent feed mask and its 64.455% severe rupture region is not physically validated

The 200 rpm failure is useful evidence. Rotation creates a low-pressure diverging-film region. A liquid-only CFD model tries to continue that pressure below physical vacuum. This is why the next question is not merely ”which solver setting is wrong?” but ”what film-rupture physics represents this oil and its dissolved or entrained gas?”

The current branch starts from the exact zero-flow field, fixes all three openings at the same ambient absolute pressure, and activates cavitation at 0 rpm. The 10-to-15-rpm audit showed that abrupt restart changes are the numerical trigger. Future speed changes therefore use OpenFOAM's native continuous `omega` function followed by a constant-speed hold.

Reading order

1. OpenFOAM build – AUR failures, source build, exact revisions, and environment.
2. S100 mesh import – the mesh selected, physical patches, conversion, and quality checks.
3. Zero-flow proof – the smallest solver test and what it does and does not prove.
4. Paper inputs – every physical value, unit conversion, mismatch, and assumption.
5. Single-phase results – the accepted 0 rpm state and rejected 200 rpm diagnostic.
6. Cavitation model – why this model was tried and which inputs are only placeholders.
7. Cavitation results – uniform start, mapped start, continuation, and the interesting physics.
8. Replay commands – copy-and-run commands in one place.
9. Artifact map – every human-authored file and every generated file family.
10. Problems and decisions – symptom, cause, response, and the revised experiment.
11. Zero-pressure-bias ramp – clean seed, equal-pressure boundaries, accepted checkpoints, and continuation policy.
12. Five-rpm jump diagnosis – controlled experiments that isolate the direct-jump failure and the native-ramp solution.
13. Guarded ramp automation – one command that continues toward the paper speeds, holds for convergence, and stops at the first unvalidated phase onset.
14. Native OpenFOAM Reynolds/JFO candidate – converged high-speed candidate, severe-cavity warning, narrow paper comparison, and validation blockers.
15. JFO validation and Fluent path – published kernel benchmark, grid/feed audit, remaining numerical gates, and the separate Fluent vapour-or-air model.

Evidence locations

- Source tree: OpenFOAM-14
- Third-party tree: ThirdParty-14
- Imported S100 mesh: structured$_{\text{hexopenfoam}}$.msh
- Zero-flow proof: openfoam$_{\text{s100zeroflow}}$
- Paper-grounded single-phase case: openfoam$_{\text{s100paperbaseline}}$
- Exploratory cavitation case: openfoam$_{\text{s100cavitationexploratory}}$
- Zero-pressure-bias hydrodynamic case: openfoam$_{\text{s100purehydrodynamic}}$
- Native Reynolds/JFO source: jfoBearingFoam.C
- Generated native Reynolds/JFO case: `out/openfoam_jfo_native_256x80/`
- Direct conical-bearing paper: Performance Analysis of a Conical Hydrodynamic Journal Bearing
- Earlier Fluent S100 log: log-inlet-100.txt

Meaning of status words

- **Accepted** means the stage passed the checks appropriate to its stated purpose. It does not promote a zero-flow test into a physical bearing result.
- **Exploratory** means the setup is useful research work but contains uncalibrated inputs or lacks convergence.
- **Rejected** means it is retained as evidence of a failure mode and must not be reported as bearing performance.
- **Unvalidated candidate** means numerical checks passed inside the current implementation, but full-model external verification and valid grid-convergence evidence are still missing. It must not be promoted to a validated physical result.

Version-control boundary

The handoff documents are now intended to be versioned. Generated CFD case directories remain under the ignored `out/` tree; only selected plots, animations, and compact result ledgers should be promoted into documentation.

Next: OpenFOAM build

01 - Building OpenFOAM 14

Source: `docs/handoff/01-openfoam-build.org`

Index | Next: mesh import

Why we built from source

The first attempt used Arch packaging:

```
pacman -Ss openfoam
pacman -S openfoam-org
paru -S openfoam-org
```

`pacman -Ss` returned no repository package. The direct installation then failed correctly because it was not run as root:

```
error: you cannot perform this operation unless you are root.
```

The first AUR retry failed while cloning Zoltan:

```
fatal: unable to access 'https://aur.archlinux.org/zoltan/':
OpenSSL SSL_read: OpenSSL/3.6.3: error:0A000126:SSL routines::unexpected eof
while reading, errno 0
```

That was a network transfer failure, so retrying was reasonable. The second attempt got further: it installed `fast_float`, `scotch`, `boost`, `eigen`, `cgal`, and `utf8cpp` and built `gklib` and `metis`. It then failed in the AUR `parmetis` recipe:

```
CMake Error: The warning category "l,-z,relro,-z,now" is not known.
make: *** [Makefile:77: config] Error 1
==> ERROR: A failure occurred in prepare().
error: failed to build 'parmetis-1:4.0.3+p7-1':
error: can't build zoltan-3.901-1, deps not satisfied: parmetis
error: can't build openfoam-13.20250911+cde978a-1 (openfoam-org),
deps not satisfied: parmetis  zoltan
```

This was not an OpenFOAM solver failure. The AUR `parmetis` recipe passed linker flags as separate CMake arguments, and CMake 4.4 interpreted `-Wl,...` as a warning-category option. Patching a chain of out-of-tree packages would have added more uncertainty than building the Foundation release by its documented route.

OpenFOAM's own Linux instructions say that users of non-Ubuntu distributions should compile the source. See OpenFOAM 14 for Linux, source download, and third-party software.

Source checkout

The source was kept inside the project workspace rather than installed system-wide:

```
mkdir -p /home/aniketnegi/code/ankt/btp/.openfoam
cd /home/aniketnegi/code/ankt/btp/.openfoam

git clone --depth 1 https://github.com/OpenFOAM/OpenFOAM-14.git
git clone --depth 1 https://github.com/OpenFOAM/ThirdParty-14.git
```

The exact revisions used are:

Tree	Revision
OpenFOAM-14	`7b05503f98a85be88af930df48623b4d152bfc35`
ThirdParty-14	`6f75cb6aa1a3d8f1ba768840f9ee535244b4c70a`
OpenFOAM build label	`14-7b05503f98a8`

Verify the local checkout:

```
git -C /home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14 rev-parse HEAD
git -C /home/aniketnegi/code/ankt/btp/.openfoam/ThirdParty-14 rev-parse HEAD
cat /home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14/.build
```

First build problem: environment not loaded

The first build command was:

```
cd /home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14
./Allwmake -q -j 6
```

It stopped immediately:

```
Allwmake error: The OpenFOAM environment is not set.
Check the OpenFOAM entries in your dot-files and source them.
```

The solution was to source the release environment in the same shell:

```
cd /home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14
source etc/bashrc
./Allwmake -q -j 6
```

`Allwmake` called the adjacent `ThirdParty-14/Allwmake` automatically; no separate third-party build command was run. The main build began at 2026-07-25 11:45:19 UTC and was verified complete at 13:58:37 UTC, about two hours and thirteen minutes later. An incremental repeat completed in about 17 seconds. These are historical observations, not portable build-time promises.

Build warning and why it did not block us

The managed execution environment printed:

```
mkdir: cannot create directory '/home/aniketnegi/.OpenFOAM':
Read-only file system
```

The source and platform output directories under the writable workspace were still built. A normal interactive user shell has a writable home directory, so this sandbox-specific warning should not appear there.

ParaView preference

Before the final build environment was settled, Bash emitted:

```
.../etc/config.sh/paraview: line 95: [: ≠: unary operator expected
```

OpenFOAM had defaulted to `ParaView_TYPE=system`. We did not need to compile OpenFOAM's ParaView reader because `foamToVTK` and the system ParaView application cover this project. The site preference was:

.openfoam/site/14/etc/prefs.sh

```
export ParaView_TYPE=none
```

The verified environment then used `SYSTEMOPENMPI`, ThirdParty Scotch/PT-Scotch and Zoltan, and `ParaView_TYPE=none`.

Verification

Running the same build command again returned successfully and mostly said that no work was required. The third-party summary included:

```
ThirdParty installation of Scotch decomposition found
ThirdParty installation of PT-Scotch decomposition found
METIS decomposition is disabled
ParMETIS decomposition is disabled
ThirdParty installation of Zoltan decomposition found
Done ThirdParty Allwmake
```

ParMETIS was not required for the serial tests or the four-way decomposition used here. Scotch was available and was used by `decomposePar`.

The active build reports:

```
WM_PROJECT_VERSION=14
WM_OPTIONS=linux64GccDPInt32Opt
```

The required executables were verified:

```
bash -lc '
  source /home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14/etc/bashrc
  foamVersion
  for command_name in foamRun gmshToFoam checkMesh decomposePar \
      reconstructPar foamToVTK foamDictionary foamPostProcess; do
    command -v "$command_name"
  done
'
```

The resulting build configuration was Linux/GCC, double precision, 32-bit labels, optimized, and system OpenMPI. Versions recorded on the build machine were GCC 16.1.1, Make 4.4.1, Flex 2.6.4, Bison 3.8.2, CMake 4.4.0, OpenMPI 5.0.10, and Scotch 7.0.12. The local trees occupied about 2.4 GiB for OpenFOAM and 85 MiB for ThirdParty, with 151 executables and 152 shared libraries at audit time.

Zsh activation detail

Sourcing `etc/bashrc` directly in the current zsh currently emits:

```
.../etc/config.sh/paraview:42: no matches found:
.../ThirdParty-14/platforms/linux64Gcc/cmake-*
```

This comes from zsh's unmatched-glob behavior and the absent locally built ParaView/CMake directory. It is not a failure of `foamRun` or `checkMesh`. The shortest reliable approach is to execute OpenFOAM commands through Bash:

```
bash -lc '
  source /home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14/etc/bashrc
  foamVersion
'
```

The build also reported:

```
Warning: ParaView not found in . Skipping.
```

This was consistent with the explicit `ParaView_TYPE=none` preference. It only skipped OpenFOAM's optional locally built ParaView reader. The installed ParaView application and `foamToVTK` were used for visualization.

Local references

- OpenFOAM source README
- ThirdParty source README
- OpenFOAM user guide
- OpenFOAM 14 source API

Index | Next: mesh import

02 - S100 Mesh Selection and OpenFOAM Import

Source: `docs/handoff/02-mesh-import.org`

Previous: build | Index | Next: zero-flow proof

Why S100 was selected

The OpenFOAM baseline used the already validated S100 staircase surface inlet. It was chosen because it was available, solver-neutral, well within the project mesh-quality gates, and close enough to a circular 4 mm inlet for the first solver audit.

This is a **zero-length surface pressure inlet**. It has no feed tube, recess, mouth cap, or hole volume. One hundred bore-surface Quad4 faces are labelled `pressure_feed`; the rest of that bore surface is `stationary_wall`.

Canonical source:

structured$_{\text{hexopenfoam}}$.msh

The exact mesh-generation command recovered from the run transcript was:

```
cd /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
UV_CACHE_DIR=.uv-cache uv run python meshing/structured_hex_surface_inlet.py \
  --params out/strict_default/params.json \
  --outdir out/professor_surface_inlet_variants/inlet100_moderate_ngap12 \
  --n-theta 256 --n-axial 96 --gap-levels 12 --preview-ngap 12 \
  --gap-inflation-ratio 1 --inlet-cluster-strength 0.60 \
  --max-projected-area-relative-error 0.01 \
  --max-inlet-rim-error-mm 0.23 \
  --openfoam skip --no-gui
```

The generator-level result is run$_{\text{report}}$.json, which records an overall `PASS`.

SHA-256:

```
3dcd8dcb8c49a31ded67f69a793d64ed608e1fd20920a0a394b780dc7de6058b
```

Geometry and topology

Quantity	Value
Points	322,816
Hex8 volume cells	294,912

Quantity	Value
Boundary Quad4 faces	55,296
Circumferential master cells	256
Axial master cells	96
Uniform gap layers	12
Nominal inlet diameter	4 mm
Pressure faces	100
Staircase rim edges	48
Projected inlet area	12.6593 mm2
Analytic circular area	12.5664 mm2
Area error	+0.7391%
Equivalent diameter	4.014755 mm

S100 is not a mathematically perfect circle. It is the union of 100 faces on the structured bore grid. The approximation affects the inlet footprint and very local jet/pressure gradients; it does not explain a pressure prediction far below absolute vacuum. That distinction matters in the later 200 rpm diagnosis.

Physical groups

The Gmsh file contains the stable project IDs:

ID	Name	Imported OpenFOAM patch	Face count
101	$\text{journal}_{\text{wall}}$	`journal_wall`	24,576
102	$\text{stationary}_{\text{wall}}$	`stationary_wall`	24,476
103	$\text{axial}_{\text{endz0}}$	`axial_end_z0`	3,072
104	$\text{axial}_{\text{endzL}}$	`axial_end_zL`	3,072
106	$\text{pressure}_{\text{feed}}$	`pressure_feed`	100
201	fluid	cell region	294,912

The corresponding mesh evidence is beside the MSH file:

- manifest.json
- $\text{mesh}_{\text{report}}$.json
- $\text{physical}_{\text{groups}}$.json
- $\text{mesh}_{\text{arrays}}$.npz

The canonical NPZ SHA-256 is:

```
5de6833e0bc06c2757b15553edd2c9ab7bab20c85b06bc62ad51cfb2015ea405
```

Conversion command

The zero-flow case directory was created first, then the mesh was converted:

```
foam_root=/home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14
repo=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
case_dir="$repo/out/openfoam_s100_zero_flow"
mesh_file="$repo/out/professor_surface_inlet_variants/inlet100_moderate_ngap12/nGap_12/structured_hex_openfoam.msh"

bash -lc "
  source '$foam_root/etc/bashrc'
  gmshToFoam -case '$case_dir' '$mesh_file' 2>&1 |
    tee '$case_dir/log.gmshToFoam'
"
```

The import log confirmed the intended format and element types:

```
Read format version 2.2 ascii 0
Physical names:6
Surface 101 journal_wall
...
```

```
Volume 201 fluid
Vertices to be read:322816
...
Cells:
    total:294912
    hex:294912
...
Patch 4 gets name pressure_feed
```

Full evidence: log.gmshToFoam.

Patch-type correction

A historical `createPatch` step was run:

```
createPatch -overwrite -case "$case_dir" 2>&1 |
  tee "$case_dir/log.createPatch"
```

Its log said:

```
Deprecated option 'overwrite' specified, this is now the default behaviour
Patch 'journal_wall' already exists. Only moving patch faces -
type will remain the same
```

The important nuance is that `gmshToFoam` initially made all five surfaces generic `patch` types. `createPatch` did not change the two intended walls from `patch` to `wall`; it only tried to move faces. The actual solution was to edit only `journal_wall` and `stationary_wall` in `constant/polyMesh/boundary`:

```
type            wall;
physicalType    wall;
```

All `nFaces` and `startFace` values were preserved. The axial ends and feed remained `patch`. The current file confirms exactly five patches and no default or seam patch: constant/polyMesh/boundary.

The retained dictionary and log record the unsuccessful historical attempt:

- createPatchDict
- log.createPatch

Mesh checks

The normal solver-facing check was:

```
checkMesh -case "$case_dir" 2>&1 |
  tee "$case_dir/log.checkMesh.standard"
```

Important lines:

```
Number of regions: 1 (OK).
Max aspect ratio = 58.1235901219 OK.
Mesh non-orthogonality Max: 9.99988191989 ...
Max skewness = 0.440812139371 OK.
Mesh OK.
```

The project-native mesh validation also had positive volume, Gauss determinant, face pyramids, Gmsh `minDetJac`, and `minSICN`. Selected values:

Check	Value
Maximum non-orthogonality	9.99988191989 degrees
Maximum skewness	0.440812139371
Minimum cell volume	8.1791235e-13 m3
Total fluid volume	9.42847623656e-7 m3
Gmsh minimum determinant	1.0218615e-13
Gmsh minimum SICN	0.0015049843

An extended diagnostic was also run:

```
checkMesh -case "$case_dir" -allTopology -allGeometry 2>&1 |
  tee "$case_dir/log.checkMesh"
```

It reported:

```
Cell determinant ... minimum:4.2518769755e-10 average:0.000144537317435
***Cells with small determinant (< 0.001) found, number of cells: 286106
Failed 1 mesh checks.
```

This generic normalized determinant threshold flags most deliberately thin film cells. It was not hidden, but it also was not treated alone as proof of inverted cells: the standard OpenFOAM check, exact project geometry checks, positive cell/Gauss/pyramid measures, one-region topology, and later zero-flow conservation all passed. The rotating pressure failure therefore cannot be assigned to a negative-volume mesh without contrary evidence.

Previous: build | Index | Next: zero-flow proof

03 - Zero-Flow Solver Proof

Source: `docs/handoff/03-zero-flow-proof.org`

Previous: mesh import | Index | Next: paper inputs

Why this test came first

Before adding feed pressure, rotation, or cavitation, we asked the smallest possible question: can OpenFOAM read this mesh, recognize all five patches, and preserve an exactly motionless solution?

Every velocity and pressure was zero, and the journal rotation was 0 rpm. That makes the exact answer obvious. If this case moved, the error would be in conversion, patch definitions, fields, or solver plumbing rather than in bearing physics.

Case: out/openfoam$_{\text{s100zeroflow}}$

Human-authored files

File	Purpose
0/U	Zero velocity field and velocity boundary conditions
0/p	Zero kinematic-gauge pressure field
constant/physicalProperties	Single-liquid kinematic viscosity
constant/momentumTransport	Laminar momentum model
system/controlDict	`incompressibleFluid` solver, run length, and writes
system/fvSchemes	Spatial and time discretization
system/fvSolution	Linear solvers and SIMPLE settings
system/functions	Flow, pressure, velocity, and load monitors
system/createPatchDict	Historical patch-normalization attempt; it did not change existing patch types
S100-zero-flow.foam	Empty ParaView case marker

The imported `constant/polyMesh/` files were generated by `gmshToFoam` and are inventoried separately in the artifact map.

Commands

After the import and mesh checks from the preceding chapter:

```
foam_root=/home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14
case_dir=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_zero_flow

bash -lc "
  source '$foam_root/etc/bashrc'
  foamRun -case '$case_dir' 2>&1 | tee '$case_dir/log.foamRun'
"
```

Result

The solver retained the exact stationary state:

```
maxMag(all) of U = 0
maxMag(all) of p = 0
continuity errors : sum local = 0, global = 0, cumulative = 0
SIMPLE solution converged in 1 iterations
```

Evidence:

- log.foamRun
- $RUN_{RESULTS}$.md
- time 1 fields
- postProcessing monitors

What this proves

- OpenFOAM can read all 294,912 Hex8 cells.
- The volume is one connected region.
- The five physical boundaries retain their names.
- The field files parse.
- The incompressible solver and function objects execute.
- No patch silently injects velocity or pressure in the zero state.

What this does not prove

- It does not validate a feed-pressure boundary condition.
- It does not validate journal rotation.
- It does not validate cavitation or film rupture.
- It does not prove that every high-aspect-ratio discretization remains accurate under a large pressure/velocity gradient.
- It is a plumbing proof, not a bearing-performance result.

That narrow interpretation is why the next case was created separately instead of overwriting this checkpoint.

The later zero-pressure-bias branch deliberately returned to this exact `0/U` and `constant/polyMesh` rather than using the 0.5 MPa time-285 field. Its cavitation-enabled 0-rpm gate and first 1-rpm result are documented in the hydrodynamic-ramp chapter.

Previous: mesh import | Index | Next: paper inputs

04 - Paper Inputs, Unit Conversions, and Assumptions

Source: `docs/handoff/04-paper-inputs.org`

Previous: zero-flow proof | Index | Next: single-phase results

Primary reference

The physical starting point was Table 1 on PDF page 9 of:

Performance Analysis of a Conical Hydrodynamic Journal Bearing

The paper uses a Newtonian, laminar, incompressible, isothermal lubricant. It sets both axial edges to atmospheric pressure and applies a Reynolds cavitation boundary in its reduced-order model.

Bibliographic identifier: DOI 10.1007/s40997-018-0217-2.

Operating-point taxonomy

Pressure supply and journal rotation are independent energy inputs. The following cases therefore answer different questions:

Rotation	Feed-to-end pressure difference	Classification
0 rpm	0	quiescent equilibrium/plumbing proof
0 rpm	0.5 MPa	stationary pressure-driven commissioning case
greater than 0	0	pure hydrodynamic bearing case
greater than 0	0.5 MPa	hybrid hydrostatic/hydrodynamic case

The accepted time-285 result belongs to the second row. It cannot initialize the third row without carrying a 0.5 MPa pressure-driven velocity field into an experiment that is supposed to have zero pressure bias.

The direct paper includes a supply location but does not specify a complete three-dimensional CFD reservoir/feed boundary treatment. Its pressure normalization and supply description are not sufficient to uniquely justify `601325 Pa` absolute at the mesh inlet. The 0.5 MPa boundary is therefore a declared commissioning/hybrid-case assumption, not a hidden definition of "hydrodynamic."

The new pure-hydrodynamic branch keeps the feed and both axial ends open at the same ambient pressure. Continuous makeup flow is allowed, but rotation is the only energy source. None of the inspected papers established a "fill once, then close the feed" procedure for this model.

Values used

Quantity	Paper value or declared convention	OpenFOAM value
Density	860 kg/m3	`rho = 860`
Dynamic viscosity	0.0277 Pa.s	converted to `nu`
Kinematic viscosity	derived	3.22093023256e-5 m2/s
Feed pressure	paper: 0.5 MPa, reference unspecified; project: 0.5 MPa gauge	581.395348837 m2/s2 kinematic gauge
Axial-end pressure	atmosphere	0 m2/s2 gauge
Mean radius	50 mm	mesh geometry
Radial clearance ratio	0.001	50 micrometres at 50 mm
Cone semi-angle	10 degrees	mesh geometry
Journal axis	eccentric	+Z through `(0,-0.000030,z)` m

The conversions are:

```
nu = mu/rho = 0.0277/860
   = 3.22093023256e-5 m2/s

p_feed,kinematic = 500000/860
                 = 581.395348837 m2/s2
```

In the single-phase incompressible solver, `p` is pressure divided by density, not pressure in pascals. This is why the feed boundary is 581.395 rather than 500,000.

Geometry mismatch retained explicitly

The paper's bearing length is 100 mm. The current mesh length is 60 mm. The radius, clearance, and 10 degree cone match the selected paper values, but the length does not. No hidden scaling was introduced to force a match. Therefore even a converged CFD result is a paper-grounded baseline, not an exact reproduction of the paper geometry.

Speed inconsistency in the paper

The paper reports both a relative surface velocity of 2.6 m/s and a speed of 2000 rpm. At 50 mm radius:

```
rpm = v/(2*pi*R) * 60
    = 2.6/(2*pi*0.05) * 60
    = 496.6 rpm

surface speed at 2000 rpm
    = 2*pi*0.05*(2000/60)
    = 10.472 m/s
```

Those are not the same operating point. The intended ladder therefore was `0, 200, 500, 1000, 2000 rpm`: 200 rpm as a gentle startup stage, about 500 rpm for 2.6 m/s, and 2000 rpm as the paper's separately stated validation point.

Only 0 and 200 rpm were attempted. The 200 rpm single-liquid state already became physically impossible, so higher speeds were deliberately not run with the same incomplete model.

Boundary and material assumptions

The following pressure choices describe the old pressure-driven baseline:

- `pressure_feed`: 0.5 MPa gauge pressure as a declared project convention.
- `axial_end_z0` and `axial_end_zL`: atmospheric pressure, or zero gauge.
- `journal_wall`: rotating no-slip wall about +Z and origin `(0,-3e-5,0)` m.
- `stationary_wall`: fixed no-slip wall.
- Flow is steady, laminar, incompressible, and isothermal.
- Gravity and hydrostatic head are excluded.
- The inlet is a pressure patch on the bore, not a feed-pipe flow domain.
- No turbulence, thermal viscosity variation, elastic deformation, feed-line loss, or dissolved-gas model was added.

For the zero-pressure-bias branch, all three openings instead use 101,325 Pa absolute. See the hydrodynamic-ramp chapter.

The physical pressure gate

At an atmospheric reference of 101,325 Pa, zero absolute pressure corresponds to the following kinematic gauge pressure:

```
p_zero-absolute = -101325/860
                = -117.819767442 m2/s2
```

Any computed `p` below that is not a low but valid liquid pressure; it implies negative absolute pressure. This gate, rather than solver exit code alone, was used to reject the rotating single-liquid run.

Assumption record

The case-local record is:

out/openfoam$_{s100paperbaseline}$/ASSUMPTIONS.md

Previous: zero-flow proof | Index | Next: single-phase results

05 - Paper-Grounded Single-Phase Run

Source: `docs/handoff/05-single-phase-results.org`

Previous: paper inputs | Index | Next: cavitation model

Case creation

The accepted zero-flow case was preserved. A separate case copied its mesh and dictionaries:

```
repo=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
source_case="$repo/out/openfoam_s100_zero_flow"
target_case="$repo/out/openfoam_s100_paper_baseline"

mkdir -p "$target_case/0" "$target_case/constant" "$target_case/system"
cp -a "$source_case/constant/polyMesh" "$target_case/constant/polyMesh"
cp -a "$source_case/0/U" "$source_case/0/p" "$target_case/0/"
cp -a "$source_case/constant/physicalProperties" \
      "$source_case/constant/momentumTransport" "$target_case/constant/"
cp -a "$source_case/system/controlDict" \
      "$source_case/system/fvSchemes" \
      "$source_case/system/fvSolution" \
      "$source_case/system/functions" "$target_case/system/"
cp -a "$source_case/S100-zero-flow.foam" \
      "$target_case/S100-paper-baseline.foam"
```

The copied text files were then edited to contain the values in the paper-input chapter. The current case files, not the copy command, are the canonical record.

File roles

File	Important choice
0/U	`rotatingWallVelocity` on journal; initially 0 rpm
0/p	Feed 581.395348837; both axial ends zero gauge

File	Important choice
constant/physicalProperties	`nu` = 3.22093023256e-5 m2/s
constant/momentumTransport	laminar
system/controlDict	`incompressibleFluid`, steady run, binary writes
system/fvSchemes	`steadyState`, upwind convection, corrected gradients
system/fvSolution	GAMG pressure, smooth velocity, SIMPLE residual controls
system/functions	flows, extrema, journal forces and moments at rho 860
system/decomposeParDict	four domains using Scotch

Pressure-only, 0 rpm initialization

Several serial tuning attempts preceded the accepted run. The first used one non-orthogonal corrector and pressure `relTol 0.01`; pressure took 334 then 1000 linear iterations and the run was stopped at time 4. Later trials used zero non-orthogonal correctors, `relTol 0.1`, a pressure `maxIter 100` cap, and velocity relaxation 0.7. Serial progress reached about time 106 but remained slow. `log.pressure-only` was overwritten during that tuning and ends mid-iteration, so it is diagnostic history, not the accepted evidence.

The final stage was run in four partitions:

```
foam_root=/home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14
case_dir=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_paper_baseline

bash -lc "
  source '$foam_root/etc/bashrc'
  decomposePar -case '$case_dir' -latestTime
  mpirun -np 4 foamRun -parallel -case '$case_dir' 2>&1 |
    tee '$case_dir/log.pressure-only-parallel'
  reconstructPar -case '$case_dir' -latestTime
"
```

In the managed runner, `mpirun --allow-run-as-root` was required because that runner identifies as root. Do not add that flag in a normal user terminal.

The first sandboxed MPI attempt also printed:

```
pmix_ifinit: socket() failed with errno=1
```

MPI needed permission to create local loopback sockets. Once allowed, the same four-rank command ran normally. This was an execution-sandbox problem, not a mesh or decomposition failure.

The solution converged at time/iteration 285:

```
SIMPLE solution converged in 285 iterations
```

Final state:

Quantity	Value
Maximum velocity	2.518357103601 m/s
Minimum kinematic gauge pressure	0.04072111336 m2/s2
Maximum kinematic gauge pressure	582.6261478525 m2/s2
Feed flow	-1.466299141522e-6 m3/s
z0 outlet flow	+7.722772418870e-7 m3/s
zL outlet flow	+6.940219014703e-7 m3/s
Relative flux imbalance	1.25165e-9

The negative feed sign means flow enters the domain. Positive axial flows leave it.

OpenFOAM integrates `U dot n` using each patch's **outward** normal. Inlet flow therefore has a negative sign; it does not mean that the result or pressure direction is wrong.

Zero journal speed does not imply zero fluid speed in this case because the feed is held 0.5 MPa above the axial ends. That pressure difference is an energy source. The logged feed area is `1.28571014247e-5 m2`, so the mean feed-normal speed is:

```
abs(Qfeed)/Afeed
= 1.466299141522e-6 / 1.28571014247e-5
= 0.1140458563 m/s
```

The reported `2.518357 m/s` is the maximum magnitude in any cell, not the mean speed through the feed. It is produced by the pressure-driven flow accelerating through the thin film even though the wall is stationary.

Useful commissioning quantities are:

Quantity	Value
Hydraulic resistance `Delta p/Q`	3.40994539137e11 Pa.s/m3
Conductance `Q/Delta p`	2.93259828304e-12 m3/(Pa.s)
Flow leaving through z0	52.6685%
Flow leaving through zL	47.3315%

Final journal loads:

```
pressure force  = ( 2.39734e-05, -431.426517, -95.091235 ) N
viscous force   = ( 1.2998e-09, -0.2897636, -0.0684192 ) N
pressure moment = ( -3.93130266, -2.409e-08, approximately 0 ) N.m
viscous moment  = ( -0.00300512, approximately 0, -4.878e-10 ) N.m
```

This result is numerically accepted and physically valid for a continuously pressure-fed stationary bearing. It is not accepted as a pure-hydrodynamic bearing result. The feed remained connected and pressurized throughout; it was never used merely to fill the mesh and then close.

The clean zero-bias experiment therefore does not restart from time 285. It is documented in the hydrodynamic-ramp chapter.

The 200 rpm diagnostic

Rotation was enabled from the converged field:

```
foamDictionary "$case_dir/285/U" \
  -entry boundaryField/journal_wall/omega -set '200 [rpm]'
decomposePar -case "$case_dir" -latestTime -force
mpirun -np 4 foamRun -parallel -case "$case_dir" 2>&1 |
  tee "$case_dir/log.rpm-200"
reconstructPar -case "$case_dir" -latestTime
```

At the first new update:

```
min(all) of p = -119.899239344
```

The physical zero-absolute-pressure floor is `-117.819767442 m2/s2`. Therefore the liquid-only solution became nonphysical immediately. The first update was already about -1,788 Pa absolute.

By diagnostic time 359:

```
GAMG ... No Iterations 100
min(all) of p = -1155.49896434
max(all) of p = 1158.55941024
maxMag(all) of U = 2.8577664739
```

Converted back to absolute pressure:

```
p_abs = 101325 + 860*(-1155.49896434)
      = approximately -892404 Pa
```

That is impossible. Pressure GAMG was also repeatedly reaching its 100-iteration cap, so the field was neither physically valid nor numerically converged. The run was manually interrupted during time 360; it has no normal `End` record, and the latest written/reconstructed field is time 340. The 500, 1000, and 2000 rpm stages were not attempted with the same single-liquid model.

At the retained diagnostic time, boundary flux was still nearly closed (relative imbalance 3.6669e-5). This and the passing mesh evidence show that mesh import remained coherent; they do not rescue the missing cavitation physics.

Relation to the earlier Fluent failure

The supplied Fluent S100 log first confirms that import completed with the same 322,816 nodes, 294,912 Hex8 cells, expected patch counts, positive minimum volume, and `Checking mesh... Done.`. It also contains material setup warnings:

```
Error: lubricant already exists.
delete-material: still in use by fluid
delete-material: still in use by air
```

Hybrid initialization did not meet its tolerance. The residual history then diverged catastrophically:

```
4 continuity 0.55689 ...
5 continuity 1 ...
6 continuity 21.591 ...
7 continuity 1.1320e+3 ...
201 continuity 5.9914e+150 ... monitors nan ...
Divergence detected in AMG solver ...
Error at Node ... floating point exception
Operation stopped.
```

See log-inlet-100.txt.

All 100 inlet faces were reporting reversed flow near failure. These values are numerical overflow, not forces or flow rates.

This is supporting evidence that the problem is not unique to the OpenFOAM executable. It is not proof that both solvers used identical boundary conditions or failed for one identical numerical reason. The text log does not establish the final material assignment, pressure units, rotation axis/origin/speed, turbulence choice, or initialization. A native Fluent `.cas.h5` or complete setup journal is needed for that audit.

Inspecting the result

Generate or refresh VTK:

```
foamToVTK -case "$case_dir" -time 285,340 \
  -fields '(p U)' -allPatches -useTimeName
```

Open the case:

```
paraview "$case_dir/S100-paper-baseline.foam"
```

Useful images:

- accepted 0 rpm pressure
- rejected 200 rpm pressure
- rejected 200 rpm velocity

Full result summary: $\mathrm{RUN}_{\mathrm{RESULTS}}$.md.

Previous: paper inputs | Index | Next: cavitation model

06 - Why and How the Cavitation Model Was Added

Source: `docs/handoff/06-cavitation-model.org`

Previous: single phase | Index | Next: cavitation results

Why cavitation became the next model

The 200 rpm liquid-only solution crossed the zero-absolute-pressure floor on its first update. Raising speed cannot repair that missing physics. The next controlled question was whether OpenFOAM's native cavitation machinery could represent a low-pressure phase while preserving the accepted S100 mesh and pressure-fed initialization. That describes the first historical experiment, not the later pure-hydrodynamic branch.

A separate case was created so that the accepted single-phase checkpoint remained untouched:

out/$\mathrm{openfoam}_{\mathrm{s100cavitationexploratory}}$

It copied the 0.5 MPa time-285 velocity, changed directly to 200 rpm, and activated cavitation at the same time. Those three simultaneous changes made it a poor seed for a zero-pressure-bias study. It is retained as failure evidence.

The replacement branch starts from the exact zero-flow field, sets feed and ends to equal ambient pressure, establishes a cavitation-enabled 0-rpm equilibrium, and only then advances speed: zero-pressure-bias hydrodynamic ramp.

Model selected

The case uses the OpenFOAM 14 native combination:

```
solver: incompressibleVoF
fvModel: VoFCavitation
mass-transfer model: SchnerrSauer
time treatment: localEuler pseudo-transient stepping
```

The written directories `1, 2, 3, ... 20` are pseudo-time counters used to seek a steady state. They are not physical seconds.

The implementation reference is OpenFOAM 14 VoFCavitation API. The local working example was cavitatingBullet. The exact local sources are SchnerrSauer.H, SchnerrSauer.C, and localEulerDdtScheme.H.

This model was chosen because it already exists in the installed solver and could answer a compatibility question without custom code. It was not chosen because the direct conical-bearing paper uniquely identifies it.

Reynolds boundary versus a VOF mass-transfer model

These two uses of the word cavitation are related but not interchangeable:

- The paper's Reynolds cavitation boundary is a thin-film rupture/free-boundary treatment. It prevents an unphysical tensile liquid pressure and supplies a rule for the ruptured region.
- Schnerr-Sauer VOF is a three-dimensional liquid/vapour mass-transfer model. It needs vapour properties, saturation pressure, surface tension, and bubble-nuclei parameters.

The direct paper does not provide those extra VOF inputs. Therefore this case is an exploratory model branch, not a literal reproduction of the paper's Reynolds condition.

Calling Schnerr-Sauer ”proper treatment” here means that phase change and phase transport replace impossible tensile liquid pressure. It does not mean that its present placeholder nuclei and vapour inputs are validated for the oil, nor that it is equivalent to a mass-conserving Reynolds/JFO rupture law.

A later, separate native Reynolds/JFO midsurface implementation is documented in the native JFO candidate chapter. It removes the tensile-pressure branch and conserves its discrete film content, but its severe cavity has not passed external numerical validation. It does not retroactively validate this rejected Schnerr–Sauer experiment.

Reused values

Quantity	Value	Source
Mesh	S100, 294,912 Hex8	accepted mesh
Initial velocity	converged time 285	accepted 0 rpm single-phase state
Journal speed	200 rpm	failed stage being investigated
Journal axis/origin	+Z, `(0 -3e-5 0)` m	geometry
Oil density	860 kg/m3	direct paper
Oil kinematic viscosity	3.22093023256e-5 m2/s	direct paper conversion
Feed pressure	601,325 Pa absolute	0.5 MPa gauge plus atmosphere
Axial ends	101,325 Pa absolute	atmosphere

Gravity is zero. Consequently `p_rgh` is the absolute static pressure here; there is no hydrostatic correction.

Exploratory values

Quantity	Value	Status
Saturation pressure `pSat`	0.5 Pa	screening value from `<0.5 Pa at 20 C`
Surface tension `sigma`	0.03 N/m	engineering placeholder
Vapour density	2.8e-5 kg/m3	engineering placeholder
Vapour dynamic viscosity	1e-5 Pa.s	engineering placeholder
Vapour kinematic viscosity	0.357142857 m2/s	derived placeholder
Nuclei density `n`	1.6e13 1/m3	OpenFOAM water tutorial value
Nucleus diameter `dNuc`	2e-6 m	OpenFOAM water tutorial value
Condensation coefficient `Cc`	1	OpenFOAM example

Quantity	Value	Status
Vaporization coefficient `Cv`	1	OpenFOAM example

The vapour-pressure source used for screening is the current Shell Tellus S2 MX 32 safety data sheet: Shell SDS. It reports vapour pressure below 0.5 Pa at 20 degrees C, density 854 kg/m3 at 15 degrees C, and kinematic viscosity 32 mm2/s at 40 degrees C. The direct paper's density and viscosity were retained; values were not mixed silently.

The closest local OpenFOAM cavitation paper is ilt-01-2024-0031.pdf. It uses a different textured cylindrical bearing, a different oil, and a homogeneous-equilibrium treatment. Its `pSat = 29185 Pa`, liquid density 822 kg/m3, vapour density 0.14 kg/m3, and vapour viscosity 5.95e-6 Pa.s are kept only as a future sensitivity set (DOI 10.1108/ILT-01-2024-0031).

Another local paper, ilt-04-2015-0055.pdf, uses a different ADINA pressure-window model and different phase properties. It also was not mixed into the present case (DOI 10.1108/ILT-04-2015-0055).

1-s2.0-S0301679X08000546-main.pdf notes that a fuller cavitation treatment additionally needs vaporization pressure, surface tension, and non-condensable-gas mass fraction. It was used to identify missing inputs, not as a source of silently transferred values (DOI 10.1016/j.triboint.2008.03.002).

OpenFOAM defines the Schnerr-Sauer nuclei fraction from these tutorial parameters as:

```
Vnuc      = n*pi*dNuc^3/6
alphaNuc = Vnuc/(1 + Vnuc)
```

For the selected `n` and `dNuc`, `alphaNuc` is about 6.70e-5. This is not a measured value for the present oil.

Case construction

The mesh and accepted velocity checkpoint were copied without modifying the single-phase case:

```
repo=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
case_dir="$repo/out/openfoam_s100_cavitation_exploratory"
baseline="$repo/out/openfoam_s100_paper_baseline"

mkdir -p "$case_dir/0" "$case_dir/constant" "$case_dir/system"
cp -a "$baseline/constant/polyMesh" "$case_dir/constant/polyMesh"
cp -a "$baseline/285/U" "$case_dir/0/U"
touch "$case_dir/S100-cavitation-exploratory.foam"

foamDictionary "$case_dir/0/U" -entry FoamFile/location -set '"0"'
foamDictionary "$case_dir/0/U" \
  -entry boundaryField/journal_wall/omega -set '200 [rpm]'
```

The remaining small dictionaries listed below were authored for the VOF case.

Files created

1. Initial fields

File	Role
0/U	Baseline velocity with the journal set to 200 rpm
0/alpha.oil	Oil fraction; initially one everywhere
0/p_{rgh}	Absolute pressure field used by the solver
0/p	Copied single-phase kinematic pressure; mapping input
0/pAtm	Uniform 101,325 Pa mapping input
0/pGaugePa	Generated `860*p` intermediate

2. Physical/model dictionaries

File	Role
constant/g	zero gravity
constant/phaseProperties	phases `(oil vapour)` and `sigma`
constant/physicalProperties.oil	oil `rho` and `nu`
constant/physicalProperties.vapour	exploratory vapour `rho` and `nu`

File	Role
constant/momentumTransport	laminar phase-mixture transport
constant/fvModels	`VoFCavitation/SchnerrSauer` inputs

3. Numerical and monitoring dictionaries

File	Role
system/controlDict	`incompressibleVoF` and pseudo-time range
system/fvSchemes	VOF, momentum, and pressure discretization
system/fvSolution	phase, pressure, velocity solvers and coupling
system/functions	phase, pressure, flow, velocity, and load monitors
system/mapPressureFunctions	native `p` → `pGaugePa` → `p_rgh` mapping

Why pressure was not clipped

A `limitPressure` constraint was considered and rejected:

- the single-phase source field used kinematic pressure while the VOF solver uses dimensional pressure;
- clipping would hide the missing phase/rupture physics;
- an arbitrary floor can damage mass conservation and loads;
- solver survival is not evidence of a correct cavitation field.

The safer experiment was to initialize dimensional absolute pressure correctly, run briefly, and judge phase, pressure, loads, and boundary flux together.

The case-local assumptions are recorded in ASSUMPTIONS.md.

Previous: single phase | Index | Next: cavitation results

07 - Cavitation Experiments and What Is Interesting

Source: `docs/handoff/07-cavitation-results.org`

Previous: cavitation model | Index | Next: replay commands

The experiments in this chapter remain rejected or low-speed liquid-branch evidence. The later converged Reynolds/JFO field is a different reduced model and remains an unvalidated candidate; see Chapter 14.

Experiment 1: uniform atmospheric pressure

The first smoke test paired the already moving baseline velocity with a uniform `p_rgh = 101325 Pa` internal field. The boundary feed was 601,325 Pa. This was a poor initial condition: it threw away the accepted pressure-fed single-phase pressure distribution while keeping its velocity.

The case passed the standard mesh check, and `foamRun` exited normally, but the solution immediately made a huge pressure correction:

Quantity	Time 1	Time 3
Minimum `p_rgh`	-3,639,880.5 Pa	-1,564,003.1 Pa
Minimum `alpha.oil`	not used for acceptance	0.0079086
Mean `alpha.oil`	not used for acceptance	0.556070
Maximum velocity	not used for acceptance	2.77299 m/s
Cumulative continuity	drifting	-4.37297
Relative boundary-flow imbalance	drifting	161.9%

Relevant log: log.smoke.uniformPressure.

This run proved that the dictionaries parsed and the model executed. It did not produce a useful physical field. Its original state was moved into uniformPressureSmoke rather than discarded.

Historical improvement: map the pressure-fed field

The accepted single-phase time-285 field stores kinematic gauge pressure. OpenFOAM function objects converted it without an external script:

```
pGaugePa = 860*p
p_rgh    = 101325 Pa + pGaugePa
```

Several small parser/setup problems were found before the working command:

- using a case-prefixed path after `-dict` and also `-case` duplicated the path; the correct argument is `-dict system/mapPressureFunctions`;
- an extra top-level `functions {}` wrapper made `type` undefined; the function objects must be at the mapping dictionary's top level;
- including the ordinary runtime monitors during mapping asked for unavailable `phi`, `U`, and `p` fields; the solution was a pressure-only mapping dictionary;
- the min/max objects needed `writeFields false` explicitly.

Command:

```
foamPostProcess \
  -case /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory \
  -dict system/mapPressureFunctions \
  -time 0 \
  -fields '(p pAtm)' 2>&1 |
  tee /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory/log.mapP
```

The mapped internal range was:

```
minimum p_rgh = 101360.020157 Pa
maximum p_rgh = 602383.487153 Pa
```

The mapping function objects generate calculated patch entries, so the intended pressure patch types and values were restored:

```
field=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory/0/p_rgh

foamDictionary "$field" -entry boundaryField/journal_wall/type \
  -set fixedFluxPressure
foamDictionary "$field" -entry boundaryField/stationary_wall/type \
  -set fixedFluxPressure
foamDictionary "$field" -entry boundaryField/axial_end_z0/type \
  -set fixedValue
foamDictionary "$field" -entry boundaryField/axial_end_z0/value \
  -set 'uniform 101325'
foamDictionary "$field" -entry boundaryField/axial_end_zL/type \
  -set fixedValue
foamDictionary "$field" -entry boundaryField/axial_end_zL/value \
  -set 'uniform 101325'
foamDictionary "$field" -entry boundaryField/pressure_feed/type \
  -set fixedValue
foamDictionary "$field" -entry boundaryField/pressure_feed/value \
  -set 'uniform 601325'
```

Experiment 2: mapped-pressure smoke test

The three-step mapped run completed with exit code zero:

```
foamRun -case /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory
  2>&1 | tee /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory/l
```

Comparison at time 3:

Quantity	Uniform start	Mapped start
Minimum `p_rgh` at time 1	-3,639,880 Pa	-437.634 Pa
Minimum `p_rgh` at time 3	-1,564,003 Pa	-7,067.872 Pa
Cumulative continuity	-4.37297	+0.00746233

Quantity	Uniform start	Mapped start
Final local continuity sum	6,508.93	3,827.97
Minimum `alpha.oil`	0.00790864	0.324920
Mean `alpha.oil`	0.556070	0.950734
Maximum velocity	2.77299 m/s	2.54462 m/s
Net boundary volume flow	-2.63737e-6 m3/s	+7.39824e-9 m3/s
Relative flow imbalance	161.9%	0.505%

The pressure undershoot became about 221 times smaller, and cumulative continuity magnitude became about 586 times smaller. Correct initialization was therefore a major numerical solution.

It was not the whole solution. Pressure remained below `pSat`, local continuity was large, and the phase field was moving. The time-3 state is a better starting point, not a converged cavitation result.

Again, `time 3` means the third pseudo-time step, not three physical seconds.

Experiment 3: direct continuation to pseudo-time 20

The mapped state was continued without a speed or pressure ramp:

```
foamRun -case /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory
  2>&1 | tee /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory/l
```

It exited normally, but mass balance became much worse:

Quantity	Time 3	Time 20
Minimum `p_rgh`	-7,067.872 Pa	-3,443.254 Pa
Minimum `alpha.oil`	0.324920	0.0557785
Mean `alpha.oil`	0.950734	0.913002
Maximum velocity	2.54462 m/s	2.78885 m/s
Relative flow imbalance	0.505%	43.898%

At time 20:

```
feed flow = -1.5071483e-6 m3/s
z0 flow   = +4.5453570e-7 m3/s
zL flow   = +3.9101181e-7 m3/s
```

The missing outlet flow is not a small rounding error. Phase fraction, velocity, loads, and flow balance were still drifting. The continuation was stopped and rejected as a physical result.

What is scientifically interesting

1. The failure reveals a physical regime

   The liquid-only run did not merely have a noisy residual. It demanded negative absolute pressure as soon as rotation began. That is evidence that the diverging part of this eccentric film wants to rupture or admit another phase. The failure is therefore a useful model-selection signal.

2. Oil "cavitation" may not be pure oil vapour

   Shell's data puts pure-oil vapour pressure below 0.5 Pa at 20 degrees C. Journal bearings can form cavities at much higher observed pressures because dissolved or entrained air comes out of solution, ambient air enters from an edge, or the liquid film ruptures. A pure oil-vapour Schnerr-Sauer model may be solving the wrong second phase even if its numerics are stabilized.

   This makes a gas-release or rupture model scientifically interesting:

   - pure vapour cavitation should occur only near vacuum for this oil;
   - gaseous cavitation depends on dissolved-gas content and pressure history;
   - edge air ingestion depends on the axial boundary and cavity transport;
   - the paper's Reynolds boundary represents film rupture without resolving bubble nucleation.

3. Initialization matters, but cannot replace convergence

Mapping the accepted pressure field improved continuity by roughly 586 times. The later 43.898% imbalance shows that a good initial condition cannot rescue an unstaged or inappropriate physical model. Exit code zero only means that the executable completed its requested steps.

4. The picture is a failure-mode picture

   S100-cavitation-smoke-alpha-oil.png shows patchy circumferential and axial oil-fraction bands at time 20. Its title explicitly says `NOT CONVERGED`. It is useful for seeing where the current model is changing phase; it must not be presented as a stable cavity shape.

Why time 285 was not reused for pure hydrodynamics

Mapping time 285 was the right numerical comparison inside the old pressure-fed experiment: it avoided pairing a developed velocity field with an unrelated uniform pressure field. It is the wrong physical seed for a zero-pressure-bias experiment because it already contains flow driven by a 0.5 MPa feed-to-end pressure difference.

The replacement branch therefore returned to `openfoam_s100_zero_flow/0/U`, created a uniform ambient absolute-pressure field, activated cavitation at 0 rpm, and only then moved to 1 rpm. See the hydrodynamic-ramp results.

ParaView export

```
foamToVTK -case /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_explorator
  -time 3
foamToVTK -case /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_explorator
  -time 20
paraview /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory/S100-
```

Full local report: SMOKE$_{\text{TESTRESULTS}}$.md.

Previous: cavitation model | Index | Next: replay commands

08 - Commands to Rebuild, Replay, and Inspect

Source: `docs/handoff/08-replay-commands.org`

Previous: cavitation results | Index | Next: artifacts

Important replay warning

The commands below target the existing case paths. Running a solver from `latestTime` can add or replace later time directories and logs. Preserve a checkpoint before experimenting:

```
repo=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
cp -a "$repo/out/openfoam_s100_paper_baseline" \
      "$repo/out/openfoam_s100_paper_baseline_replay"
cp -a "$repo/out/openfoam_s100_cavitation_exploratory" \
      "$repo/out/openfoam_s100_cavitation_exploratory_replay"
```

The time-285 mapping and direct 200-rpm cavitation commands later in this file reproduce the **historical rejected pressure-fed branch**. They are not the recommended pure-hydrodynamic path. The clean zero-flow construction and accepted 0/1/5-rpm replay and continuation commands are in the zero-pressure-bias chapter.

Use the copied paths for trial runs. The commands in the historical sections below use the original names to match the retained logs.

Build OpenFOAM 14

For the same source revisions:

```
workspace=/home/aniketnegi/code/ankt/btp
mkdir -p "$workspace/.openfoam"
cd "$workspace/.openfoam"

git clone https://github.com/OpenFOAM/OpenFOAM-14.git
git -C OpenFOAM-14 checkout 7b05503f98a85be88af930df48623b4d152bfc35

git clone https://github.com/OpenFOAM/ThirdParty-14.git
git -C ThirdParty-14 checkout 6f75cb6aa1a3d8f1ba768840f9ee535244b4c70a
```

```
mkdir -p site/14/etc
printf '%s\n' 'export ParaView_TYPE=none' > site/14/etc/prefs.sh

cd OpenFOAM-14
source etc/bashrc
./Allwmake -q -j 6
```

The site preference disables only OpenFOAM's optional ParaView integration. The system ParaView application and `foamToVTK` remain available.

For zsh, use a Bash login command:

```
bash -lc '
  source /home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14/etc/bashrc
  foamVersion
'
```

To keep a future build log:

```
cd /home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14
bash -o pipefail -c '
  source etc/bashrc
  ./Allwmake -q -j 6 2>&1 |
    tee ../OpenFOAM-14.Allwmake.log
'
```

No standalone log was saved for the original full build; its exact commands and output remain in the Codex session transcript.

Regenerate the S100 mesh

```
cd /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
UV_CACHE_DIR=.uv-cache uv run python meshing/structured_hex_surface_inlet.py \
  --params out/strict_default/params.json \
  --outdir out/professor_surface_inlet_variants/inlet100_moderate_ngap12 \
  --n-theta 256 --n-axial 96 --gap-levels 12 --preview-ngap 12 \
  --gap-inflation-ratio 1 --inlet-cluster-strength 0.60 \
  --max-projected-area-relative-error 0.01 \
  --max-inlet-rim-error-mm 0.23 \
  --openfoam skip --no-gui
```

Expected mesh SHA-256:

```
3dcd8dcb8c49a31ded67f69a793d64ed608e1fd20920a0a394b780dc7de6058b
```

Reimport S100 and run the zero-flow proof

This assumes the case dictionaries in `out/openfoam_s100_zero_flow` have been preserved. `out/` is ignored and is not regenerated by Git checkout alone.

```
foam_root=/home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14
repo=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
case_dir="$repo/out/openfoam_s100_zero_flow"
mesh_file="$repo/out/professor_surface_inlet_variants/inlet100_moderate_ngap12/nGap_12/structured_hex_openfoam.msh"

bash -lc "
  source '$foam_root/etc/bashrc'
  gmshToFoam -case '$case_dir' '$mesh_file' 2>&1 |
    tee '$case_dir/log.gmshToFoam'
"
```

After a fresh import, change only `journal_wall` and `stationary_wall` in `constant/polyMesh/boundary` from `patch` to:

```
type            wall;
physicalType    wall;
```

Preserve every `nFaces` and `startFace` value. Perform this before `checkMesh` or `foamRun`. The known-good result is linked in the mesh-import chapter.

Then:

```
bash -lc "
  source '$foam_root/etc/bashrc'
  checkMesh -case '$case_dir' 2>&1 |
    tee '$case_dir/log.checkMesh.standard'
  checkMesh -case '$case_dir' -allTopology -allGeometry 2>&1 |
    tee '$case_dir/log.checkMesh'
  foamRun -case '$case_dir' 2>&1 |
    tee '$case_dir/log.foamRun'
"
```

Expected zero-flow ending:

```
maxMag(all) of U = 0
maxMag(all) of p = 0
SIMPLE solution converged in 1 iterations
```

Replay the accepted 0 rpm pressure-fed state

On a copied case, remove no evidence from the original. If the case already contains reconstructed time 285, it can be inspected without rerunning.

```
foam_root=/home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14
case_dir=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_paper_baseline_replay

bash -lc "
  source '$foam_root/etc/bashrc'
  checkMesh -case '$case_dir'
  decomposePar -case '$case_dir' -latestTime -force
  mpirun -np 4 foamRun -parallel -case '$case_dir' 2>&1 |
    tee '$case_dir/log.pressure-only-replay'
  reconstructPar -case '$case_dir' -latestTime
"
```

If running inside a container or managed environment that identifies as root, add `--allow-run-as-root` after `mpirun`. Do not use it unnecessarily in the normal user shell.

Replay the rejected 200 rpm screen

Starting from the converged time-285 checkpoint:

```
foamDictionary "$case_dir/285/U" \
  -entry boundaryField/journal_wall/omega -set '200 [rpm]'
decomposePar -case "$case_dir" -latestTime -force
mpirun -np 4 foamRun -parallel -case "$case_dir" 2>&1 |
  tee "$case_dir/log.rpm-200-replay"
reconstructPar -case "$case_dir" -latestTime
```

Stop and reject the stage when `min(p)` crosses `-117.819767442 m2/s2`, the zero-absolute-pressure gate for density 860 kg/m3. Do not continue to higher speed merely to obtain a final iteration number.

Recreate the mapped cavitation initial pressure

Use a copy of the cavitation case. It must contain:

- `0/p` copied from the accepted single-phase `285/p`;
- `0/pAtm` equal to 101,325 Pa;
- `system/mapPressureFunctions` from the current exploratory case.

The original branch began by copying the accepted mesh and velocity:

```
repo=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
baseline="$repo/out/openfoam_s100_paper_baseline"
template="$repo/out/openfoam_s100_cavitation_exploratory"
```

```
cav_case="$repo/out/openfoam_s100_cavitation_exploratory_replay"

mkdir -p "$cav_case/0" "$cav_case/constant" "$cav_case/system"
cp -a "$baseline/constant/polyMesh" "$cav_case/constant/polyMesh"
cp -a "$baseline/285/U" "$cav_case/0/U"
cp -a "$baseline/285/p" "$cav_case/0/p"
cp -a "$template/0/alpha.oil" "$template/0/pAtm" "$cav_case/0/"
cp -a "$template/constant/g" \
      "$template/constant/phaseProperties" \
      "$template/constant/physicalProperties.oil" \
      "$template/constant/physicalProperties.vapour" \
      "$template/constant/momentumTransport" \
      "$template/constant/fvModels" "$cav_case/constant/"
cp -a "$template/system/controlDict" \
      "$template/system/fvSchemes" \
      "$template/system/fvSolution" \
      "$template/system/functions" \
      "$template/system/mapPressureFunctions" "$cav_case/system/"
touch "$cav_case/S100-cavitation-exploratory.foam"

foamDictionary "$cav_case/0/U" -entry FoamFile/location -set '"0"'
foamDictionary "$cav_case/0/U" \
  -entry boundaryField/journal_wall/omega -set '200 [rpm]'
foamDictionary "$cav_case/0/p" -entry FoamFile/location -set '"0"'
```

Copy the authored VOF dictionaries and `0/alpha.oil`, `0/pAtm` from the retained exploratory case. Then map pressure:

```
foam_root=/home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14
cav_case=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory_repla

bash -lc "
  source '$foam_root/etc/bashrc'
  foamPostProcess -case '$cav_case' \
    -dict system/mapPressureFunctions -time 0 \
    -fields '(p pAtm)' 2>&1 | tee '$cav_case/log.mapPressure'

  field='$cav_case/0/p_rgh'
  foamDictionary \"\$field\" -entry boundaryField/journal_wall/type \
    -set fixedFluxPressure
  foamDictionary \"\$field\" -entry boundaryField/stationary_wall/type \
    -set fixedFluxPressure
  foamDictionary \"\$field\" -entry boundaryField/axial_end_z0/type \
    -set fixedValue
  foamDictionary \"\$field\" -entry boundaryField/axial_end_z0/value \
    -set 'uniform 101325'
  foamDictionary \"\$field\" -entry boundaryField/axial_end_zL/type \
    -set fixedValue
  foamDictionary \"\$field\" -entry boundaryField/axial_end_zL/value \
    -set 'uniform 101325'
  foamDictionary \"\$field\" -entry boundaryField/pressure_feed/type \
    -set fixedValue
  foamDictionary \"\$field\" -entry boundaryField/pressure_feed/value \
    -set 'uniform 601325'
"
```

Expected mapped internal pressure range:

```
101360.020157 to 602383.487153 Pa
```

Run the cavitation smoke test

Set `endTime 3` for the compatibility test:

```
foamDictionary "$cav_case/system/controlDict" -entry endTime -set 3

bash -lc "
  source '$foam_root/etc/bashrc'
  checkMesh -case '$cav_case'
  foamRun -case '$cav_case' 2>&1 |
    tee '$cav_case/log.smoke.mappedPressure'
  foamToVTK -case '$cav_case' -time 3 2>&1 |
    tee '$cav_case/log.foamToVTK.time3'
"
```

The retained experiment continued directly to 20 and failed the mass-balance gate. A new physical study should not repeat that exact continuation as its main path; use the ramp in the zero-pressure-bias chapter. To reproduce the rejected diagnostic exactly:

```
foamDictionary "$cav_case/system/controlDict" -entry startFrom -set latestTime
foamDictionary "$cav_case/system/controlDict" -entry endTime -set 20
foamDictionary "$cav_case/system/controlDict" -entry writeInterval -set 5

bash -lc "
  source '$foam_root/etc/bashrc'
  foamRun -case '$cav_case' 2>&1 |
    tee '$cav_case/log.continue.to20'
  foamToVTK -case '$cav_case' -time 20 2>&1 |
    tee '$cav_case/log.foamToVTK.time20'
"
```

These are `localEuler` pseudo-time counters, not physical seconds.

Open results

```
paraview /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_paper_baseline/S100-paper-ba
```

```
paraview /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_cavitation_exploratory/S100-
```

```
paraview /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_pure_hydrodynamic/S100-pure-
```

```
gmsh /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/professor_surface_inlet_variants/inlet100_mode
```

ParaView can also open the explicit volume files under each case's `VTK/` directory. Gmsh is best for inspecting the original named mesh; ParaView is best for solution fields and cutaways. FreeCAD can inspect the separate context STEP geometry, but it does not carry the OpenFOAM volume fields or physical patch data.

Previous: cavitation results | Index | Next: artifacts

09 - File and Artifact Map

Source: `docs/handoff/09-artifact-map.org`

Previous: replay commands | Index | Next: problems and next steps

Why this inventory is grouped

Every human-authored dictionary, report, log, and visualization is named below. Generated mesh primitives and time fields are listed by family because `polyMesh`, `processor0` through `processor3`, `postProcessing`, and `VTK` contain many mechanical outputs with the same role.

Source-build files

Path	Role
.openfoam/OpenFOAM-14	solver and utilities source/build tree
.openfoam/ThirdParty-14	Scotch, PT-Scotch, Zoltan, and third-party build tree
OpenFOAM-14/.build	exact local build label
OpenFOAM-14/etc/bashrc	environment activation

Path	Role
.openfoam/site/14/etc/prefs.sh	disables unneeded OpenFOAM ParaView integration

These are local build products, not files added to the bearing repository.

Canonical S100 mesh bundle

Directory: $\mathrm{inlet100}_{\mathrm{moderatengap12}}/\mathrm{nGap}_{12}$

The parent directory also contains `run_report.json`, the generator-level PASS record.

File	Role
`structured_hex_openfoam.msh`	Gmsh 2.2 ASCII import source
`structured_hex.msh`	Gmsh 4.1 binary mesh
`mesh_arrays.npz`	canonical nodes, Hex8, boundaries, and fields
`manifest.json`	geometry, provenance, status, and artifact hashes
`mesh_report.json`	topology, geometry, and quality validation
`physical_groups.json`	physical IDs and names
`volume.vtu` and boundary VTU files	solver-neutral visualization
`*.cgns` if present	ANSYS/Fluent interchange
`*.png`	mesh, inlet, footprint, and quality views

The exact filenames present in this bundle remain the authoritative inventory; the table describes their roles rather than inventing missing formats.

Zero-flow case

Directory: $\mathrm{openfoam}_{\mathrm{s100zeroflow}}$

1. Authored inputs
   - `0/U`
   - `0/p`
   - `constant/physicalProperties`
   - `constant/momentumTransport`
   - `system/controlDict`
   - `system/fvSchemes`
   - `system/fvSolution`
   - `system/functions`
   - `system/createPatchDict` (historical attempt; it did not change types)
   - `S100-zero-flow.foam`
2. Generated mesh

   `constant/polyMesh/` contains:

   - `points` – imported point coordinates;
   - `faces` – face node connectivity;
   - `owner` and `neighbour` – cell ownership/connectivity;
   - `boundary` – five patch ranges and types;
   - `cellZones`, `faceZones`, and `pointZones` when generated by conversion;
   - `sets/` and other diagnostic subsets when created by `checkMesh`.
3. Logs and reports
   - `log.gmshToFoam`
   - `log.createPatch`
   - `log.checkMesh.standard`
   - `log.checkMesh`
   - `log.foamRun`
   - `RUN_RESULTS.md`
4. Generated results

- `1/U`, `1/p`, `1/phi` and uniform metadata;
- `postProcessing/` monitor tables;
- `S100-zero-flow.png`;
- possible `VTK/` conversion output.

Paper-grounded single-phase case

Directory: $\text{openfoam}_{\text{s100paperbaseline}}$

1. Authored inputs and notes
   - `0/U` and `0/p`
   - `constant/physicalProperties`
   - `constant/momentumTransport`
   - `system/controlDict`
   - `system/fvSchemes`
   - `system/fvSolution`
   - `system/functions`
   - `system/decomposeParDict`
   - `ASSUMPTIONS.md`
   - `RUN_RESULTS.md`
   - `S100-paper-baseline.foam`
2. Logs
   - `log.checkMesh`
   - `log.pressure-only` – early serial attempt/history;
   - `log.pressure-only-parallel` – accepted 0 rpm run;
   - `log.rpm-200` – rejected rotating diagnostic.
3. Generated result families
   - `80/` and `100/` – interrupted serial-tuning history;
   - `285/` – reconstructed accepted 0 rpm checkpoint;
   - `340/` – latest written/reconstructed rejected rotating diagnostic;
   - `processor0/` through `processor3/` – decomposed fields during parallel runs;
   - `postProcessing/` – flows, pressure/velocity extrema, forces, and moments;
   - `VTK/` – exported volume and patch fields.
4. Images
   - `S100-0rpm-pressure.png` – accepted initialization;
   - `S100-200rpm-rejected-pressure.png` – rejected diagnostic;
   - `S100-200rpm-rejected-velocity.png` – rejected diagnostic.

Exploratory cavitation case

Directory: $\text{openfoam}_{\text{s100cavitationexploratory}}$

1. Authored and mapped fields
   - `0/U`
   - `0/alpha.oil`
   - `0/p_rgh`
   - `0/p`, `0/pAtm`, and `0/pGaugePa` used for mapping.
2. Authored dictionaries
   - `constant/g`
   - `constant/phaseProperties`
   - `constant/physicalProperties.oil`
   - `constant/physicalProperties.vapour`
   - `constant/momentumTransport`
   - `constant/fvModels`
   - `system/controlDict`
   - `system/fvSchemes`
   - `system/fvSolution`
   - `system/functions`

- `system/mapPressureFunctions`

3. Notes and marker
   - `ASSUMPTIONS.md`
   - `SMOKE_TEST_RESULTS.md`
   - `SMOKE_TEST_RESULTS_UNIFORM_PRESSURE.md`
   - `S100-cavitation-exploratory.foam`

4. Logs
   - `log.checkMesh.standard`
   - `log.checkMesh`
   - `log.mapPressure`
   - `log.smoke.uniformPressure`
   - `log.smoke.mappedPressure`
   - `log.continue.to20`
   - `log.foamToVTK.time3`
   - `log.foamToVTK.time20`

5. Generated result families
   - `1/`, `2/`, and `3/` – mapped smoke-test fields;
   - `5/`, `10/`, `15/`, and `20/` – rejected continuation;
   - `postProcessing/` – pressure, alpha, flows, velocities, and loads;
   - `uniformPressureSmoke/` – preserved first failed initialization;
   - `VTK/` – time-3/time-20 volumes and five patch exports.

6. Image
   - `S100-cavitation-smoke-alpha-oil.png` – explicitly marked `NOT CONVERGED`.

Zero-pressure-bias hydrodynamic case

Directory: $\text{openfoam}_{\text{s100purehydrodynamic}}$

1. Clean inputs and notes
   - `0/U` – exact zero-flow velocity, 0 rpm;
   - `0/p_rgh` – 101,325 Pa absolute everywhere;
   - `0/alpha.oil` – oil fraction one with reversible opening conditions;
   - `constant/polyMesh/` – byte-identical zero-flow mesh;
   - `constant/fvModels`, phase properties, and transport dictionaries;
   - `system/controlDict`, `fvSchemes`, `fvSolution`, and `functions`;
   - `ASSUMPTIONS.md` and `RUN_RESULTS.md`;
   - `S100-pure-hydrodynamic.foam`.

2. Accepted checkpoints
   - `checkpoints/0rpm/3/` – cavitation-enabled quiescent equilibrium;
   - `checkpoints/1rpm/75/` – first accepted zero-bias hydrodynamic field;
   - `checkpoints/5rpm/150/` – accepted second continuation field;
   - `checkpoints/10rpm/225/`, `checkpoints/12rpm/300/`, and `checkpoints/14rpm/350/` – accepted liquid continuation fields;
   - `checkpoints/15rpm/400/` – accepted 15-rpm constant-speed field.

3. Rejected and controlled diagnostics

   `diagnostics/direct-20rpm-from-zero/` preserves the abrupt 20-rpm jump, including its starting field, times 4 through 8, post-processing tables, and log. It is retained as failure evidence and is not on the accepted ramp.

   `diagnostics/15rpm-from-10rpm-unstable/` preserves the rejected direct 10-to-15-rpm restart. The following sibling cases isolate its cause:

   - `direct-10to15-damped-case/` – lower Courant and pseudo-time limits;
   - `direct-10to15-fresh-start-flux-corrected-case/` – startup `correctPhi` path forced by renumbering the seed as time zero;
   - `direct-10to15-cavitation-inactive-case/` – Schnerr-Sauer still loaded, but `pSat-1e9` Pa= prevents phase change;
   - `direct-10to15-no-cavitation-case/` – secondary model-removal control;

- `direct-10to15-native-omega-ramp-case/` – actual cavitation model with OpenFOAM's native continuous wall-speed function and a 15-rpm hold.
- `guarded_speed_ramp.py` – standard-library driver that seeds an isolated case from `checkpoints/15rpm/400`, applies native continuous ramps, evaluates the existing monitors, and stops at saturation or loss of numerical health.
- `guarded-ramp-to-2000/` – default generated work case when the guarded driver is run; its `ramp-state.json` and `ramp-results.csv` are the campaign status and study table.

The root time `425` and `log.ramp-16rpm-400-425` are an unaccepted diagnostic. There is deliberately no `checkpoints/16rpm/` directory.

4. Logs and visualization

- `log.checkMesh.standard`;
- `log.equilibrate-0rpm`;
- `log.ramp-1rpm*` – successive holds at the same 1-rpm stage;
- `log.ramp-5rpm-75-100`, `log.ramp-5rpm-100-125`, and `log.ramp-5rpm-125-150` – successive holds at the same 5-rpm stage;
- `log.ramp-10rpm*`, `log.ramp-12rpm*`, `log.ramp-14rpm*`, and `log.ramp-15rpm-from-14*` – accepted continuation and settling logs;
- `log.foamToVTK.zero-and-1rpm` and `log.foamToVTK.5rpm`;
- `postProcessing/` – opening flows, pressure/velocity/phase extrema, and gauge loads;
- `VTK/` – explicit time-0, time-3, time-75, and time-150 volume/combined-patch files.

Earlier Fluent evidence

- out/cfd/log-inlet-100.txt – S100 residual history ending in AMG divergence and floating-point exceptions.
- The user-supplied screenshots show force, mass-flow, and residual plots

exploding for S100 and S390. They are conversation evidence; unless copied into the repository, their temporary clipboard paths are not durable project artifacts.

Native Reynolds/JFO candidate

1. Versioned source

- `openfoam/jfoBearingFoam/` – native OpenFOAM application and build files;
- `openfoam/cases/jfoPaperExact/` – 256 by 80 source case;
- `openfoam/check_jfoBearingFoam.sh` – build and zero-speed acceptance check;
- `openfoam/visualize_jfo.py` – checkpoint parser, plots, CSV, MP4, and GIF;
- `reynolds_jfo.py` – separate Python implementation of the same model;
- `tests/test_reynolds_jfo.py` – current minimal regressions;
- `docs/handoff/media/jfo_candidate/` – tracked current-candidate figures, animations, and checkpoint metrics;
- native JFO candidate handoff – result boundary and severe-cavity warning.

2. Ignored generated result

`out/openfoam_jfo_native_256x80/` contains the continuation fields, final `RUN_RESULTS.md`, and VTK export. Its final stored OpenFOAM time is `17.261090445375`. The candidate result remains unvalidated even though the case is converged and globally conservative.

This handoff

Files `00-index.org` through `14-native-jfo-candidate.org` are the versioned explanatory handoff. Large generated cases remain ignored under `out/`.

Previous: replay commands | Index | Next: problems and next steps

10 - Problem Ledger, Decisions, and Next Experiment

Source: `docs/handoff/10-problems-and-next.org`

Previous: artifacts | Index | Next: zero-pressure-bias ramp

Problem and solution ledger

Problem or observation	Evidence	Response	Present status
No OpenFOAM package in configured Arch repositories	`pacman -Ss openfoam` returned nothing	Tried AUR, then used official source route	solved
AUR Zoltan clone failed with OpenSSL EOF	quoted in build chapter	Retried once	transient failure passed
AUR ParMETIS CMake invocation malformed	unknown warning category `l,-z,...`	Stopped repairing packaging chain; built OpenFOAM 14 source	solved
`Allwmake` said environment not set	exact build log quote	`source etc/bashrc` before build	solved
zsh reports unmatched ParaView CMake glob	environment activation	execute commands through `bash -lc`	solved for CLI
OpenFOAM did not build its bundled ParaView reader	`ParaView not found ... Skipping`	use system ParaView and `foamToVTK`	solved
`gmshToFoam` imported walls as generic patches	boundary plus createPatch log	manually change only journal/stationary types; preserve ranges	solved
`createPatch` could not change existing types	`Only moving patch faces - type will remain the same`	retain as history; omit on replay	solved
Extended `checkMesh` flags thin-cell determinant	286,106 cells below generic 0.001	retain warning; cross-check standard and exact project metrics	understood, not hidden
Need distinguish plumbing from physics	zero-flow exact solution	made a separate zero-flow checkpoint	solved
0-rpm pressure-fed case reaches 2.518 m/s	0.5 MPa feed-to-end pressure difference	distinguish stationary wall speed from pressure-driven fluid speed; feed mean is 0.1140 m/s	explained
Feed flow is negative	OpenFOAM uses each patch's outward normal	negative `U dot n` means inflow; positive axial values mean outflow	explained
Pressure-only time 285 was called an initialization	it is a continuously pressure-fed stationary solution	reclassify it as commissioning/hydrostatic component, not pure hydrodynamics	corrected
Serial pressure run was slow	early serial history	four-way Scotch decomposition	solved
First sandbox MPI call could not create a PMIx interface socket	`pmix_ifinit ... errno=1`	permit local loopback sockets; ordinary user run needs no root flag	solved
Paper quotes both 2.6 m/s and 2000 rpm	radius conversion gives 496.6 rpm versus 10.472 m/s	preserve both as separate intended stages	documented
Current mesh is 60 mm, paper is 100 mm	geometry/manuscript comparison	call it paper-grounded, not exact reproduction	documented
200 rpm liquid pressure crosses absolute vacuum immediately	`min(p)-119.899=` versus floor `-117.820`	reject higher-speed single-phase sweep	correctly stopped
Pressure solver later reaches 100 iterations and enormous negative pressure	`log.rpm-200`	retain only as diagnostic; add rupture/cavitation physics	unresolved physically
Fluent S100 also diverges	`log-inlet-100.txt`, NaN/FPE	use as corroboration, not proof of identical cause	separate Fluent audit still needed
Direct paper lacks VOF phase properties	paper inspection	label vapour/sigma/nuclei values as placeholders	documented
Uniform VOF pressure start explodes	`-3.64 MPa` at first pseudo-step	map accepted single-phase pressure into absolute pascals	greatly improved

Problem or observation	Evidence	Response	Present status
Pressure-map path duplicated	`-case` plus case-prefixed `-dict`	use `-dict system/mapPressureFunctions`	solved
Mapping wrapper made `type` undefined	parser error	put function objects at top level	solved
Runtime monitors needed missing fields during mapping	missing `phi`/`U`/`p` errors	use dedicated pressure-only mapping dictionary	solved
Min/max mapping objects lacked `writeFields`	dictionary error	add `writeFields false`	solved
Mapping generated wrong patch entry types	mapped `p_rgh` output	restore wall/end/feed types with `foamDictionary`	solved
Three-step mapped run still below `pSat` and locally unsteady	smoke-test monitors	call it compatibility test, not convergence	correctly limited
Direct continuation loses 43.898% boundary flow	time-20 monitors	stop and reject	correctly stopped
VOF directories look like seconds	`localEuler` setup	label them pseudo-time counters, never physical time	documented
Pressure clipping would hide missing physics	dimensional/model audit	no `limitPressure`	deliberate decision
Time 285 carries 0.5 MPa-driven velocity into a nominal zero-bias study	field provenance	seed the new branch from `openfoam_s100_zero_flow/0/U` instead	solved
Equal ambient absolute pressure produces a fake conical-surface load	force object defaulted `pRef` to zero	set `pRef 101325` and take moments about `(0 -3e-5 0.03)`	solved
Direct 0 to 20 rpm jump produces negative pressure and large continuity error	preserved times 4 through 8	reject and archive it; introduce 1 rpm first	correctly stopped
Cavitation-enabled 0-rpm zero-bias field remains quiescent	time-3 monitor values	freeze `checkpoints/0rpm/3`	accepted
1-rpm zero-bias field settles with positive pressure and no vapour	time-75 monitors; 0.0064% flow imbalance	freeze `checkpoints/1rpm/75` before any further speed	accepted continuation
5-rpm shear field was still drifting at times 100 and 125	viscous torque changed 1.56% and 0.33% in the preceding steps	hold the same speed through time 150 instead of advancing	accepted at time 150
5-rpm zero-bias field settles with positive pressure and no vapour	time-150 monitors; 0.00096% flow imbalance	freeze `checkpoints/5rpm/150`	accepted continuation
Low-speed branch remains liquid at 10, 12, 14, and 15 rpm	checkpoints 225, 300, 350, and 400	preserve every accepted field before diagnosis	accepted through 15 rpm
Direct 10-to-15-rpm restart crosses `pSat` on its first update	time 226: `p_min`-1.448 Pa=; time 227: `alpha_min=0.5803`	preserve and reject the run	diagnosed failure
Smaller `maxCo`, `maxAlphaCo`, and `maxDeltaT` do not remove the first pressure undershoot	damped control reproduces `p_min`-1.448 Pa=	stop treating this as a time-step-only problem	hypothesis rejected
Forcing startup `correctPhi` does not rescue the jump	fresh-start control reproduces the same first two updates	rule out missing restart flux correction	hypothesis rejected
The same direct jump survives when cavitation cannot activate	`pSat-1e9` Pa= control keeps `alpha=1` and corrected local continuity near `1e-7`	identify active phase-change feedback as the amplifier	causal control passed
The actual model survives the same 10-to-15-rpm change when `omega` is continuous	native `Function1` ramp keeps `alpha=1` and corrected continuity near `1e-8`	use native ramp plus target-speed hold	solution passed

Problem or observation	Evidence	Response	Present status
A 16-rpm field exists at root time 425	it was run before the jump audit	do not promote it or create a checkpoint	unaccepted diagnostic

What increasing inlet size would and would not do

Increasing inlet area lowers the local inlet velocity and pressure loss for a given flow, and a better-resolved circular footprint can change the local pressure distribution. It does not supply missing film-rupture physics.

In this setup the feed pressure is prescribed, the S100 mesh has low non-orthogonality, and the rotating low-pressure region is produced by the eccentric bearing film. A larger inlet might shift the pressure field or delay the threshold, but it is not a principled fix for negative absolute pressure. Inlet shape/size should be studied only after a physical cavitation or rupture model can converge.

What the circular and staircase inlet variants are for

- S16, S100, and S390 ask how a mesh-aligned staircase approximation changes inlet area, perimeter, and local flow while leaving the global structured mesh family familiar.
- Tensor-warp cases keep the same cell count/connectivity but deform nodes toward an analytic circular rim. They isolate geometry from cell count, though strong square-to-circle distortion can fail the strict 45 degree non-orthogonality gate.
- O-grid cases add a small number of cells around a circular insert to improve body fitting and local quality. They trade a larger topology change for a cleaner inlet neighborhood.
- Inscribed variants keep the nominal drilling radius.
- Equal-area variants enlarge the nodal radius slightly so the straight-edge polygon has the target circular area. They are controlled research variants, not nominal geometry.

For the present solver failure, those variations are a later numerical sensitivity study. They cannot turn a one-phase liquid model into a film-rupture model.

Current experiment and next stages

The old recommendation to start from mapped time 285 has been retired. It would import pressure-driven flow into a pure-hydrodynamic experiment.

The replacement branch has now:

1. started from the exact zero-flow field;
2. set feed and both axial ends to equal ambient absolute pressure;
3. activated `incompressibleVoF/VoFCavitation/SchnerrSauer` at 0 rpm;
4. accepted the quiescent 0-rpm gate;
5. rejected a diagnostic direct jump to 20 rpm;
6. accepted a settled 1-rpm checkpoint;
7. accepted a settled 5-rpm checkpoint after extending the constant-speed hold to pseudo-time 150;
8. accepted 10, 12, 14, and 15-rpm liquid checkpoints;
9. rejected a direct 10-to-15-rpm restart after controlled comparisons;
10. reached 15 rpm cleanly with the actual cavitation model by replacing the discontinuous restart with a native continuous `omega` ramp.

The important result is no longer "use enough manually chosen intermediate speeds." The direct restart changes the wall boundary by 50% while the interior velocity, pressure, and flux fields still describe 10 rpm. Its first coupled update briefly crosses `pSat`. The active Schnerr-Sauer source then couples pressure, phase fraction, and a liquid-to-vapour density ratio of about 3.07e7, amplifying the transient into a continuity failure.

That explanation is an inference from the controlled runs, not a claim made by the bearing paper. The decisive observations are:

- lower local time-step limits leave the first undershoot unchanged;
- startup flux correction leaves it unchanged;
- disabling only the cavitation threshold lets the same direct jump recover;
- continuously changing only `omega` keeps pressure far above `pSat` with the real cavitation model active.

Future speed changes use one native continuous `omega` table followed by a constant target-speed hold. This is numerical homotopy in `localEuler` pseudo-time, not a physical acceleration history. Near a genuine cavitation onset, use a shallower ramp and stop at the first sustained phase change to audit the uncalibrated cavitation inputs and pressure/phase coupling.

At each stage require all of:

- pressure residual/field no longer drifting;
- bounded and stable `alpha.oil`;
- journal force and moment plateaus;
- inlet plus outlet flow imbalance below a declared tolerance;
- no negative absolute pressure outside a small iterative transient.

Only after a ramped 20-rpm state passes should the study widen to `50, 100, 150, 200 rpm`. Only after 200 rpm passes should it test about 496.6 rpm for the paper's 2.6 m/s. Treat 2000 rpm as a later, separate operating point.

Exact setup, results, replay commands, and ParaView paths: zero-pressure-bias hydrodynamic ramp. That chapter also explains why intermediate speeds distinguish a failed nonlinear jump from a repeatable operating-point/model failure.

The physics decision needed after the staged audit

Run at least two explicitly named model branches:

- **Pure-vapour branch**: Schnerr-Sauer with temperature-specific oil vapour properties and sensitivity to `pSat`, vapour density/viscosity, `n`, and `dNuc`.
- **Gas/rupture branch**: a model for released/entrained air or an appropriate Reynolds mass-conserving rupture boundary.

The important scientific question is whether the observed cavity is limited by pure oil vaporization near vacuum or by gas release/edge ingestion at a much higher pressure. For a low-volatility hydraulic oil, the second mechanism may dominate.

Acceptance language for the next result

Do not call a run converged because `foamRun` exits zero. Report:

- mesh and model revision;
- every non-paper material parameter;
- initialization and ramp history;
- minimum absolute pressure;
- minimum/mean phase fraction;
- all boundary flows and relative imbalance;
- journal force/moment histories;
- sensitivity result, not only one chosen parameter set;
- accepted/rejected status on every image.

Until those checks pass, the honest result is:

> The stationary 0.5 MPa case is an accepted pressure-driven commissioning result, not pure hydrodynamics. A separate zero-pressure-bias cavitation branch has accepted liquid checkpoints through 15 rpm. A controlled audit shows that its failed direct 10-to-15-rpm restart was a discontinuous-update instability, not proof that 15 rpm is physically unsolvable. It has not yet produced or validated a cavity, so no cavitating bearing-performance claim is made.

Later reduced-model candidate

A native OpenFOAM Reynolds/JFO midsurface application was subsequently continued to 2000 rpm. It converges and closes its own discrete flow balance, but predicts a severe 64.455% rupture footprint, with 47.820% of area below `thetaFill=0.5`. Internal C++/Python agreement is not external validation because both implementations share the same governing assumptions.

The candidate, plots, and explicit validation blockers are recorded in Chapter 14. This does not change the rejected status of the 3-D Schnerr–Sauer experiments above.

Previous: artifacts | Index | Next: zero-pressure-bias ramp

11 - Zero-Pressure-Bias Hydrodynamic Ramp

Source: `docs/handoff/11-pure-hydrodynamic-ramp.org`

Previous: problems and decisions | Index | Next: five-rpm jump diagnosis

What changed

The new branch answers a different question from the old 0.5 MPa case:

> Can journal rotation alone generate a stable hydrodynamic pressure field when all openings have the same ambient pressure and rupture/cavitation physics is active before rotation starts?

Case: out/openfoam$_{\text{s100purehydrodynamic}}$

This case does **not** use the pressure-fed time-285 field. It starts from the exact zero-velocity field in openfoam$_{\text{s100zeroflow}}$.

Four experiments that must not be mixed

Rotation	Pressure bias	Meaning
0 rpm	0	quiescent solver/cavitation equilibrium
0 rpm	0.5 MPa	stationary pressure-driven commissioning case
greater than 0	0	pure hydrodynamic bearing experiment
greater than 0	0.5 MPa	hybrid hydrostatic/hydrodynamic experiment

Here, "pure hydrodynamic" means that rotation is the only imposed energy source. The feed and axial ends remain connected to ambient reservoirs so that oil can enter or leave as the solution requires. It does not mean "fill the domain and then close the feed."

Closing the feed would change `pressure_feed` into a wall. That is a separate sealed-cavity experiment, not an initialization step for the present reservoir-connected bearing.

Exact initial state and boundary meaning

- Mesh: byte-for-byte copy of `openfoam_s100_zero_flow/constant/polyMesh`.
- Velocity: copy of `openfoam_s100_zero_flow/0/U`; internal `U` is zero and journal speed is 0 rpm.
- Pressure: uniform `p_rgh = 101325 Pa` absolute.
- Oil fraction: uniform `alpha.oil = 1`.
- `pressure_feed`, `axial_end_z0`, and `axial_end_zL` all use `p_rgh = 101325 Pa`.
- All three opening `alpha.oil` conditions are `inletOutlet` with oil fraction one on inflow. Vapour can be advected out instead of being overwritten by a fixed oil value.
- `journal_wall` and `stationary_wall` remain no-slip walls.
- Gravity is zero, so `p_rgh` is absolute pressure.

The 0.5 MPa field, `285/U`, `285/p`, and all pressure-mapping helper fields were deliberately excluded.

Rupture/cavitation treatment

The model is active from the 0-rpm gate:

```
solver            incompressibleVoF
fvModel           VoFCavitation
mass transfer     SchnerrSauer
time treatment    localEuler steady pseudo-time
```

This is preferable to allowing a single liquid to carry negative absolute pressure or clipping pressure after the solve. It conserves a transported liquid/vapour fraction and supplies mass-transfer source terms.

It is not yet a validated oil-bearing cavitation law. The direct conical paper uses a Reynolds rupture condition, not three-dimensional Schnerr-Sauer. The current `pSat` is a lubricant screening value, while surface tension, vapour properties, nuclei density, and nucleus diameter are declared assumptions copied or adapted from available OpenFOAM practice. Those inputs need oil-specific calibration or sensitivity study.

An important physical distinction remains:

- pure oil vapour needs pressure close to the very low vapour pressure;
- dissolved or entrained air can appear at much higher pressure;
- a Reynolds/JFO rupture model represents a broken film without resolving individual bubbles.

The present branch is therefore a native liquid/vapour baseline, not proof that real bearing cavities contain pure oil vapour.

Force-reference correction

Absolute ambient pressure acting on a conical surface has a nonzero resultant. OpenFOAM's force object defaults `pRef` to zero, so an absolute-pressure field would include a large meaningless atmospheric load.

The new monitor uses:

```
pRef  101325;
CofR  (0 -3e-05 0.03);
```

Loads are therefore gauge loads, and moments are taken about the bearing midpoint. Without `pRef`, even the exact ambient 0-rpm field reported a spurious axial force. This was a monitoring error, not a fluid force.

Verification and results

1. Mesh gate

   `checkMesh` retained the accepted imported topology:

   ```
   points: 322816
   cells: 294912
   hexahedra: 294912
   regions: 1
   max non-orthogonality: 9.99988191989 degrees
   max skewness: 0.440812139371
   Mesh OK.
   ```

2. 0 rpm cavitation-enabled gate

   The three-step gate ended normally:

Quantity	Result
Maximum velocity	1.40308779143e-10 m/s
Absolute pressure range	101324.999199 to 101325.000001 Pa
Minimum/mean/maximum `alpha.oil`	1 / 1 / 1
Opening flows	order 1e-16 m3/s

   This proves that adding the phase model does not disturb the exact zero-pressure-bias equilibrium. The checkpoint is `checkpoints/0rpm/3`.

3. Why the direct 20 rpm jump was rejected

   A diagnostic jump from 0 directly to 20 rpm was intentionally screened. By pseudo-time 8 it had:

Quantity	Result
Minimum absolute pressure	-839.93 Pa
Maximum absolute pressure	206215 Pa
Minimum `alpha.oil`	0.42547
Mean `alpha.oil`	0.971189

   Local continuity errors were still very large and all three openings showed inflow. Those values describe an abrupt numerical transient, not a stable cavity. The complete diagnostic was moved, not deleted:

   diagnostics/direct-20rpm-from-zero

4. Accepted 1 rpm checkpoint

   The 0-rpm field was restored and held at 1 rpm through pseudo-time 75:

Quantity	Pseudo-time 75
Maximum velocity	5.536280758681e-3 m/s
Absolute pressure range	96371.0996 to 106278.9770 Pa
Minimum/mean/maximum `alpha.oil`	1 / 1 / 1
Feed flow	-8.773155737709e-12 m3/s
z0 flow	+4.731998688713e-12 m3/s

Quantity	Pseudo-time 75
zL flow	+4.040026601934e-12 m3/s
Relative boundary-flow imbalance	6.4431e-5
Hydrodynamic pressure force, X	17.2442084 N
Hydrodynamic pressure moment, Y	-0.1617377 N.m
Viscous torque, Z	-0.00387446 N.m

From pseudo-time 74 to 75, maximum velocity changed by 0.0045%, pressure force by 0.0050%, and viscous torque by 0.0923%. This passes the low-speed continuation gate.

No vapour formed at 1 rpm because pressure remained far above `pSat`. The cavitation model was active; its source simply had no physical reason to create vapour at this speed.

Checkpoint: `checkpoints/1rpm/75`.

5. Accepted 5 rpm checkpoint

   The accepted 1-rpm field was restarted at 5 rpm and held through pseudo-time

   (a) Time 100 was not accepted because the last-step viscous-torque drift was

   1.56%. Time 125 was also held back because torque and viscous force were still changing by 0.33% and 0.46% per step. No speed change was made during those extensions.

   At time 150:

Quantity	Pseudo-time 150
Maximum velocity	2.76875124245e-2 m/s
Absolute pressure range	76544.1090 to 126106.2355 Pa
Minimum/mean/maximum `alpha.oil`	1 / 1 / 1
Feed flow	-4.698286385345e-11 m3/s
z0 flow	+2.865263203689e-11 m3/s
zL flow	+1.832932999584e-11 m3/s
Relative boundary-flow imbalance	9.59743e-6
Pressure-force magnitude	86.2697629 N
Pressure-moment magnitude	0.8089296 N.m
Viscous-force magnitude	0.0975269 N
Viscous torque, Z	-0.01928550 N.m

   Last-step drifts were 0.0030% for maximum velocity, 0.0047% for pressure force, 0.0874% for viscous force, and 0.0603% for viscous torque. Their five-step drifts were 0.0170%, 0.0238%, 0.5010%, and 0.3473%. The final corrected local continuity error was 5.44e-9.

   Pressure stayed far above `pSat` and `alpha.oil` remained exactly one, so this is a settled single-liquid point within the cavitation-enabled branch, not a cavity result.

   Checkpoint: `checkpoints/5rpm/150`.

   Case-local summary: $\mathrm{RUN}_{\mathrm{RESULTS}}$.md.

Recreate the clean branch

The shortest exact replay reuses only the retained clean inputs, not any generated high-pressure field:

```
repo=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
zero="$repo/out/openfoam_s100_zero_flow"
template="$repo/out/openfoam_s100_pure_hydrodynamic"
case="$repo/out/openfoam_s100_pure_hydrodynamic_replay"

test ! -e "$case" || {
  echo "Refusing to overwrite $case"
  return 1
}
```

```
mkdir -p "$case/0" "$case/constant" "$case/system"
cp -a "$zero/constant/polyMesh" "$case/constant/"
cp -a "$zero/0/U" "$case/0/"
cp -a "$template/0/alpha.oil" "$template/0/p_rgh" "$case/0/"
cp -a "$template/constant/fvModels" \
      "$template/constant/g" \
      "$template/constant/momentumTransport" \
      "$template/constant/phaseProperties" \
      "$template/constant/physicalProperties.oil" \
      "$template/constant/physicalProperties.vapour" "$case/constant/"
cp -a "$template/system/controlDict" \
      "$template/system/decomposeParDict" \
      "$template/system/functions" \
      "$template/system/fvSchemes" \
      "$template/system/fvSolution" "$case/system/"
touch "$case/S100-pure-hydrodynamic.foam"

foam_root=/home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14

bash -lc "
  source '$foam_root/etc/bashrc'
  foamDictionary '$case/system/controlDict' -entry startFrom -set startTime
  foamDictionary '$case/system/controlDict' -entry endTime -set 3
  foamDictionary '$case/system/controlDict' -entry writeInterval -set 1
"
```

The source `0/U` still contains `omega 0 [rpm]`. Verify before running:

```
bash -lc "
  source '$foam_root/etc/bashrc'
  foamDictionary '$case/0/U' \
    -entry boundaryField/journal_wall/omega
  foamDictionary '$case/0/p_rgh' \
    -entry boundaryField/pressure_feed/value
  foamDictionary '$case/system/functions' \
    -entry journalForces/pRef
"
```

Run the 0-rpm gate:

```
bash -lc "
  set -o pipefail
  source '$foam_root/etc/bashrc'
  checkMesh -case '$case' 2>&1 | tee '$case/log.checkMesh.standard'
  foamRun -case '$case' 2>&1 | tee '$case/log.equilibrate-0rpm'
"
```

Preserve it, then start the first constant-speed stage:

```
mkdir -p "$case/checkpoints/0rpm"
cp -a "$case/3" "$case/checkpoints/0rpm/3"

bash -lc "
  set -o pipefail
  source '$foam_root/etc/bashrc'
  foamDictionary '$case/3/U' \
    -entry boundaryField/journal_wall/omega -set '1 [rpm]'
  foamDictionary '$case/system/controlDict' -entry startFrom -set latestTime
  foamDictionary '$case/system/controlDict' -entry endTime -set 75
  foamDictionary '$case/system/controlDict' -entry writeInterval -set 5
  foamRun -case '$case' 2>&1 | tee '$case/log.ramp-1rpm'
"
```

Do not automate speed advancement blindly. Inspect the monitors and keep the same speed until the gate passes.

Next controlled stages

The accepted branch is now:

`0 → 1 → 5 → 10 → 12 → 14 → 15 rpm`

All seven checkpoints are liquid states: `alpha.oil` is exactly one and minimum absolute pressure stays above `pSat`. The accepted 15-rpm state is `checkpoints/15rpm/400`.

The manual intermediate ladder was useful for diagnosis, but it is not the future operating procedure. A direct restart from the accepted 10-rpm field to 15 rpm failed even though 15 rpm itself is solvable. Controlled copies showed:

- lower `maxCo`, `maxAlphaCo`, and `maxDeltaT` did not change the first negative-pressure update;
- forcing startup `correctPhi` did not change it;
- preventing Schnerr-Sauer activation while leaving the model loaded let the same direct jump recover;
- OpenFOAM's native continuous `omega` ramp reached 15 rpm with the actual cavitation model active, `alpha.oil` equal to one, and corrected local continuity of order `1e-8`.

The failure mechanism is therefore a discontinuous nonlinear update. The wall velocity changes by 50% at restart while the interior fields still describe 10 rpm. The first coupled pressure update briefly crosses `pSat`; active phase-change feedback then amplifies the transient. This conclusion comes from the controlled calculations, not from the paper.

The detailed evidence, commands, and control cases are in the five-rpm jump diagnosis.

1. Continuation policy

   Use a native `Function1` table for wall `omega`:

   ```
   omega
   {
       type table;
       values
       (
           (225 1.0471975511965976)
           (250 1.5707963267948966)
           (1000 1.5707963267948966)
       );
   }
   ```

   This example changes 10 to 15 rpm over 25 pseudo-steps and then holds 15 rpm. It is a solver homotopy, not a physical acceleration schedule: `localEuler` time labels are not seconds.

   For the next target:

   (a) copy an accepted checkpoint to an isolated case;
   (b) ramp `omega` continuously instead of replacing it with a new constant;
   (c) hold the target speed until loads and flows plateau;
   (d) promote only after all gates pass.

   Near the expected pressure threshold, make the native ramp shallower and stop at the first sustained `alpha.oil < 1`. That point requires a separate audit of the uncalibrated cavitation inputs and coupling; it must not be forced through by pressure clipping.

   For every stage require:

   - `0` < alpha.oil <= 1=;
   - no material negative-pressure undershoot after settling;
   - pressure, phase fraction, flows, force, and moment have plateaued;
   - relative opening-flow imbalance below 0.5% for continuation and preferably below 0.1% for a reported point;
   - no fatal error, floating-point exception, or growing continuity failure.

   Only after a ramped 20-rpm state passes should the study widen toward `50, 100, 150, 200 rpm` and then the paper's approximately `496.6 rpm` surface-speed point. The paper's separately quoted 2000 rpm remains a later operating point.

Animation decision

No scientific animation has been produced through 15 rpm. Every accepted field has `alpha.oil` exactly one everywhere, so an oil-fraction video would show no cavity motion. The rejected jump is failure evidence, not a physical cavity animation.

Generate the first scientific animation only after a rotating stage both forms a nontrivial phase/pressure structure and passes the conservation and drift gates. Animate the held-speed convergence separately from any future physical-time startup; pseudo-time frames must never be labelled as seconds.

Inspect in ParaView

```
case=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing/out/openfoam_s100_pure_hydrodynamic

paraview "$case/S100-pure-hydrodynamic.foam"

foamToVTK -case "$case" -time '0,3,75,150,225,300,350,400' \
  -fields '(U p p_rgh alpha.oil)' -allPatches -useTimeName
```

The existing explicit `VTK/` export covers the exact zero seed and accepted 1-rpm and 5-rpm fields; the command above adds all later accepted checkpoints. Gmsh remains best for the original mesh; ParaView is best for pressure, velocity, oil fraction, streamlines, and cutaways.

Previous: problems and decisions | Index | Next: five-rpm jump diagnosis

12 - Five-RPM Jump Diagnosis

Source: `docs/handoff/12-five-rpm-jump-diagnosis.org`

Previous: zero-pressure-bias ramp | Index | Next: guarded ramp automation

Question and status

The accepted 10-rpm field failed when its wall speed was replaced directly with 15 rpm. This chapter isolates why; it does not promote the rejected transient into bearing physics.

All runs use `incompressibleVoF`, Schnerr-Sauer, the same S100 mesh, equal 101325-Pa openings, and `localEuler`. Thus each time label is a **steady pseudo-time continuation index**, not seconds. Neither the manual RPM ladder nor the native `omega` ramp is a physical spin-up calculation.

Experiment	Result	Diagnostic meaning
direct 10 -> 15 rpm	rejected	first update crosses `pSat`; phase/pressure runaway follows
accepted 14 -> 15 rpm	accepted	15 rpm itself has a bounded liquid solution
direct jump, lower Co limits	rejected	smaller local steps soften but do not remove the trigger
fresh start with `correctPhi` path	rejected	startup flux correction is not the missing mechanism
direct jump, `pSat`-1e9 Pa=	control only	suppressing mass transfer prevents collapse
`VoFCavitation` removed	secondary control only	bounded, but startup correction confounds the comparison
native 10 -> 15 `omega` ramp and hold	accepted diagnostic	continuous continuation reaches the same target cleanly

Decisive comparison

The direct jump copied `checkpoints/10rpm/225`, changed only wall `omega` to 15 rpm, and ran to 250. Its first new step already had `pmin`-1.44820 Pa=, below the configured `pSat` of 0.5 Pa. At 227, `pmin`-1168.97 Pa= and `alpha.oil_min=0.580278`. At 250, `pmin`-3478.14 Pa=, `alpha.oil` was `0.0637792/0.995928/1`, and corrected local continuity was 122.656.

All three openings were inward and no monitored quantity plateaued. The run ended numerically, but the state is rejected. Evidence: diagnostic README and complete direct-jump log.

By contrast, the accepted 14-rpm checkpoint at 350 was changed to 15 rpm and held through 400. At its first new step, 351, `pmin` was 22739.98 Pa, `alpha.oil` remained one, and corrected local continuity was `2.28e-8`. At 400, `pmin/pmax` was `26770.25/175874.34 Pa`, `alpha.oil` was `1/1/1`, and corrected local continuity was `2.00e-8`. The opening-flow imbalance was `4.72e-6` as a fraction of the sum of absolute opening flows; five-step viscous-torque drift was 0.174%. Evidence: 350–375 log, 375–400 log, and accepted checkpoint.

Controls

1. Lower Courant and pseudo-step ceilings

The direct jump was repeated with `maxCo` 0.25, `maxAlphaCo` 0.125, and `maxDeltaT` 0.1. The first-step `pmin`-1.44814 Pa= was effectively unchanged. At 250, `pmin`-1010.18 Pa=, `alpha.oil_min=0.524214`, and corrected local continuity was 46.30. Damping reduced the final excursion but did not stabilize the branch. Evidence: lower-Co log.

2. Fresh-start `correctPhi` control

   The same checkpoint was relabelled as time 0 and started with `startFrom startTime`. This forced the startup `pcorr` path before the time loop: residual `0.999999` to `2.25992e-5` in 100 iterations. Nevertheless, time 1 gave `pmin`-1.44818 Pa=, matching the original first update. By time 25, `pmin`-2493.67 Pa=, `alpha.oil_min=0.0636895`, and corrected local continuity was 123.77. The `pcorr` solve hit its iteration limit, so this is not proof that every possible flux correction is irrelevant; it does show that the observed startup correction does not cure this jump. Evidence: fresh-start log.

3. Inactive-cavitation threshold control

   Schnerr-Sauer remained loaded, but `pSat` was set to `-1e9 Pa` so its source could not activate on this trajectory. The direct jump then stayed single-liquid: at 226, `pmin=9997.75 Pa` and corrected local continuity was `1.42e-7`; at 250, `pmin=26950.53 Pa`, `alpha.oil` was `1/1/1`, and all opening flows balanced. Loads were still settling, so this is a diagnostic, not an accepted operating point. The threshold is deliberately unphysical and must not replace the declared 0.5-Pa model value. Evidence: inactive-cavitation log.

4. Model-removal secondary control

   A secondary copy removed `VoFCavitation` completely. It stayed bounded through 250, with `pmin=51687.49 Pa` and corrected local continuity `1.15e-8`. However, removing the model also caused the startup `pcorr` path to run, and the interior field advanced only slightly from its 10-rpm seed. It is therefore consistent with the threshold control but is not the causal comparison. Evidence: model-removal log.

5. Native `omega Function1` ramp

   The accepted 10-rpm checkpoint used OpenFOAM's native table: `values ((225 1.0471975511965976) (250 1.5707963267948966) (1000 1.5707963267948966))`.

   At 250, after the ramp, `pmin` was 24642.20 Pa, `alpha.oil` was `1/1/1`, and corrected local continuity was `1.26e-8`. The first 15-rpm hold to 275 remained bounded, but its five-step viscous-torque drift was still 2.02%; it was evidence of recovery, not yet the settled gate. A second hold to 300 ended with `pmin/pmax 26860.42/175787.75 Pa`, `alpha.oil 1/1/1`, corrected local continuity `1.75e-8`, and relative opening-flow imbalance `4.71e-7`. Pressure and loads were close, but the feed and z0 flows still changed by 2.27% and 2.73% over five steps, so time 300 did not pass the full plateau gate.

   The unchanged 15-rpm hold therefore continued to time 400. Its final values were `pmin/pmax 26760.11/175883.51 Pa`, `alpha.oil 1/1/1`, and corrected local continuity `1.81e-8`. The feed, z0, and zL flows were `-1.67681e-10`, `+1.45378e-10`, and `+2.23024e-11 m3/s`; their relative sum imbalance was `2.09e-6`. Over the final five steps those three flow magnitudes changed by 0.0553%, 0.1077%, and 0.2790%. `Umax` changed by 0.0000173%, pressure force X by 0.00596%, and viscous torque by 0.0000553%. The native ramp and hold therefore pass the complete diagnostic gate. Against the independently accepted 15-rpm checkpoint, pressure, `Umax`, and load differences are all below 0.7%. Individual opening-flow percentages differ more because each signed integral is only of order `1e-10 m3/s`; both states have negligible, well-balanced net exchange.

   Evidence: 225–250 ramp log, 250–275 hold log, and 275–300 hold log, 300–350 hold log, and 350–400 hold log.

What is evidence and what is inference

Evidence: 15 rpm is solvable from 14 rpm; lower Co limits and startup `correctPhi` do not prevent the direct jump's first `pSat` crossing; disabling source activation lets the direct jump recover; and a continuous wall-speed ramp stays above `pSat` with the declared model active.

Inference: the discontinuous 50% wall-speed change leaves 10-rpm interior fields inconsistent with the boundary, creates the pressure undershoot, and active phase change amplifies that transient. The controls identify this as the mechanism of the observed failure, not as a proof about every coupling scheme.

The controls do **not** prove that physical cavitation is absent, validate Schnerr-Sauer for this oil, justify changing `pSat`, or turn pseudo-time into a physical acceleration history.

Exact isolated reruns

Run the following in one Bash shell. Each replay path must be absent; the guard preserves all original checkpoints, logs, and diagnostic directories.

```
repo=/home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing
root="$repo/out/openfoam_s100_pure_hydrodynamic"
source /home/aniketnegi/code/ankt/btp/.openfoam/OpenFOAM-14/etc/bashrc
set -eo pipefail
seed_case () {
  local target=$1 checkpoint=$2
  test ! -e "$target" || { echo "Refusing to overwrite $target"; return 1; }
  mkdir -p "$target"
  cp -a "$root/constant" "$root/system" "$target/"
  cp -a "$checkpoint" "$target/"
}
set_15rpm () {
  foamDictionary -writePrecision 17 "$1/U" -entry \
    boundaryField/journal_wall/omega -set '1.5707963267948966 [rad/s]'
}
run_to () {
  local target=$1 end=$2 log=$3
  foamDictionary "$target/system/controlDict" -entry startFrom -set latestTime
  foamDictionary "$target/system/controlDict" -entry endTime -set "$end"
  foamRun -case "$target" 2>&1 | tee "$target/$log"
}
```

Accepted 14 -> 15 control and rejected direct 10 -> 15 baseline:

```
case="$root/diagnostics/replay-accepted-14to15"
seed_case "$case" "$root/checkpoints/14rpm/350"
set_15rpm "$case/350"
run_to "$case" 375 log.accepted-14to15-350-375
run_to "$case" 400 log.accepted-14to15-375-400
case="$root/diagnostics/replay-direct-10to15"
seed_case "$case" "$root/checkpoints/10rpm/225"
set_15rpm "$case/225"
run_to "$case" 250 log.direct-10to15
```

Lower-Co and inactive-cavitation controls:

```
case="$root/diagnostics/replay-direct-10to15-lower-Co"
seed_case "$case" "$root/checkpoints/10rpm/225"
set_15rpm "$case/225"
foamDictionary "$case/system/fvSolution" -entry PIMPLE/maxCo -set 0.25
foamDictionary "$case/system/fvSolution" -entry PIMPLE/maxAlphaCo -set 0.125
foamDictionary "$case/system/fvSolution" -entry PIMPLE/maxDeltaT -set 0.1
run_to "$case" 250 log.direct-10to15-lower-Co
case="$root/diagnostics/replay-direct-10to15-inactive-cavitation"
seed_case "$case" "$root/checkpoints/10rpm/225"
set_15rpm "$case/225"
foamDictionary "$case/constant/fvModels" -entry \
  VoFCavitation/SchnerrSauer/pSat -set -1e9
run_to "$case" 250 log.direct-10to15-inactive-cavitation
```

Secondary model-removal control:

```
case="$root/diagnostics/replay-direct-10to15-no-cavitation"
seed_case "$case" "$root/checkpoints/10rpm/225"
set_15rpm "$case/225"
foamDictionary "$case/constant/fvModels" -entry VoFCavitation -remove
run_to "$case" 250 log.direct-10to15-no-cavitation
```

Fresh-start `correctPhi` control:

```
case="$root/diagnostics/replay-direct-10to15-fresh-start"
seed_case "$case" "$root/checkpoints/10rpm/225"
mv "$case/225" "$case/0"
for field in U alpha.oil alphaPhi.oil p p_rgh phi rDeltaT rho; do
  foamDictionary "$case/0/$field" -entry FoamFile/location -set '"0"'
done
```

```
foamDictionary "$case/0/uniform/time" -entry FoamFile/location -set '"0/uniform"'
foamDictionary "$case/0/uniform/time" -entry value -set 0
foamDictionary "$case/0/uniform/time" -entry name -set '"0"'
foamDictionary "$case/0/uniform/time" -entry index -set 0
set_15rpm "$case/0"
foamDictionary "$case/system/controlDict" -entry startFrom -set startTime
foamDictionary "$case/system/controlDict" -entry startTime -set 0
foamDictionary "$case/system/controlDict" -entry endTime -set 25
foamRun -case "$case" 2>&1 | tee "$case/log.fresh-start-correctPhi"
rg -n 'Solving for pcorr|Starting time loop' "$case/log.fresh-start-correctPhi"
```

Native ramp and target-speed holds:

```
case="$root/diagnostics/replay-direct-10to15-native-ramp"
seed_case "$case" "$root/checkpoints/10rpm/225"
foamDictionary -writePrecision 17 "$case/225/U" \
  -entry boundaryField/journal_wall/omega -set \
  '{ type table; values ((225 1.0471975511965976) (250 1.5707963267948966) (1000 1.5707963267948966)); }'
run_to "$case" 250 log.native-omega-ramp
run_to "$case" 275 log.native-omega-hold-250-275
run_to "$case" 300 log.native-omega-hold-275-300
run_to "$case" 350 log.native-omega-hold-300-350
run_to "$case" 400 log.native-omega-hold-350-400
```

The lower-Co, inactive-cavitation, and native-ramp changes are full control recipes. Do not combine them: doing so would destroy the causal comparison.

Previous: zero-pressure-bias ramp | Index | Next: guarded ramp automation

Guarded OpenFOAM Ramp toward 2000 RPM

Source: `docs/handoff/13-guarded-2000rpm-ramp.org`

Previous: five-rpm jump diagnosis | Index | Next: native JFO candidate

What was automated

The local driver is:

out/openfoam$_{s100purehydrodynamic}$/guarded$_{speedramp}$.py

It starts only from the accepted `checkpoints/15rpm/400` field, creates a separate work case, and uses the same native `omega` `Function1` table that survived the controlled 10-to-15-rpm experiment. It does not repeatedly replace a wall-speed constant.

The default requested milestones are:

RPM	Why it is present
20	next low-speed threshold test
50, 100, 150, 200	widening sequence already declared in the continuation policy
496.6	approximately the paper's stated surface-speed operating point
1000, 1500	guarded intermediate points
2000	later operating point quoted by the paper

`2000 rpm` is an upper request, not a promised valid result. Extrapolating the accepted 10-rpm and 15-rpm minimum pressures predicts a first saturation-pressure crossing near 20.4 rpm. That estimate is an inference, not a computed result. The driver is therefore expected to pause near the first threshold so the cavitation inputs and model can be audited.

Assumed numerical values

- 15 to 20 rpm: at most 0.05 rpm per pseudo-step, hence 100 ramp steps.
- Later ramps: at most 0.2 rpm per pseudo-step, matching the demonstrated 10-to-15-rpm native-ramp slope.
- Solver chunks: 25 pseudo-steps.
- Minimum target hold: 100 pseudo-steps.
- Maximum target hold: 500 pseudo-steps.
- Output retention: two rolling field times plus a copied checkpoint for every accepted milestone.

These are `localEuler` continuation steps. They are not seconds and do not describe a physical rotor acceleration.

Automatic stops and acceptance

The solver is stopped immediately when any monitored line reports:

- `p_min < pSat`, recorded as `SATURATION_THRESHOLD`;
- two consecutive `alpha.oil_min < 0.999999` observations, recorded as `CAVITATION_ONSET`;
- final-corrector local continuity at or above `1e-3`;
- unbounded phase fraction, NaN/Inf, a pressure solve at its iteration ceiling, a fatal error, or a floating-point exception.

A target can be accepted only after its minimum hold and all of these gates:

- finite fields, `alpha.oil` still on the single-liquid branch, and pressure above `pSat`;
- corrected local continuity below `1e-5`;
- opening-flow imbalance below 0.5%;
- last-five-step pressure, velocity, pressure-force, and pressure-moment drift below 0.5%;
- viscous force, viscous Z torque, and each opening-flow drift below 1%;
- mean phase-fraction drift below `1e-5`.

This is deliberately a liquid-branch continuation driver. It will not automatically bless a two-phase field. The first phase onset requires a human check of `pSat`, oil vapour properties, nuclei values, Schnerr–Sauer applicability, conservation, mesh resolution, and comparison with a mass-conserving film-rupture model or experiment.

1. Startup-guard correction

   The first real invocation stopped before `Create time` completed because the guard matched the normal OpenFOAM startup line:

   ```
   sigFpe : Enabling floating point exception trapping (FOAM_SIGFPE).
   ```

   That line says trapping is enabled; it is not an exception. The field never advanced beyond the accepted time-400 seed. The guard now matches actual `Floating point exception` and `FOAM FATAL` messages, while explicitly testing that the normal startup line is harmless. The original log is preserved as `guarded-ramp-to-2000/logs/guard-false-positive-sigfpe-startup.log`.

   The resume loader recognizes only this exact historical false failure when the current time and RPM still equal the last accepted checkpoint. It removes the false study row and safely resumes from 15 rpm.

First guarded campaign result

The corrected campaign accepted 20 rpm after a 100-step ramp and a 100-step hold. At pseudo-time 600:

Quantity	Accepted value
Minimum/maximum absolute pressure	1836.891944 / 200802.525362 Pa
Minimum/mean/maximum `alpha.oil`	1 / 1 / 1
Maximum velocity	0.1107952821 m/s
Corrected local continuity	3.01595e-8
Relative opening-flow imbalance	6.95131e-6
Feed / z0 / zL flow	-2.23120e-10 / +2.03440e-10 / +1.96773e-11 m3/s
Largest last-five-step drift	0.00527273

Every declared acceptance gate passed. The largest drift is the tiny zL opening flow and remains below its 1% secondary-field gate.

The next native table requested 20 to 50 rpm at 0.2 rpm per pseudo-step. On its first update, pseudo-time 601 and 20.2 rpm:

```
p_min = -0.177021858623 Pa
pSat  =  0.5 Pa
alpha.oil_min = 1
corrected local continuity = 3.82067e-5
```

The driver therefore stopped with `SATURATION_THRESHOLD` and preserved `accepted/20rpm/600`. This is a clean bracket for this continuation schedule: `20.0` rpm is an accepted liquid point, while the first `20.2`-rpm update crosses the configured saturation pressure.

It is **not** a converged cavitation result. The phase field was still exactly liquid because the stop occurred before Schnerr–Sauer could evolve a cavity, and the 20.2-rpm field had not settled. The next experiment is a finer 20.0-to-20.2-rpm onset bracket followed by the already-required vapour versus gas/rupture model audit. The terminal stop must not be bypassed into the 50-rpm campaign.

Commands

Run from the repository root:

```
cd /home/aniketnegi/code/ankt/btp/code/eccentric-conical-bearing

uv run python \
  out/openfoam_s100_pure_hydrodynamic/guarded_speed_ramp.py \
  --self-test

uv run python \
  out/openfoam_s100_pure_hydrodynamic/guarded_speed_ramp.py
```

The second command is a dry run. It prints the full speed and pseudo-step schedule without copying a case or starting OpenFOAM.

Start the actual isolated campaign:

```
uv run python \
  out/openfoam_s100_pure_hydrodynamic/guarded_speed_ramp.py \
  --run
```

Resume only when interruption occurred between completed ramp chunks:

```
uv run python \
  out/openfoam_s100_pure_hydrodynamic/guarded_speed_ramp.py \
  --run --resume
```

The driver refuses to overwrite an existing work case. It also refuses to resume `HOLDING`, `SATURATION_THRESHOLD`, `CAVITATION_ONSET`, `FAILED`, or `COMPLETE`. An interrupted hold is restarted as a new work case from the last accepted checkpoint; this prevents an unfinished hold from being mistaken for an accepted speed.

To perform a smaller first campaign, use a distinct output path:

```
uv run python \
  out/openfoam_s100_pure_hydrodynamic/guarded_speed_ramp.py \
  --target-rpm 20 \
  --work-case out/openfoam_s100_pure_hydrodynamic/guarded-ramp-to-20 \
  --run
```

Results and inspection

The default work case is `out/openfoam_s100_pure_hydrodynamic/guarded-ramp-to-2000/`. Its useful outputs are:

- `ramp-state.json` – atomic current status, safe time, current speed, last accepted speed, and stop reason;
- `ramp-results.csv` – one compact study row per accepted or stopped target, including the interpolated observed time and speed for a mid-ramp stop;
- `logs/` – one solver log per 25-step ramp or hold chunk;
- `accepted/<rpm>/<time>/` – preserved field for every accepted milestone;
- normal `postProcessing/` monitor tables.

Inspect a generated campaign in ParaView:

```
touch out/openfoam_s100_pure_hydrodynamic/guarded-ramp-to-2000/case.foam
paraview out/openfoam_s100_pure_hydrodynamic/guarded-ramp-to-2000/case.foam
```

Do not animate a stopped or rejected phase field as physical cavitation. The first useful video comes after a two-phase operating point passes a separately justified cavitation and conservation audit.

Previous: five-rpm jump diagnosis | Index | Next: native JFO candidate

14 - Native OpenFOAM Reynolds/JFO Candidate

Source: `docs/handoff/14-native-jfo-candidate.org`

Previous: guarded 3-D ramp | Index | Next: validation and Fluent path

Status

> UNVALIDATED NUMERICAL CANDIDATE. Preserve and inspect this result, but do not present its cavity fraction, load, flow, or torque as validated bearing performance.

The 3-D single-liquid OpenFOAM branch reached its declared saturation guard near 20 rpm and could not be continued honestly to the paper's high-speed points without rupture physics. A separate native OpenFOAM application, `jfoBearingFoam`, was therefore built on the unwrapped conical film midsurface. It solves a conservative Reynolds/JFO pressure–fill complementarity problem with OpenFOAM finite-volume fields and matrices.

This is not a 3-D Navier–Stokes or VOF solution. It is also not validated merely because a separate Python implementation gives nearly the same numbers: both implementations encode the same local equations, boundary choices, and active-set policy.

Source and reproducibility

Artifact	Role
jfoBearingFoam.C	native OpenFOAM finite-volume implementation
jfoPaperExact	256 by 80 source case and physical inputs
check$_{\text{jfoBearingFoam}}$.sh	build plus zero-speed acceptance check
reynolds$_{\text{jfo}}$.py	separate Python/SciPy implementation of the same model
test$_{\text{reynoldsjfo}}$.py	current minimal regression checks
test$_{\text{jfopublishedbenchmark}}$.py	published 1-D graph-consistency check for the complementarity kernel
`out/openfoam_jfo_native_256x80/`	ignored generated case, checkpoints, result ledger, and VTK

The source case uses:

- length 100 mm, mean journal radius 50 mm, and cone semi-angle 10 degrees;
- radial clearance 50 micrometres and eccentricity 30 micrometres, so `epsilon = 0.6`;
- dynamic viscosity 0.0277 Pa s;
- a central 4 mm feed region at 0.5 MPa gauge;
- atmospheric axial edges and atmospheric JFO rupture pressure;
- 256 circumferential by 80 axial control volumes.

The paper separately states 2.6 m/s and 2000 rpm. At the stated 50 mm mean radius, 2.6 m/s is 496.563 rpm. Both points were solved rather than silently equated.

Run the durable zero-speed check from the repository root:

```
bash openfoam/check_jfoBearingFoam.sh
```

Regenerate the saved plots and animations from the retained OpenFOAM fields:

```
MPLCONFIGDIR=/tmp/jfo-matplotlib \
  .venv/bin/python openfoam/visualize_jfo.py
```

Converged candidate result

The continuation `0 → 20 → 496.563 → 1000 → 2000 rpm` converged at each stored point. At 2000 rpm the 256 by 80 field reports:

Quantity	Candidate value
Minimum absolute pressure	101,325 Pa
Maximum absolute pressure	17,521,898 Pa
Maximum gauge pressure divided by 0.5 MPa	34.8411
Minimum fill fraction	0.255817
Area-weighted mean fill fraction	0.635710
Area classified as ruptured, `thetaFill < 1 - 1e-8`	64.455%
Feed flow	4.409971e-6 m3/s
Relative global flow imbalance	4.76e-8
Total force `(Fx,Fy,Fz)`	`(37045.810, 50425.411, -12113.821)` N

Quantity	Candidate value
Journal torque	-9.688458 N m
Final convergence history	1103 steps, 4.309 characteristic revolutions

The minimum is the prescribed rupture floor and occurs over many cavity cells. The maximum cell is at approximately `theta=331.172 degrees` and `z=0.048125 m`.

Why the 64.455 percent cavity is a red flag

The large area is not a threshold-counting illusion:

Diagnostic	Value
Area with `thetaFill < 0.9`	61.379%
Area with `thetaFill < 0.5`	47.820%
Film-volume-weighted liquid-content deficit	47.082%
Cavity-only area-weighted mean fill	0.435
Cavity-only film-volume-weighted mean fill	0.377

Therefore the candidate predicts a broad and strongly depleted rupture region. Classical journal-bearing rupture commonly occupies much of the diverging film, and `thetaMin` being close to the geometric `hMin/hMax approximately 0.25` is an encouraging consistency observation. Neither observation proves that this discretization, feed treatment, or reformation boundary is correct.

The shared Python active-set kernel is now consistent with graph-read pressure peaks and fronts from the published sinusoidal slider comparison described in the next handoff chapter. This is a coarse code-verification check for one kernel, not validation of this conical result. The candidate remains blocked from engineering use until the remaining checks pass:

1. an equivalent published benchmark exercised through the native OpenFOAM assembly, not only the shared Python active-set function;
2. manufactured full-film solutions that isolate the transformed conical diffusion and Couette terms;
3. a minimum three-grid convergence study for pressure, load, flow, rupture boundary, and integrated liquid deficit;
4. local as well as global conservation checks;
5. complementarity residual checks in full-film and ruptured cells;
6. sensitivity to feed-patch discretization, axial boundaries, rupture pressure, and continuation step;
7. comparison with a calibrated 3-D Fluent phase-change or gas-ingestion model, with the models named as different physics.

Saved diagnostics

- Speed sweep (PNG) and vector PDF
- Final pressure, fill, and rupture fields (PNG) and vector PDF
- Fill-severity diagnostic (PNG) and vector PDF
- Five-checkpoint H.264 animation and GIF fallback
- Checkpoint metrics CSV

The animation contains five converged operating points, not time-resolved interpolation between speeds.

What the paper comparison does and does not establish

Figure 6b of the direct conical-bearing paper is the only close published comparison currently used. At `gamma=10 degrees`, `lambda=1`, `epsilon=0.6`, and 2000 rpm, graph reading gives approximately `Pmax/Ps=35.6` for its FEA curve and `32.0` for its Fluent curve. The candidate gives 34.841 using gauge pressure or 35.044 using absolute pressure.

That agreement is useful but narrow. It checks one maximum-pressure scalar and does not validate the cavity topology, fill fraction, mass-conserving front, load vector, feed flow, or torque. The paper uses a Reynolds rupture condition rather than the present active-set JFO implementation and does not fully specify the CFD feed-port geometry.

Relation to Fluent

Fluent's standard cavitation models are 3-D liquid–vapour mass-transfer models. Selecting one does not apply the same JFO pressure–fill complementarity law. A final Fluent run must therefore be labelled as one of:

- a calibrated 3-D phase-change/gas model used as an independent physical comparison; or

- a custom UDF/UDS implementation whose discrete equations and complementarity enforcement are separately verified.

The first option is the defensible near-term comparison. The second is a solver-development project, not a settings translation.

See Validation evidence and the Fluent path for the benchmark, grid audit, pressure convention, and exact distinction between vapour cavitation and air-connected film rupture.

Previous: guarded 3-D ramp | Index | Next: validation and Fluent path

15 - JFO Validation Evidence and Fluent Path

Source: `docs/handoff/15-jfo-validation-and-fluent.org` Updated: [2026-07-29 Wed]

Previous: native JFO candidate | Index

Verdict

> The shared active-set complementarity kernel is consistent with graph-read peaks and fronts from a published 60-cell slider comparison. The 2000-rpm conical result remains an unvalidated numerical candidate because the feed footprint is grid-dependent, the native conical assembly has not passed an external solution, and the physical cavity mechanism has not been fixed as vapour formation or air ventilation.

The reported `64.455%` is high. It is surface area with `thetaFill < 1 - 1e-8`, not 64.455% vapour volume. The lower fill thresholds show that the depletion is real inside the current model, but neither that metric nor the prescribed 101,325 Pa floor can be compared directly with Fluent vapour volume fraction.

Why Reynolds and why JFO

Layer	Meaning	Current role
Reynolds equation	Thin-film mass conservation and pressure equation derived under lubrication assumptions	Governing equation on the unwrapped conical film
Simple rupture boundary	Clips or terminates the pressure solution where it would become nonphysical	Cheap, but generally loses mass and does not handle reformation correctly
JFO conditions	Jakobsson–Floberg–Olsson mass-conserving rupture and reformation law	Declares full-film pressure cells and partially filled rupture cells
Elrod–Adams/complementarity	Numerical formulations for solving Reynolds plus JFO	The current solver uses an active-set complementarity form
Fluent cavitation model	3-D liquid–vapour mass transfer derived from Rayleigh–Plesset bubble dynamics	A different physical model and an independent comparison

The Reynolds reduction is appropriate when film thickness is much smaller than bearing radius and length and the requested outputs are film pressure, load, leakage, and torque. It avoids resolving many cells through a 50-micrometre gap. It is not sufficient by itself after a full-film solution demands negative absolute pressure.

JFO was used instead of pressure clipping because it conserves lubricant mass through rupture and reformation. It is not universally "better." Use:

- thermohydrodynamic Reynolds analysis when viscosity and temperature feedback matter;
- elastohydrodynamic analysis when surface deformation matters;
- 3-D liquid–vapour cavitation when oil phase change is the mechanism;
- oil–air VOF or a related interface model when the open ends ventilate the film.

Published graph-consistency check

The automated check in $test_{jfopublishedbenchmark}$.py reuses the production Python `solve_lcp` active-set function in the incompressible limit of the 60-cell sinusoidal slider comparison in Miraskari et al. Their compressible Elrod result uses a 100 GPa bulk modulus and Figure 9 compares it with Giacopini's incompressible complementarity result:

Input	Value
Slider length	125 mm
Film thickness	15 to 25 micrometres, sinusoidal

Input	Value
Dynamic viscosity	0.015 Pa s
Sliding speed	4 m/s
Cavitation datum	=Pc=0 MPa in the paper
Bulk modulus	100 GPa in the published compressible comparison; not used in the local incompressible assembly
Grid	60 uniform cells

End pressure	Published graph	Local result
0 MPa	peak approximately 5.9 MPa; rupture at `x/a approximately 0.65`; no reformation	5.918 MPa; first cavity-cell centre 0.658; cavity reaches outlet
1 MPa	peak approximately 6.6 MPa; rupture approximately 0.65; reformation approximately 0.95	6.618 MPa; first and last cavity-cell centres 0.675 and 0.942

Run it with:

```
PYTHONDONTWRITEBYTECODE=1 \
  .venv/bin/pytest -q \
  tests/test_jfo_published_benchmark.py \
  tests/test_reynolds_jfo.py
```

The recorded result is `3 passed`. The assertions use broad graph-read bands; they do not digitize or compare the full Figure 9 pressure curves and therefore must not be described as a complete benchmark reproduction.

This coarse check exercises the shared active-set LCP policy and its ability to place rupture and reformation in the expected range. It does **not** exercise the conical metric, feed mask, axial boundary, transient transport, force integration, or native OpenFOAM matrix assembly.

Primary sources:

- Miraskari et al., *Journal of Tribology* 139 (2017), 031703;
- Giacopini et al., *Journal of Tribology* 132 (2010), 041702;
- Elrod, *Journal of Lubrication Technology* 103 (1981), 350–354.

Feed and grid audit

The midsurface solver currently assigns the circular feed to every cell whose centre lies within the 2 mm radius. That Boolean footprint changes discontinuously with resolution:

Grid	Feed cells	Effective area (mm2)	Error from `pi (2 mm)^2`
128 by 40	4	24.927	+98.4%
192 by 60	4	11.079	-11.8%
256 by 80	10	15.559	+23.8%
384 by 120	16	11.079	-11.8%
512 by 160	32	12.464	-0.8%

The 64 by 20 native case was correctly rejected because no cell centre resolved the feed. Direct, fully filled Python starts at 2000 rpm further show the non-monotonic response:

Grid	Steps	Cavity area	Maximum absolute pressure
128 by 40	604	64.485%	17.926 MPa
192 by 60	972	66.446%	17.143 MPa
256 by 80	1210	64.460%	17.526 MPa

A native 512 by 160 sensitivity run in `out/openfoam_jfo_grid_512x160_mapped/` was initialized by mapping the converged 256 by 80 field, then continued at 2000 rpm until the original convergence gate passed after 1,452 steps, or 2.836 characteristic revolutions. The exact terminal summary is retained in $grid_{512x160result}$.txt:

Quantity	256 by 80	512 by 160	Fine change
Maximum absolute pressure	17.521898 MPa	17.358535 MPa	-0.932%
Cavity area	64.4549%	64.2944%	-0.161 percentage point
Feed flow	4.409971e-6 m3/s	4.431422e-6 m3/s	+0.486%
Total `Fx`	37,045.810 N	37,031.446 N	-0.039%
Total `Fy`	50,425.411 N	49,767.590 N	-1.305%
Total `Fz`	-12,113.821 N	-12,012.649 N	-0.835% in magnitude
Journal torque	-9.688458 N m	-9.690175 N m	+0.018% in magnitude

The fine sensitivity is encouraging, but it is not a formal three-grid convergence result. The feed geometry does not form a consistent refinement family, and the coarse/intermediate sequence oscillates. No observed order, Richardson extrapolation, or GCI is reported from those grids.

The defensible fix is a feed boundary whose physical area and location are preserved during refinement, preferably a conforming or locally refined circular footprint. Only after that should the project calculate GCI for pressure, load components, feed flow, torque, cavity area, and front position using the Celik et al. procedure.

Physical-model sensitivity

The current inputs set both axial ambient pressure and JFO rupture pressure to 101,325 Pa absolute. This imposes an ambient-pressure rupture floor and is consistent with an assumed ventilated interpretation. The solver has no air phase, atmospheric-connectivity model, or gas transport, so it does not itself prove or simulate ventilation.

Changing only the rupture pressure to an illustrative 0.5 Pa in the 128 by 40 case changed cavity area from 64.485% to 53.879% and maximum absolute pressure from 17.926 MPa to 18.848 MPa. This does not establish 0.5 Pa as a correct oil property. It demonstrates that the predicted cavity depends materially on whether the model assumes an ambient-pressure rupture or an oil-vapour floor.

The axial conditions also fix pressure at the rupture value while extrapolating fill. This is consistent with a vented/starved-end interpretation and can allow the rupture region to connect to both ends, but it does not transport an air phase. Flooded or oil-wetted ends require a different, explicit replenishment condition.

Remaining numerical acceptance gates

Before presenting the conical outputs as validated:

1. preserve the feed geometry across refinement and obtain monotonic grid-convergence evidence;
2. exercise the native OpenFOAM conical assembly against an exact or published field, not only the Python LCP kernel;
3. implement a separate forced, smoothly regularized diagnostic using the published Reynolds–Elrod manufactured solution from Gravenkamp, Pfeil, and Codina and its forcing data, then measure the local observed order; the paper's second-order result belongs to its stabilized FEM, while its artificial-diffusion scheme is first order and is the relevant initial expectation for this upwind FVM;
4. report local control-volume imbalance as well as global flow imbalance;
5. report pressure, fill, and normalized complementarity residual bounds;
6. declare and test flooded versus vented ends, feed representation, rupture pressure, initial fill, and continuation sensitivity;
7. compare pressure profiles, fronts, load, flow, and torque with independent numerical or experimental data—not only one peak-pressure scalar.

For every accepted field require:

$$p - p_{\text{cav}} \geq 0, \qquad 0 \leq \theta \leq 1, \qquad (p - p_{\text{cav}})(1-\theta) = 0,$$

plus conservative flux balance and stable integrated outputs.

How to run the final Fluent comparison

1. Near-term defensible route: independent 3-D physical model

   Do not call Fluent's built-in cavitation model "the same JFO calculation." Ansys documents Schnerr–Sauer and Zwart–Gerber–Belamri as liquid–vapour mass-transfer models based on Rayleigh–Plesset bubble dynamics: Fluent cavitation theory.

Use the statically audited 3-D film/feed mesh only after it passes a live Fluent import, Mesh Check, and Report Quality audit, then:

(a) run double precision and converge the single-liquid case first;

(b) set pressure values unambiguously:

Fluent operating pressure	Pressure-inlet Gauge Total Pressure	Pressure-outlet Gauge Pressure
0 Pa	601,325 Pa	101,325 Pa
101,325 Pa	500,000 Pa gauge	0 Pa gauge

Both rows implement this project's adopted convention: supply is 0.5 MPa **above atmosphere**. The source paper states only `Ps=0.5 MPa` and does not identify its pressure reference, so gauge must not be presented as paper-confirmed. Fluent's definition is absolute pressure = operating pressure + gauge pressure. A Fluent pressure inlet specifies total pressure, whereas the JFO feed cells are fixed at static pressure. That boundary-model mismatch must be reported, especially for a surface feed without a resolved passage.

(c) decide the mechanism before selecting phases:

- for vapour cavitation, use liquid oil plus its vapour, with measured vapour pressure, density, viscosity, and operating temperature;
- for atmospheric air ingestion, use an oil–air interface model; do not use oil vapour properties to imitate ventilation;

(d) for vapour cavitation, start with Mixture plus Schnerr–Sauer, then use Zwart as a model-form sensitivity; use pressure-based coupled pressure– velocity treatment and PRESTO as recommended by the Fluent theory guide;

(e) start with robust first-order numerics, then require the reported quantities to remain stable after switching to the selected higher-order scheme;

(f) use a transient calculation if the multiphase field does not settle to a true steady state.

Monitor and save:

- global minimum and maximum **absolute** static pressure and their locations;
- inlet and both outlet mass flow rates and imbalance;
- vapour volume and thresholded vapour footprint, or air volume for the ventilation model;
- journal force components, moment/torque, and pressure profiles;
- residual histories and time histories of all integrated quantities;
- mesh and timestep/iteration sensitivity.

The global minimum need not equal vapour pressure exactly; local numerical undershoot is possible. Acceptance comes from bounded, mesh-stable fields and mass/quantity histories, not from forcing the monitor to land on one number.

2. Exact JFO parity inside Fluent: separate solver development

A `DEFINE_CAVITATION_RATE` or `DEFINE_MASS_TRANSFER` UDF changes phase mass transfer; it does not replace Fluent's pressure equation with the JFO complementarity system. Exact parity would require a custom unwrapped two-dimensional UDS/UDF implementation for pressure/fill, including anisotropic diffusion, conservative Couette flux, feed constraints, transient storage, active-set or equivalent complementarity enforcement, and its own benchmark suite. Fluent supports custom scalar transport ingredients through user-defined scalars, but this is a solver project, not a model-panel setting.

For the final thesis comparison, the shorter defensible route is to keep OpenFOAM Reynolds/JFO as the reduced film model and Fluent as an independently validated 3-D vapour or air-ingestion model. Compare only quantities that mean the same thing in both.

Previous: native JFO candidate | Index

Compiled source manifest

Every project Org source included in this PDF:

- `docs/index.org` (SHA-256 prefix `82e7d68efa67`)
- `docs/project/pipeline.org` (SHA-256 prefix `f930e2006286`)
- `docs/project/architecture.org` (SHA-256 prefix `12318e733992`)
- `docs/project/decisions.org` (SHA-256 prefix `2af760703335`)

- `docs/physics/lubrication-context.org` (SHA-256 prefix `932f12c578bd`)
- `docs/geometry/index.org` (SHA-256 prefix `221a2a5e1d95`)
- `docs/geometry/coordinates-and-parameters.org` (SHA-256 prefix `05cb3cc3032f`)
- `docs/geometry/clearance-mathematics.org` (SHA-256 prefix `81d148ee61cc`)
- `docs/geometry/cad-construction.org` (SHA-256 prefix `c0342062971e`)
- `docs/geometry/cad-validation.org` (SHA-256 prefix `30243f6d52c7`)
- `docs/geometry/acceptance-results.org` (SHA-256 prefix `39c5acfb71cb`)
- `docs/geometry/interchange.org` (SHA-256 prefix `b325d620cbfa`)
- `docs/meshing/index.org` (SHA-256 prefix `c8173e1ad50c`)
- `docs/meshing/brep-preflight.org` (SHA-256 prefix `bc9303159071`)
- `docs/meshing/surface-smoke.org` (SHA-256 prefix `ef30e47acd33`)
- `docs/meshing/structured-hex-no-port.org` (SHA-256 prefix `1a329eaeca71`)
- `docs/meshing/structured-hex-surface-inlet.org` (SHA-256 prefix `8686d8096b6f`)
- `docs/meshing/layered-prism-central-feed.org` (SHA-256 prefix `474276726098`)
- `docs/meshing/topology-and-quality.org` (SHA-256 prefix `8a2d70ecc808`)
- `docs/interchange/index.org` (SHA-256 prefix `9c62b237a5e8`)
- `docs/interchange/canonical-source.org` (SHA-256 prefix `f3d27d743876`)
- `docs/interchange/discrete-model.org` (SHA-256 prefix `72d05945fa3c`)
- `docs/interchange/static-topology-and-geometry.org` (SHA-256 prefix `bd8ae9e44a8c`)
- `docs/interchange/round-trip-validation.org` (SHA-256 prefix `383de98cdbb8`)
- `docs/interchange/formats-and-cgns.org` (SHA-256 prefix `ee0da5cb0d93`)
- `docs/interchange/fluent-import-audit.org` (SHA-256 prefix `52206b39c76a`)
- `docs/interchange/operations-and-reports.org` (SHA-256 prefix `83b5ef005a4b`)
- `docs/operations/index.org` (SHA-256 prefix `aab9c5235458`)
- `docs/operations/code-reading-guide.org` (SHA-256 prefix `8b671ba34726`)
- `docs/operations/running-and-testing.org` (SHA-256 prefix `430adb4ca50d`)
- `docs/operations/outputs-and-reports.org` (SHA-256 prefix `2714545dd598`)
- `docs/operations/visualization.org` (SHA-256 prefix `befb1d93fd7e`)
- `docs/operations/openfoam-handoff.org` (SHA-256 prefix `7566391eb517`)
- `docs/concepts/glossary.org` (SHA-256 prefix `3dc0bc6195e3`)
- `docs/references/index.org` (SHA-256 prefix `9e38a073f472`)
- `docs/references/literature.org` (SHA-256 prefix `ac8b12c38735`)
- `docs/references/software.org` (SHA-256 prefix `53935e5d0a6b`)
- `docs/references/provenance.org` (SHA-256 prefix `dcc00d2d9492`)
- `docs/contributing.org` (SHA-256 prefix `10df78424638`)
- `docs/handoff/00-index.org` (SHA-256 prefix `46602702b5b6`)
- `docs/handoff/01-openfoam-build.org` (SHA-256 prefix `d48ee264aace`)
- `docs/handoff/02-mesh-import.org` (SHA-256 prefix `4b85b31b67ce`)
- `docs/handoff/03-zero-flow-proof.org` (SHA-256 prefix `b0e894979395`)
- `docs/handoff/04-paper-inputs.org` (SHA-256 prefix `738e04c669bf`)
- `docs/handoff/05-single-phase-results.org` (SHA-256 prefix `0c51f5699cc8`)
- `docs/handoff/06-cavitation-model.org` (SHA-256 prefix `93d67e5f49c6`)
- `docs/handoff/07-cavitation-results.org` (SHA-256 prefix `a8e0ff55e72e`)
- `docs/handoff/08-replay-commands.org` (SHA-256 prefix `955d8bbe3c37`)
- `docs/handoff/09-artifact-map.org` (SHA-256 prefix `3275fd6808c3`)
- `docs/handoff/10-problems-and-next.org` (SHA-256 prefix `e2c211498175`)
- `docs/handoff/11-pure-hydrodynamic-ramp.org` (SHA-256 prefix `d343beabcf2d`)
- `docs/handoff/12-five-rpm-jump-diagnosis.org` (SHA-256 prefix `e144235e773c`)
- `docs/handoff/13-guarded-2000rpm-ramp.org` (SHA-256 prefix `3dd360d75162`)
- `docs/handoff/14-native-jfo-candidate.org` (SHA-256 prefix `185a4c698a85`)
- `docs/handoff/15-jfo-validation-and-fluent.org` (SHA-256 prefix `328e4b9a4f66`)